سلسلة «الكتاب المهندسي العملي» ▪ الطبعة الأولى ▪ 2026

OpenCV 5 والذكاء الاصطناعي — من البكسل إلى الفهم الحاسوبي

دليل عملي شامل بالأسلوب السردي القصصي — من الصفر إلى الاحتراف

كلّ فصل قصة — كلّ مفهوم سؤال — كلّ كود مثال محلول. **OpenCV 5+ / Python 3.13–3.14**

م/أحمد صالح المغز

OpenCV 5.0 | Python 3.13–3.14 | رؤية حاسوبية | ذكاء اصطناعي

جدول

جدول المحتويات

أقسام الكتاب وصفحات بدايتها

مقدمة الكتاب: كيف تقرأ هذا الكتاب؟ 1

البيئة والأساس: ثورة OpenCV 5 في يد المهندس 6

الصورة الرقمية من الداخل: البكسلات والقنوات والعمق 21

الأدوات المصرفية للصور: القراءة، الكتابة، العرض، والفيديو 32

العمليات على البكسل: التسوية الديناميكية والحسابات النقطية 42

الترشيح والالتفاف: المرشحات، التنعيم، وحدود النظام 52

التدرجات ومرشحات الحواف: من Sobel إلى Canny 61

التحولات الهندسية: التدوير، التشوه، وإعادة الاستمثال 70

العتبات والتقسيم: من الإضاءة إلى القرار 79

المورفولوجيا الرياضية: الأشكال والبنكسات 88

الكائنات المتصلة والكونتورات: قراءة الأشكال 96

التقسيم المتقدم: GrabCutg Watershed والتجميع 105

الرسم والتراكب والنصوص: توثيق المخرجات 113

مطابقة القوالب: إيجاد الجسم المعروف 122

تحويل هوف: الخطوط والدوائر والقياسات 130

الخصائص ومعايرة الكاميرا: من الاختصاصات إلى العالم ثلاثي الأبعاد 139

تعلم الآلة الكلاسيكي: cv2.ml من KNN إلى SVM 150

التعلم العميق: cv2.dnn وONNX وYOLO 158

التتبع وطرح الخلفية: من التدفق الضوئي إلى التتبّع الحيّ 166

الأداء والتحسين: من بايثون البطيء إلى خط إنتاج حيّ 174

الملحق: المرجع السريع، الإجابات، ومسرد المصطلحات 182

مقدمة

مقدمة الكتاب: كيف تقرأ هذا الكتاب؟

> «كل أمر كبير يبدأ بسؤال صغير يُطرَق باب الفهم» — مثلٌ يذوينه هذا الكتاب

مقدمة الكتاب: كيف تقرأ هذا الكتاب؟

0.1 لماذا وُلد هذا الكتاب؟

بدأت قصة هذا الكتاب في ليلةٍ متأخرة حول طاولة مهندسين: مهندس متحكمات من مصنع تعليب، ومطوّرة تطبيقات طبية، وثالث يعمل في منظومة مراقبة. كلهم وصلوا إلى الجملة نفسها: «الكتب مقتبسة أو مثقلة بالمفردات، والدروس المتناثرة لا تبني مهندسًا... ونحن لدينا أنظمة حقيقية تنتظر». هذا السؤال هو ذرّة هذا الكتاب: مرجع عربي واحد متكامل بأسلوب ساحر — يأخذك من *«ما هي الـ OpenCV؟»* إلى *«سأبني بها منظومة أداءٍ وكشفٍ هندسيًا متماسكًا»*، ومن *عدادٍ بسيط لعناصر في صورة* إلى *رؤيةٍ متكاملة تعمل على فيديو حي*.

الكتب الإنجليزية قوية — لكن شجرة الفهم تحتاج جذورًا في لغة تفكرك بها؛ والرؤية الحاسوبية من التخصصات التي تُفلِت من «حافظ الكلمات» وتبقى مع من يفسر ويسأل. أردت هنا نص عربي يُقرأ ككتاب كُتب لا كترميم لترجمة.

ما الذي ستصبح عليه في نهاية هذا الكتاب؟ قائمة وعدٍ أرسمها لك منذ البداية:

- **تفهم** كيف يُترجَم العالم البصري إلى أرقام داخل الحاسوب: البكسل، القناة، عمق البت — ولماذا تُشكّل المصفوفات بذاكرة متتالية على أنحاء الذاكرة.
- **تترجم** أي مفهوم رياضي في الرؤية إلى كود Python سريع ونظيف على أي نظام / Windows Linux / Mac.
- **تصمم** منظومة عالمية من (مشكلة → ميزات → خوارزمية → أداء) بكفاءة وقابلية صيانة.
- **تجنب** الأخطاء الشائعة التي أجمعتها على مدار سنين من المشاريع الميدانية، وجلست معها في جلسات تصحيح لا تنموي.

0.2 لمن هذا الكتاب؟

- **مطوّر برمجيات** يريد إضافة الرؤية الحاسوبية إلى صندوق أدواته بجدية حقيقية.
- **طالب** في علوم الحساب / الإلكترونيات / البيانات يريد موردًا يتكلم لغته وبأمثلة تحمل روحه الفكرية.
- **مهندس ذكاء اصطناعي** يعمل بـ PyTorch أو TensorFlow لكن «الخط الأمامي» — الصور القادمة من الكاميرا — لا يزال بوابة أساسية تتحكم في نجاح أو فشل نموذجك.
- **مهندس تشغيل** في مصنع أو لوجستية أو أمن: من يريد أرقامًا حقيقية من الصور لا من التقارير.

لا سابقة بالرؤية المطلوبة؛ خبرة مبتدئة ببرمجة Python (متغيرات، دوال، حلقات) تكفي تمامًا، وسأبني سويك من البكسل الأول.

0.3 الفلسفة التربوية: أربعة قوانين نفسية

بنيت الفصول — والأوراق التي جرت القهوة فوقها — على أربعة قوانين تعلّم أؤمن أنها تجعل الفهم دائمًا:

1. **القصة قبل الشرح** — عقلك يحفظ مشاهدًا لا معادلات. يفتتح كل فصل بسردية من الواقع (محادثة هاتفية، لقطات مصنع، رسالة تنبيه من نظام إنتاج) تشعل في نفوسك السؤال: لماذا نتعلم هذه الخوارزمية اليوم؟ ثم نجيب عبر الشرح.
2. **السؤال هو التلامس** — سأستعمل صيغة استفهامية «لماذا؟» «ماذا لو؟» «كيف؟» في كل مواضع الحجة الحاكمة: للحظة تعلُمك الحقيقي صدق بالسؤال الذي تسائلها في عينك.
3. **المعادلة ثم الكود ثم الصورة** — كل مفهوم يمر على ثلاث ميادين: المعادلة الرياضية (مكتوبة بـ LaTeX واحترام رياضة الكتاب)، ثم الكود في Python 3.13/3.14 (قابل للنسخ والتنفيذ مباشرة)، ثم النتيجة البصرية (الإنشاء المحلي الذي يظهر في الشكل).
4. **الخطأ جزء من الخريطة** — كل خوارزمية تُقاس بطريقة فشلها. لذلك يختم كل فصل بحصر للأخطاء الشائعة، و«تنبيه» و«نصيحة» من مهندس ميداني. الخبرة الحقيقية تُبنى فوق بقايا الأخطاء.

0.4 خريطة الكتاب: الأقسام السبعة

تمتد الرحلة عبر سبعة أقسام متدرجة — من البيئة الأولى إلى أداء الإنتاج — بين المدارات التي تسلكها عين الكاميرا وذكاء النموذج:

#	القسم	الفصول	ماذا ستتقن؟
1	التأسيس	1–2	البيئة، إصدار OpenCV 5، الصورة كـ مصفوفة، البكسل، القنوات، عمق الفتحة
2	الوسائط	3–4	القراءة والكتابة والعرض والفيديو الحي، العمليات على البكسل والأقنعة
3	المرشحات والحسابات	5–7	الالتفاف Convolution، التنعيم، السبورات والحواف Canny، التحويلات الهندسية
4	التحليل البصري	8–10	العتبات والتقسيم، المورفولوجيا، الكونتورات وحلقات الحساب الحية
5	الملامح والكاميرات	11–14	الرسم، القوالب، المطابقة، الخصائص (SIFT/ORB)، Hough، معايرة الكاميرا والاستثناءات
6	الذكاء المرجعي	15–18	تعلم الآلة التقليدية cv2.ml، الشبكات العميقة عبر DNN/ONNX/YOLO، التتبع ومعرفة الحركة
7	الإتقان والأداء	19 +	تحسين الأداء، الحوسبة المتوازية، المُلحق: مرشّد سريع + مصطلحات + إجابات تمارين

كل فصل يمكن قراءته باستقلالية مع فهرس المحطة المتوفرة في بداية الكتاب؛ وأنا أُوصي بطريقة العد التسلسلي في المرة الأولى، لأن مسار التفاهم واحد.

0.5 الأدوات التي ستحتاجها

الأداة	الإصدار الموصى به	الدور
Python	3.13 أو 3.14	لغة المعالجة
OpenCV-Python	x.5.0.0	مكتبة الرؤية الحاسوبية
NumPy	x.2	المصفوفات والعمليات الرقمية
Matplotlib	+3.8	رسم النتائج والمخططات

إن لم تدمج `pip` من قبل لا قلق: الفصل الأول يشرح التنصيب من الصفر ويوجه كل شيء.

0.6 طريقة فعّالة بالقراءة: طبق ذهب

- **نفّذ بنفسك**: كل مثال في هذا الكتاب كُتب ليعمل على جهازك كما هو؛ جهّز البيئة وأطلق الأمثلة ثم انظر نتيجتها قبل المتابعة.
- **غيّر، ثم غيّر، ثم غيّر**: كل مثال عليه «مقياس تجريب» — غيّر قيمة واحدة، نفّذ، لاحظ الاختلاف، سجّله. هذه هي أهم عضلة «تطوير هندسي» تبنيها عقلك.
- **تحدَّ نفسك**: في نهاية كل فصل تمارين على ثلاث طبقات (فهم / تطبيق / تحدٍ) — حُلولها النموذجية في الملحق.
- **اعتنِ بالملخصات**: كل فصل يختم بـ «ملخص 60 ثانية» ستُعيد بينك وبين ضوء الفهم قبل الفصول التالية.

الآن، لنجلس على الطرف الساخن للفضاء الوظيفي... إلى **الفصل الأول**: ثورة OpenCV 5 — تقنية بدأت رحلتها من مختبرات Intel بداية الألفية، وحالها أن حملة مبرمجي Python أنجحت أشهر أُفتية للصور في العالم: مفتوحة المصدر، وقوية، وتخدم الآن أنظمة تمر من عين ساعة ذكية إلى مركبة ذاتية القيادة. معنا اليوم إصدارها الخامس — في انتظارك أن تجري به كاملة التفاصيل.

نصيحة من المؤلف

ابدأ من الفصل الأول حتى لو كانت خبرتك كبيرة: إصدار OpenCV 5.x يطرح لمسات مثيرة وكثيرًا من التحسين الداخلي مقارنةً بـ x.4 — ونحن ننطلق من السياق الحديث مباشرة؛ لن تجد شرحًا ثابتًا إلا إذا نظرت إليه بعين x.5.

الفصل 1

البيئة والأساس: ثورة OpenCV 5 في يد المهندس

0.1 قصة الفصل: أول يوم في وظيفة لا تسامح

- **قصة من الواقع**

الساعة 9:01 صباح الاثنين. **عُمر** يقف أمام مكتبه الجديد في شركة «نُور للخدمات الصناعية» — وعلى الطاولة لابتوب لم يُعدّ بعد، ورسالة بريد إلكتروني واحدة من سارة، مديرة التشغيل:

> «مرحبًا عُمر، لقطة الأنابيب من الكاميرا رقم 7 تعطينا مؤشرات تهديدات مسبقة. قبل الظهر نريد أن نرى **إطارًا واحدًا على الشاشة** — لا شيء أكثر. يمكن الوثوق بك؟»

ثلاثون دقيقة فقط. في حوزة **عُمر**: لابتوب فارغ من المنصات، وبنية برمجية لم تُنصب بعد، وكاميرا معلقة فوق خط إنتاج لا تفهم لغة الطلبات. لكن سارة صممت فورًا ما تحتاجه بالضبط: لا تحليل، لا تقارير — فقط **إطار واحد يظهر على الشاشة**. وهذه أول خطوة يقطعها كل مهندس رؤية في حياته، وهي ليست بالساذجة كما تظن.

هذا كتابك؛ وكل فصل مقصود به أن يكون **درسًا واحدًا** يأخذك من سؤال محير إلى جواب يعمل. الفصل الأول ستحصل فيه على ما أخسره عُمر: **بيئة عمل نظيفة بنسخة OpenCV 5**، وفهم صحيح لبعض «الأين» و«ماذا» قبل أن نجري أول برنامج.

1.1. أين تخصص الرؤية الحاسوبية في طريقنا؟

سؤال: حين نقول «رؤية حاسوبية»، ما الذي نحتاجه بالضبط من الحاسوب؟

عينك تميّز تفاصيل وجه صديقك داخل غرفة خافتة الضوء، لكن الحاسوب لا يفهم إلا الأرقام. ما يصل إليه من الكاميرا هو: **عناصر حساسة للضوء تجمع الإضاءة وتحوّلها إلى قيم رقمية — ثم تُنقل هذه القيم إلى الحاسوب كـ مصفوفة**. كل ما سنعمل عليه في هذا الكتاب هو الاقتراب من هذه المصفوفة: **أرقام (عادة 8 بت لكل عنصر)** تحمل شدة الإضاءة والألوان.

كل ما يتبع في هذا الكتاب يمشي على هذا الفهم: نلعب على أرقام منظمة — نُعدّلها، ونُصنّفها، ونستنبط منها **كل غرض: قياسًا، كشفًا، تصنيفًا، تتبعًا، أو قرارًا فاعلًا في نظام حيّ.**

```
 import cv2
import numpy as np

# مصفوفة 5×5 من قيم 8 بت – أصغر «صورة» يمكن تحليلها!
img = np.array(
    [[10,  20,  30,  40,  50],
     [15,  25,  35,  45,  55],
     [30,  40,  50,  60,  70],
     [45,  55,  65,  75,  85],
     [60,  70,  80,  90,  100]], dtype=np.uint8)

print(img.shape, img.dtype, img.min(), img.max())
# (5, 5) uint8 10 100
```

إذا التقطتَ الصورة الآن، فهذا هو الحجم الصحيح المسحوب: مصفوفة `uint8` ضمن النطاق `[0,255]` — الصور التي قرأتها من قرصك هي بالضبط هذا الشيء، ببعد أعرق. **ولّد الرؤيا الحاسوبية هنا.**

2. 2. تاريخ موجز: كيف وصلنا من Intel إلى OpenCV 5؟

البوابة	حدثين هندسين
1999	مشروع Google في Intel: مكتبة «OpenCV» للمباحث الحاسوبي، بدأ بـ C.
2006	الإصدار 1.0.
2009	الإصدار 2.0 ودعم C++ كامل
2018–2016	إصدار x.3 أضاف عمليات معايرة تحسين، خوارزمية xfeatures2d، ونظام G-API لاحقًا
2024–2020	الإصدار x.4: واجهات بيثون جميلة + DNN التحليل العميق + OpenVINO
+2025	الإصدار x.5: نظرة حديثة، C++17/20، تحديث DNN، وإزالة واجهات قديمة متخلفة

> **معلومة مصيرية**: OpenCV الأكثر تثبيتًا بين مكتبات الرؤية في العالم. من الساعة التي يشغّل فيها المتدرب أول `pip install` يبلش اقتصاد مناسب: أدوات جاهزة تعمل في الدقيقة، لا في الأسبوع.

على هامش الإصدار: في هذا الكتاب نستخدم **OpenCV 5.0.0.93** — وهو إصدار المبرمجين الجديّين؛ لأن تطبيقات الأداء الحقيقية تحتاج دقة التوقيت الحديثة.

2.2 مقارنة إصدار x.4 وx.5: ما الذي تغيّر فعليًا؟

الجانب	OpenCV 4.x	OpenCV 5.x
قاعدة اللغة	C++11	**C++17/20** (أداء أفضل + دعم حديث)
واجهة بايثون	قديمة قسّيها x.3	أكثر نظافة وتوافق نوعي بـ NumPy 2
DNN	قاعدة دعم توكير و OnNX	مراجعة جوهرية مع رحت support + مهنة best على ONNX 18
بعض الواجهات القديمة (C-API)	بعيدة عن الاعتماد الحالي	**أزيلت كليًا** — لم نعد نرثها
الأداء الخدمي	ممتاز	ذهبيًّا من مستويات متعددة

لماذا يهمك ذلك؟ لأن كتابة كود x.5 حصرًا يعني أنك في إطار década حديث، تفي بعصر تكنولوجيات المتكن. النسخ القديمة تحتاج ترحيلًا معظمًا؛ هذا الكتاب يعلمك على الإصدار الاحترافي مباشرة.

⚠ **خطأ شائع**: أنك تنسخ من كتب قديمة استدعاءات مثل `cv2.cv2` أو `cv2.cv` أو `cv2.CV_*` — فقد فIZES يتجه لنسخة OpenCV حديثة فقط.

3. 3. تثبيت البيئة: ثلاثة مسارات تحترفها بتجرّب

3.3 المسار 1 (المفعول الكامل): Python + venv

```
 # 1) تحقق من بايثون
python --version        # 3.13.x 3.14 أو.x

# 2) أنشئ بيئة افتراضية معزولة للمشروع
python -m venv .venv

# 3) شغّل البيئة
# Windows PowerShell:
.\.venv\Scripts\Activate.ps1
# Linux / macOS:
source .venv/bin/activate

# 4) ثبّت المكتبات
pip install opencv-python==5.0.0.93 numpy matplotlib
```

لماذا بيئة افتراضية تحديدًا؟ لأن OpenCV تعتمد على نسخة دقيقة من NumPy ومن التكوين الميكانيكي؛ المعامل الأعلى في المشروع يحقق أيسر حلول لحماية البرمجيات. ويمكنك الممثلة بكل فارق من الأجهزة على كل ما نسخة معينة.

3.4 المسار 2: Conda

```
conda create -n opencv5 python=3.13 -y
conda activate opencv5
pip install opencv-python==5.0.0.93
```

3.5 المسار 3 (للمحاوس السريع): Google Colab

```
!pip install -q opencv-python==5.0.0.93
```

3.6 تحقق فوري أن كل شيء يلهث

```
import cv2
print(cv2.__version__)           # 5.0.0
print(cv2.cuda.getCUDADeviceCount())  # 0 إذا لم تكن هناك بطاقة (وبيئتنا تعمل بدون GPU)
```

⚠ **خطأ شائع**: تثبيت الحزم بالاستدعاء `pip install opencv` من غير ذلك للخلف. حزم OpenCV المهنية هي:

- `opencv-python` : الحزمة الرئيسة.
- `opencv-contrib-python` : المكتبة الأساسية + إضافات, `xfeatures2d` , `ximgproc` , `aruco` , `tracking` ...
- **لا تثبتِ الحزمتين معًا**: يتصادم الوصول لذاكرة الدوال (والخطأ الشائع أن يُنصح بذلك بدون خلف تلافيفي).

4. 4. جولة داخل أعضاء OpenCV: ما بنيته القدرة؟

صورة OpenCV في مرآة من **الوحدات (Modules)** — كل وحدة «وزارة» مسؤولية واضحة، وتضم مياهاً من الخوارزميات:

الوحدة	ماذا تفعل	أمثلة دوال
`cv2.core`	بنية البيانات الأساسية، العمليات العددية والعمومية	`Mat`, `split`, `merge`, `reduce`
`cv2.imgproc`	معالجة الصور: مرشحات، حواف، تحويلات لونية، مورفولوجيا	`cvtColor`, `GaussianBlur`, `Canny`
`cv2.imgcodecs`	قراءة/كتابة ملفات الصور مع الكودات	`imread`, `imwrite`, `imencode`
`cv2.highgui`	ترابط الواجهات الرسومية (النوافذ) وعرض/إدخال	`imshow`, `waitKey`, `setMouseCallback`
`cv2.videoio`	الفيديو والكاميرات، أعمال التسجيل	`VideoCapture`, `VideoWriter`
`cv2.video`	التحليل الزمني: تتبع الكائنات، فصل الخلفية، مسارات الحركة	`createBackgroundSubtractorMOG2`, `CamShift`
`cv2.calib3d`	الكاميرات والمكان ثلاثي الأبعاد: معايرة، stereovision، pose	`calibrateCamera`, `findHomography`
`cv2.features2d`	ملامح الصورة: نقاط مميزة، أوصاف، مطابقة	`SIFT_create`, `ORB_create`, `BFMatcher`
`cv2.objdetect`	كشف الكائنات بالمرشحيات / رؤى الشبكات القديمة والأنماط	`CascadeClassifier`, `HOGDescriptor`
`cv2.ml`	تعلم الآلة التقليدي	`SVM_create`, `KNearest_create`, `DTrees`

الوحدة	ماذا تفعل	أمثلة دوال
`cv2.dnn`	الشبكات العصبية العميقة (تحميل وتشغيل نماذج جاهزة)	`dnn.readNetFromONNX`, `blobFromImage`
`cv2.photo`	عمليات التصوير المسرحية: إصلاح صحي، توحيد إضاءة	`inpaint`, `colorChange`

في هذا الكتاب سنتواصل مع هذه الوزارات بالترتيب التشغيلي المسلح؛ فلا تقلق — ستتعلمها كلها.

5. 5. أول برنامج عامل: من النافذة إلى المصفوفة ثم إلى الذاكرة

الوقت مانك: يمكنك التشغيل في كانونièresوبدأ. أولاً لقرات القيادات في كل زائية: قد تكون `BGR` لا `RGB` — أهم خطأ في مسير OpenCV. المتعلم الأولي ما فيش ولي كالاجل، وسيرندا بارثي.

```
 import cv2
import numpy as np
import matplotlib.pyplot as plt

# 1) اصنع صورة (أحمر نقي) داخل OpenCV
img = np.zeros((300, 300, 3), dtype=np.uint8)
img[:, :, 2] = 255        # القناة الثالثة في OpenCV هي الأحمر (BGR!)

# 2) حوّل إلى RGB لنعرضها كورتية بحفظ الرياضة
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

plt.imshow(img_rgb)
plt.title('2 – والقناة الفاصلة له مربع أحمر (في BGR)')
plt.axis('off')
plt.show()
```

لماذا يقوم OpenCV بـ BGR؟ تاريخيًا: واجهات Windows صغtoArray الأنظمة المتاحة الحديثة والرؤية (كان الخط لأسفل الذي «دعا BGR» قبل أن يسمي) التصوير الدفقي قبل ما تكامل تنسيقًا يجعل RGB القاعدة. التراث أصبح اتفاقية — ولا تغير لأن معظم الأنظمة الحالية تعتقد BGR عبر الكاميرات في OpenCV.

6. الأناقة الصغيرة: القراءة فاشلة ... والتفاصل

أهم حرفية في مهام العمل: التحقق من النتيجة **قبل استخدامها** — الدالة `imread` ترجع `None` بدون أي إعلان إذا فشل الملف (مسار خاطئ؟ امتدادات؟ أذونات؟):

```
img = cv2.imread("مسارات/مخطوطة/صورة.jpg")    # None إذا فشل!
assert img is not None, "فشل القراءة: تحقق من المسار/الترميز/.TextUtils"
```

والتعامل الحقيقية واحات تستعد كل شيء:

```
def load_image(path):
    img = cv2.imread(path)
    if img is None:
        raise FileNotFoundError(f"لم أتمكن من قراءة الصورة: {path}")
    return img
```

بايثون يسمح «بحلقة حتى تعمل»، لكن مهندس الرؤية لا يقبل السبب الصامت — السلوك الصامت هو ما يميز المتدرب عن المهندس ويميز «الشغف عن الاحتراف».

7. 7. تحدّي عُمر: الأطلس السريع قبل الظهيرة

مشهد قصير لعُمر/سجرة — ولك ذلك في SQL عَقَدُ واحد معلومات صيغة:

```
 import cv2
import time

cap = cv2.VideoCapture(0)          # 0 = الكاميرا الأولى في النظام
if not cap.isOpened():
    raise RuntimeError("لم أتعرف على الكاميرا")

try:
    while True:
        ok, frame = cap.read()
        if not ok:
            break
        cv2.imshow("Frame 7", frame)
        if cv2.waitKey(1) & 0xFF == ord('q'):   # حرف q للخروج
            break
finally:
    cap.release()
    cv2.destroyAllWindows()
```

من هنا قال **له تطبيق سارة** صفحة 1 من الأوراق: الإطار على الشاشة، وتحكمي وفصل صحيح —// شركة محكمة باستحداثاً في الحالات.

8. 8. الأخطاء الشائعة (الصولة الفصلية)

- **الأخطاء الشائعة — الفصل 1**

1. **مسار Windows يحتوي** `\` : في نص بايثون يُفسر حرف `\` حرف تحكم؛ فالمسار `"\temps image.jpg"` يصور عاجزًا! الحل: `r"C:\temps\image.jpg"` أو مائلتين `"C:\\temps\\image.jpg"`.
2. **العائلة خطأ** `cv2.imshow` **على نافذة اللينوكس Headless** (الخوادم): لأنها لا تملك مكتبة للنوافذ. الحل: `<matplotlib>` للعرض المحلي أو حفظ إلى ملف ثم عرض.
3. **النسيان** `cap.release()` بعد الإطلاق عند التعثر — فتُفرّغ الكاميرا أو تبتلث؛ دمو دائماً `finally` في حلقة الحظر.
4. **BGR ليست RGB** — عند استخدام matplotlib، نسيتْ التحويل؛ راجع القسم 5 في هذا الفصل.
5. **الوضع الصحيح الحسنا القديم، عندما تقوم مزلقاً للإكراه المسارات:** العرض المخططات الأوّلي أنه كان `UIC` وقنى: عُمر لاستخدام `autolayout=True`.

9. 9. نصائح من ميدان

- **نصائح محترفي — الفصل 1**

1. سجّل дата الإستثمات في `requirements.txt` وابتعد عن أحدث «بسماعة قاقات الحديث» ما لم يكن عنصر مجال الحدث: `opencv-python` أو `opencv-contrib-python` أو MA.
2. دائمًا ابدأ برنامجك بـ `cv2.setNumThreads(0)` في مشروع مسير لا نم التهاب الفضول غير المفهومة.
3. `Image from cv2.data.haarcascades` رابطًا نعم أميز للأقنية البصرية— وأصحّ Print غير كثيف.
4. إذا قمت بتثبيت بطاقة NVIDIA: الأشعة المثالية لكافين `opencv-python` تمتلك CUDA مياهاً؛ **للاستثناءات بدست CUDA، سترث محمل مكتبة التجمع المصاحب للإستثمار مباشرة من بناء OpenCV مصدر أو track مكتبة ذات إعداد CUDA**. أن تذكر هذه في النسخ المجهولة للعمل خارج الحساب (مثل تجربة الاختراق في الموقع).

10. 10. تمارين الفصل الأول

- **تمرين — مستوى الفهم**

1. ما هي الفرق بين `opencv-python` و `opencv-contrib-python`، ولماذا يجب عليك اختيار أحدهما فقط؟
2. لماذا يعرض OpenCV الصور بـ BGR لا RGB؟ وهل هذا تغيير مقبول في eve أنت تصميم؟
3. ماذا يحدث إذا قمت باستدعاء `cv2.imshow()` على خادم بلا واجهة ورسومية؟

تطبيق — بأي ملموسة

4. اكتب برنامجًا يشغّل صورة من موقعك المتاح؛ وتغيير قيمقناة الزرقاء إلى أقصى تفاوت؛ وحفظ الإطار إلى ملف ثاني وعرضه في matplotlib بعد التحويل لـ RGB.
5. اجعل المتاد باستخدام دوال `cv2.VideoCapture` لمصدر الويب الموثق في حكملتك. اضبط زاوية «ق» للخروج، مع حماية الإطلاق في `finally` .

تحدٍ — لأصحاب الهمم

6. فعند الحاجة، إضف العدّاد على الحريق الحي: كل إطار يزيد العدّاد، والسطر الأحمر المتذبذب في الزوية يصعد كل ثانية. هل العدّاد كما تفهم لاحقًا بيق؟ (يمكنك درمانها عبر `time.time()`).

ملخص 60 ثانية — الفصل 1

- OpenCV = مكتبة رؤية حاسوبية مفتوحة المصدر، والنسخة x.5 إدارة الأداء والدعم (+ CPU C++17) — وهي الأساس.
- الصورة في OpenCV = مصفوفة `numpy` من نوع `uint8` ضمن [0,255] أو 16-bit و32-bit في التطبيقات المتخصصة.
- قراءة الصور في BGR: تحويل لـ RGB عند استعمال matplotlib ضروري.
- OpenCV تُبنى على وحدات! الصورة تصف والعرض داخل الوحدة المسؤولة (.../core/imgproc/imgcodecs/highgui) — أهمها المبتدئة `imread / imshow / waitKey / VideoCapture`.
- `imread` لا يرمى\"> استثناء؛ ترجعدا None إذا فشل — التحقق دائماً.

الفصل 2

الصورة الرقمية من الداخل: البكسلات والقنوات والعمق

0.1 قصة الفصل: فاتورة الصورة

قصة من الواقع

بعد أسبوع من التحدي الأول، احضر سارة إلى مكتب عُمر حاملًا صورة أنابيب أكبر حجمًا وأجمل عينيك: «هذه لقطة من نفس الكاميرا بدقة تحسين — لكن الأنابيب الصيدلية هنا مشوشة في الصورة». قال عُمر: «لطيف — فقط أنن يقبض بالتداعي الثاني مشكلة». ساره ساكتة للحظة ثم قالت: «هذه الصورة تُظهر خطأ الحساب الرقيقة، لكن SYSTEM غير interactive في تقنيتي، وحالاتَ الصورة تغيّرت قليلًا، ولماذا؟»

في هذه اللحظة فهم عُمر أن الفهم **العميق** للصورة الرقمية يبدأ بفهم **ما نحوها**: البكسل، القناة، والعمق.



شكل 1: فصل قنوات اللون كما تقرأها OpenCV بترتيب BGR

1.1. ماذا في الصورة الرقمية؟

سؤال: لماذا `img[100, 50]` في الصورة لا تعكس مواقعها في العالم الحقيقي؟

عنصر في sampled الرقمية هو **البكسل** — أصغر جزء من الصورة. وتبدأ المشكلة في أن مُست «**وقتك مع البكسل**»... الصورة الرقمية عبارة عناصر تنازليّة: كلupyter المصفوفة السالبة والأعمدة الس. وتعرض الرقمنة **بدائرة الشركة NumPy الرسمية**. فلماذا تُعدّ الالعذي سلبيًا؟

وكل هذا في الحقيقة مشكلة التمثيل: نطاق القيمة `[0,255]` أو `[0,65535]` — والقيم تُعرف كـ «طاقم النقل الضوئي».MIN.glro كثيرة العناصر على الحاسوب، ونسبة إضاءة الفداء أو «مكونات متساوية»؛ لكن ذلك لأنها منطق *توحيد محلي* وليس طاقمًا للمجال.

```
 import cv2
import numpy as np

img = cv2.imread("assets/samples/lena.jpg")
print(img.shape)     # (376, 500, 3)  – (الصفوف، الأعمدة، القنوات)
print(img.dtype)     # uint8

# وصول واحد إلى بكسل
print(img[100, 50])      # [B G R] – 50 عمود ،100 صف
# سطور الدلائل: أول 5 بكسلات
print(img[0, 0:5])
```

> **بند ذهبي للمبتدئين**: shape في OpenCV يعود: **(الارتفاع، العرض، القنوات)**؛ وبعد التفريغ في الأنظمة المنطقية — ستفارق بين العينings التي وضعها المستخدم والعد الذي تريب مُرضِيًا. استعن الفهم من ذاهبك في القراءة.

2. 2. القنوات: BGR وRGB وإخواتها

سؤال: لماذا يظهر تكسّس في الألوان عند الرسل أحمر؟

في OpenCV **ترتيب القنوات الأصلية هو BGR** (الأزرق ثم الأخضر ثم الأحمر) — والذي سيئ:size تلاحظ في العرض الأجنبية. عند استخدام OpenCV في بيئة Python، استخدم `cvtColor` مع السطر: `COLOR_BGR2RGB`.

```
 b, g, r = cv2.split(img)
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

لكن هل يعني ذلك أن RGB «وحدة قياس واحدة» عمرها صحَّ؟ الإجابة: **لا**. فالصورة ليست صورة الألوان حكما كإشارات عرضية؛ كما يلاحظ شاشة تصوير معدنية الأخضر:؟ والقناة الزرقاء المصدرة لرؤية قناة صوره الجميلة ميزة بالأشعة تحت الحمراء.

2.2 جدول تحويلات المساحات اللونية المهمة

المساحة	الفائدة	الدالة
BGR	التمدد قياسي OpenCV + بنفست الكاميرا	`(origin)`
RGB	أجهزة العرض (matplotlib/plt)	`cvtColor(img, COLOR_BGR2RGB)`
GRAY	الحساب الأسراع وكشف الحواف	`cvtColor(img, COLOR_BGR2GRAY)`
HSV	تدرجة الستيريالا (إنذار بدرجات الألوان)	`cvtColor(img, COLOR_BGR2HSV)`
LAB	مقاومة لتغير الإضاءة + تعريفات CIE	`cvtColor(img, COLOR_BGR2Lab)`

لماذا يهمك؟ لأنني سترى منطقة rulers قدم من شدّة استخدام الحنّاخن: فنظام «درجات اللون» في `HSV` يحمل الألوان الرمادية بشفرات أوضح، و `LAB` مفيد للتحكم في المعايرة بين الصور — كلها من الجيل الأول كالتكليف.

2.3 معادلة التحويل إلى الرمادي (Standard *BT.601*)

$$Gray = 0.114B + 0.587G + 0.299R$$

رياضياً، هذا التحويل هو مضلّل بين قنوات الصور وبنايتها: القنوات الحمراء والخضراء والزرقاء تسهم بقيمٍ محددة المعاير — لأن حساسية دفة الرؤية للشدة، الإضاءة (الرسائل) أضعف من حساسية الل αντί. الشكل المضروع هناك يكشف تفاضل الإزدواج.

```
 gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
print(gray.shape, gray.dtype)   # (376, 500) uint8
```

3. 3. عمق البكسل: من bit-1 إلى bit Float-32

سؤال: لماذا يوجد استائة «12 بت» تحتاج المستشفى «16 بت»؟

سؤال: لماذا تسأل عيّنة «12 بت» في الأجهزة الطبية «16 بت»؟ إجابتها أن المستويات القيمية للمعالجة إبراهيمية: الأعداد الصغيرة 8-بت: 256 درجة من الإضاءات. صورة **راديولوجيا** تحتاج مدىً أوسع (12 بت = 4096 درجة)؛ والصور المتتالية الأدنى 16-بت: 65536 درجة. هذا التصوير السبب المعايرة

نوع dtype	النطاق	متى تستخدمه؟
`np.uint8`	0–255	الصور العادية (وحدة العرض والتسجيل)
`np.uint16`	0–65535	صور طبية، أقمار صناعية، انتظار HDR
`np.float32`	غير محدود حدودي	الحساب الرقمي (أجل buffers المتوسطة)، والخوارزميات كالمراسلزات
`np.float64`	دقة أرضية	كل الحساب البايثوني القياسي

4. 4. العلم السريع: البكسل في NumPy vs OpenCV

كل ما في NumPy يمكن استخدامه وحّدًا — لكن مع مراعاة ع.IntegerField قويًا في OpenCV. تكلفة `cv2.split()` و«كل القنوات» سرعتها الجاجة الأtility ممّا تمثّل عبر `numpy` القناية:

```
 # الفرق الجوهري: NumPy FCT عند أن الأقنعة بطريقة:
b2 = img[:, :, 0]          # slicing – عرض (view) على الذاكرة الأصلية
cvSplit_b = cv2.split(img) # نسخة جديدة في ذاكرة مستقلة

# لكن قواعد النظام:
img[100, 50, 0] = 255      # تعديل مباشر في الصورة نفسها (view)
```

نظرية التذاكر: `slicing` NumPy يعرض **مقعً لا نسخة** — تعديل الأصلى ينقل التغير في كل الاستخدامات، والأولوي تُكسب لعمري الإنشاء — لكن الحساب الرقمي بهذا الشعوع الحقيقة حتى المجال المستخدم يعوض استراتيجيًا.

5. 5. الفائض والسلوك العددي: عندما يضع "255 + 1 = 0" نذرك

سؤال: لماذا `255 + 1` بالعربي يعطي «0»؟

في uint8 والقيم عبر حالف 255 تصبح صغيرة بتجعة (إنها تتجاوز الحد المسموح من قيمة درجات البكسل الإألوه: السبب تحتue: المعالج يجمع بسجل 8 بت، والقيمة تُخفي فوق 255 بعد إن سجل أخذ آخر جزء من الذاكرة ففقدت «الذرة القديمة»). وهذا سلوكحق. عند الحاجة فيها الطاقة (فتح العين آمنة الضيّـه): استعمل `astype(np.int16)` أو `np.clip(...)` أو دوال OpenCV (cv2.add سلو:Intell).

```
 a = np.array([250, 254, 255], dtype=np.uint8)
print(a + np.uint8(5))   # [255 255   4]  — 255+5 4 ذنت (تدعيم oversaturate)

print(cv2.add(np.uint8([[250, 254, 255]]), np.uint8([[5, 5, 5]])))
# [[255 255 255]] — الضرب (سلوك السلوك) كتصنيع الحلقة الواقعة للعلّي
```

الخلاصة: عند الحساب العددي على صور مع مرور القنوات:

1. تحول إلى أنواع متوسط (`float32` / `int16`).
2. عدم تجاوز القدرة الواجب بعد الإعداد: `np.clip(arr, 0, 255).astype(np.uint8)` .
3. أنهي استخدام دوال OpenCV (presets المرتفعة) لتستخدم , `cv2.add` , `cv2.subtract` `cv2.divide` , `cv2.multiply` .

6. ROI .6: أوراق الصورة كأوراق مكتب

سؤال: كيف تستخرج «جزء من الصورة» وترسله إلى التحليل مع السرعة كاملة؟

ROI (Region of Interest — منطقة الاهتمام) هي إجابة OpenCV على الخصاشة: استخرج الوحدة مع فهرسة NumPy، وكل عملية ستعمل بسرعة منطقية (F:\ عناصر الإ-CLA تpendicular الجزئية):

```
 h, w = img.shape[:2]
rye = img[100:250, 150:300]      # ROI = [صف :صف]، [عمود :عمود]
rye[:] = 0                       # (عرض مباشر) إزالة الألوان من القسم

# (كريب كريما على أقل هيئة ليد سيرا) إجراء مضاعف:
small = img[hp//2-100:hp//2+100, wp//2-100:wp//2+100]
```

6.4 مقارنة: NumPy Slicing vs OpenCV crop

الطريقة	الميزة	العيوبة
NumPy `img[y1:y2, x1:x2]`	سريعة، بلا نسخ (view)	تتطلب `copy()` عند البناء بذريعة جديدة
`cv2.resize` على جزء من الصورة	إعادة قياس فورًا	نسخة دائماً
`cv2.selectROI`	تحديد بشاشة تفاعلية	تحتاج واجهة رسومية (لا تعمل في ملف مواز لكل المنصات)

```
roi = img[100:250, 150:300]
roi_copy = np.copy(roi)      # نسخة مستقلة (untouched)
```

⚡ **مبدأ مهندسي**: «إذا كنت تنوي تعديل داخليًا بعد استخراج ROI، فاستخدم `np.copy()`». هذه الجملة ستفيدك من أذيّة تسريلاً كثير من برامجك.

7. 7. أسلوب الخريطة: عرض القنوات في «شرائح بصرية»

عندما تجري فciąż القنوات داخليًا — لماذا البصر الجذري منبر للفه، والعيون ترسم في matplotlib كصور رمادية. أنه самая معلومة للفه الحسابي:

```
 import matplotlib.pyplot as plt

b, g, r = cv2.split(img)
fig, axes = plt.subplots(1, 3, figsize=(12, 4))
for ax, (title, ch) in zip(axes, [('الأزرق B', b), ('الأخضر G', g), ('الأحمر
R', r)]):
    ax.imshow(ch, cmap='gray')
    ax.set_title(title); ax.axis('off')
plt.tight_layout(); plt.show()
```

في اين الطرق سترة نسبة دلالة في أسلاك التصوّر: السماح لنفسي بالأنظمة الرديف البذر / عسكري السفن. القنوات «جهاز إلكتروني» — كما أنك تجهاز الألوان من خلال أي إشارات الرسentities الدقيقة الحاملة داخل تعاريد.

8. 8. الحالة المرجعية: مقزز لتتميم إمكانات الصورة (في Color-vs-Gray)

عندما تستعلم «أي مساحة اللون تحل المشكلتي؟»، مع الأخذ بعين الاعتبار أن الأسار «العين البشرية أطح حواس في النهر أبثها orbitally» في الصورة الرقمية: التصميمakes ليس الآمن الدائق. ساختبَ قيمة دالة بسيطة لتحويل عامل المزج بين اللون والسمياتِ الملموسة (حالة الحجز المفاجئة والإعلانات اللوقة):

```
 def blend_gray_and_color(img, alpha: float = 0.5):
    """كنمزج دائم: 0 = رمادي بالكامل، 1 = اللون الكامل يستخدم alpha=0.5"""
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    gray_bgr = cv2.cvtColor(gray, cv2.COLOR_GRAY2BGR)
    out = cv2.addWeighted(img, alpha, gray_bgr, 1 - alpha, 0)
    return out

out = blend_gray_and_color(img, alpha=0.35)
```

هذا الاسم يفتح طرقًا في كشبنا الأعمال: كل (products visual) تحولات الألوان، وأنت بتحليل كهذه: وهذا مجيتة المقصود في البرنامج = فَهَم السياراتicker حِسرية الصور.

9. 9. الأخطاء الشائعة (صون الفصل الثاني)

- **خطأ شائع — الفصل 2**

1. **ختصاً في مقادير** `img[100, 50]` **أو العد pk[][]_x** — النظام يطبق مب promised (صف، عمود) لا (عمود، صف)؛ عكس التعبير ينقل أي وسون «سببه F» إلى العدئلس فقط، والمشكلة أن التفصيلة منطقاً ما تظهر في المات بعد الميران بها حف اللاشرictionٍ. حنوثا بالأعمدة تبادل بعشبات — لا تصبح.
2. **RSS «filter layer السبب السبعة مثل** `img((x, y))` : الحاوية الشمشير أحمر **(صف، عمود)** لا عكس؛ لا تحلل الوصف في خارطة التصميم. هذا وقت العرة.
3. **التحويلةّآن إلى uint8 عن طريق** `astype` : يُفرض الإشباع عنفًا بمعنى `clip(...)` أولاً: `np.clip(arr, 0, 255).astype(np.uint8)` .
4. **عندما نستخدم cv2.splitMs بدون cv2.merge** بعد استرداد القنوات: النتيجة بلا تفاصيل؛ باء كحد أن cv2.merge((b, g, r)) إلى وضع العرض ضروري.
5. **البضاعة المستخدمة بشيء من الأزواجي القياسات** `img.shape` يظهر**: الإطار ينطلق بـ «شكل الصورة» — استخراج `h, w, ch = img.shape` (أعرًا إجابني).

10. نصائح المهندسين في ميدان

نصائح محترفي — الفصل 2

1. استخدام `np.float32` قبل معالجة الأقنعة السفلى والفترك الرئيسي؛ ثم اتحّد للمخلّص النهائي فقط قبل الحفظ.
2. قيّم الإجابات عمق الفحتة بـ `dir(img)` و `.astype(np.uint8)` في الوقت الفجري حتى تلهد كلا طاقة الكاميرات الحقيقي.
3. عند التثبيت المتعادل، أستدعي `cv2.merge` على القنوات ذات التنسيق المفعل: المعيار العاقل: `np.dstack` أو `cv2.merge` .
4. احفظ نسخة من الصورة الأصلية، وأعمل على النسخ المشتقة — بذا على الغريب أن تغيير واحد فقط يصبح المنائق:
`original = img.copy()` .

11. تمارين الفصل

تمرين — مستوى الفهم

1. ما الفرق بين `img.shape` و `img.dtype` ؟ ومتى يمك في كود الشكل الرأسي ضلر `img[100, 50]` .
2. لماذا يعرض OpenCV القنوات BGR لا RGB؟ وماذا يحدث عند استخدامها مع PIL أو matplotlib؟
3. ماذا يحدث في `a = np.uint8(255); a + 1` ؟ ولماذا؟ وما دالة OpenCV المتوافقة التي تعطي سلوكاً منضبطا؟

تطبيق — بمواد ملموسة

4. «уровень الحي» — ضع صورة على أبعاد `(height, width)`: استورد صورة، اتجز بكسل الرابطة في المصفوفة حتى تصفية اللون المسموي: على نوع `uint8`.
5. صمم دالة `swap_channels(img_bgr)` تلطيف ثالث لبه بتأثيل الصورة على بغ بغ... ثم قابًلا مورفولوجيا مع توب جه الأفعال التالية للب متдипл ender analgesic.

تحدٍ — لأصحاب الهمم

6. «سرقة» اتجاه الصورة: هل img.transpose(1, 0, 2) كافية؟ جرّبها ثم قارنها بـ cv2.rotate(img, cv2.ROTATE_90_CLOCKWISE)، وسنشرح الفرق العملياتي بينهما في فصل الهندسة.

ملخص 60 ثانية — الفصل 2

- الصورة رقمية = مصفوفة numpy: shape = (الارتفاع، العرض، القنوات)، عملية uint8 هي فوق القنال الاعتيادي.
- BGR ترتيب OpenCV؛ القناة الأولى هي الأزرق — تحويل إلى RGB قبل عرضها في matplotlib.
- GRAY تحويل رياضي مقيد: `Y = 0.299 R + 0.587 G + 0.114 B`.
- العمود العمق (bit depth) يقدّر حجم المصفوفات: إليك: 8-بت (صورةٍ أعمل يومية) 16-بت (طبي/instrument) 32-بت float (معالجة الفا).
- عند مفرّقة القناة في الحساب: تحول إلى `float32` أو `clip` + `int16` قبل الدائم إلى `uint8`.
- ROI بـ NumPy slicing بلا نسخ (view)؛ و `np.copy()` لمناخة مستقلة.

الفصل 3

الأدوات المصرفية للصور: القراءة، الكتابة، العرض، والفيديو

0.1 قصة الفصل: المخزن المفقود

- **قصة من الواقع**

اتصل «مدير المشروع في مختبر الأعشاب السينمائي» (أطلقنا عليه الاسم الخفيف: بقسط «حنيض») بخط دائر يتطلب طبقة من طبقة: «لدينا مآخنصة صور فحوص تجارية على مجلدٍ — قام مرافقات прокسيد نور ومنظومة عملية الحسابات أنقذت صورة، لكن تفارقها بقي لدينا، فقط خريودة الحساب خطأ منفصلة...» — والسوق أبرز: «حفظ هذه الصورة تذكر إلى مستوى عمل صار مهم عمل حقيقياً».

هنا وأن للمهندس شكل الدندة مع الصورة بت захват التسجيل وسحبها النهائي: **imread** و**imwrite**، والبروتوكولات كتست الاتصالات كثفت المعادلات. هذه الوحدة ليست «كيف تقرأ ملفًا» فقط؛ بل هي الجسر الذي تحمل فيه *نوايا النظام*: تنسق، تضغط، تدمج، وأدواتها الحديثة الناقلة للإذاعة المستقيمة.

1.1 imread: كيف تقرأ بذكاء لا بحب صف اللحظة؟

سؤال: لماذا يطلق imread طائفًا في المساج؟

عندما تستدعي `cv2.imread(path)` فستحصل على `numpy.ndarray` مباشر (بلا تحويل)، ونقي تحميل أولي وسيط قراءة الملف ثم التمويل والعرض: بولفسنى ما يدعم كل الوظيفة: الرقي الطلبات والتنسيق كثيرة مصاده؛ ولكنها «سالبة» في الأهواء:

```
 import cv2
import numpy as np

PATH = "assets/samples/lena.jpg"
img = cv2.imread(PATH)          # BGR uint8

# أوضاع القراءة المهمة:
img_color = cv2.imread(PATH, cv2.IMREAD_COLOR)        # BGR دائمًا (3 قنوات)
img_gray = cv2.imread(PATH, cv2.IMREAD_GRAYSCALE)     # رمادي مباشرًا — 1 قناة
img_unchanged = cv2.imread(PATH, cv2.IMREAD_UNCHANGED) # مع القنوات الشفافة (PNG alpha) وبلا تحويل

print(img_color.shape, img_gray.shape, img_unchanged.shape)
# (376, 500, 3) (376, 500) (376, 500, 3)
```

متنبه تصعيد: إذا فشل المسار — لن يظهر خط، وترجع `None` فقط. والمرونة المتحفظة هي تعطل البطاقة:

```
 img = cv2.imread(PATH)
assert img is not None, "فشل القراءة — راجع المسار والامتداد"
```

> **أوهدمة الأداء**: تمرام الحمولة على الفيديو معلومة قديمة: imread يجري «قراضة/فعلاً»، لكن حكمه أثر: في إنكر الحachinery الحي (الوغط BGR والنتم الإضافات داخل اللغة).

2. 2. imwrite والضغط: حذار الصفات

سؤال: لماذا حفظ صورة مرتين بحجم مختلف؟

`imwrite` يقسم: **تنسيق الملف** (JPG / PNG / BMP / TIFF...) يدخل في السياق. التنسيقات «الكسّ ία» (مهبطها وضوعية) تُضغط الصورة — والجودة عمل OLAP online:

```
الطبية أو الأطر DockControl ضغط غير قابل للفقد (خسائر الخسائر صفرًا) – حجم كبير، عند :PNG #
cv2.imwrite("out_mask.png", img)

ضغط قابل للفقد – جودة بين 0-95 (القيمة الافتراضية 95) :JPEG #
cv2.imwrite("out_photo.jpg", img, [cv2.IMWRITE_JPEG_QUALITY, 85])

استثناء مع التعاريخ اللوني: IMWRITE_IBM_25 #
```

كثير من المشكلة التنزية الصيدل: «البكسل الشفر خارج النطاق 0-255» مع حفظ يهدف؟ جم تكن الخن ذا؟ مع الجعم المشكلة الحسابية: القيم الصفر الخلف [0,255] المستخدم أحمر.

2.2 جدول مقارنة صيغ الحفظ

الصيغة	نوع الضغط	التعليق	الأفضل لـ
PNG	لا مع تدرج (`ZLIB`)	يرحب تطبيقات الكشف الدقيقة والتنسيقات الأشعة	صور التحليل والتشغيلية
JPEG	قابل لفقد (DCT)	حدًا جودة 0-95	صور الكاميرا الطبيعية
BMP	بلا ضغوط	صفيص	اختبارات سريعة
TIFF	مُسرّع (ضغط LZW)	الصور التعبيرية	التصوير الطبي/HDR

3. 3. المعرض الحي: imshow/waitKey/تمرير القيم

سؤال: لماذا تنفذ صورة OpenCV فقط للنافذة؟ وعند اتجاه نطاقات الصور؟

ظا انتزاع الصور حبر في «عناصر ودمج/سلوم الحياة»، لذا استقر الحنفس:

```
cv2.imshow("face", img)      # نافذة اسمها "face"
cv2.waitKey(0)               # انتظر ضغطة من المستخدم (0 = إلى الأبد)
cv2.destroyAllWindows()      # نظف كل النافذات
```

قواعد النظام الحي:

1. **كل استدعاء `imshow` يجب أن يتعاوب مع `waitKey(ms)`** — وندونها لن يبدأ تحديث الواجهة.
2. `waitKey(0)` ينتظرك حتى تضغوط؛ `waitKey(ms)` ينتظر ms ميكروثانية ثم يعرض القيمة التحيّ:
3. تستفيد من القيم إلى `ord('q')` للخروج من الحلقة:

```
if cv2.waitKey(1) & 0xFF == ord('q'):
    break
```

> ⚠ في بيئات الخلايا (Colab / خادم لاينوكس بلا واجهة) `imshow` تفشل كليًّا — استخدم حفظًق أو matplotlib:

```
import matplotlib.pyplot as plt
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))  # تحويل ضروري!
plt.axis('off'); plt.show()
```

4. 4. خريطة الفيديو: Live & VideoWriter

سؤال: فيديو الحي ≈ فيديو من ملف؟

من حيث المعرفة الأساسية، نعم — كاد المكوين واحد. `cv2.VideoCapture` يعمل مع ثلاث مصائم:

- كاميرا رقم: `0` أو `1` أو المسار ip: `VideoCapture(0)`
- ملف فيديو: `VideoCapture("video.mp4")`
- بث شبكتي: `VideoCapture("rtsp://user:pass@ip/stream")`

```
 cap = cv2.VideoCapture(0)
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 1280)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 720)
cap.set(cv2.CAP_PROP_FPS, 30)

print(cap.isOpened(), cap.get(cv2.CAP_PROP_FPS))
```

والحلقة الأولى كهرمانية للتطبيقات:

```
 try:
    while cap.isOpened():
        ok, frame = cap.read()
        if not ok:
            break
        # (وحدة الإيقاع الأساس): معالجة الفيديو هنا
        cv2.imshow("live", frame)
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break
finally:
    cap.release()
    cv2.destroyAllWindows()
```

4.3 تسجيل الفيديو: VideoWriter والطاقة الخاصة

```
fourcc = cv2.VideoWriter_fourcc(*"mp4v")   # 'XVID' لـ AVI، 'mp4v' لـ MP4
out = cv2.VideoWriter("output.mp4", fourcc, 30.0, (1280, 720))
out.write(frame)
out.release()
```

⚠ **تنبيه تفشي كلاسيكي**: أبعاد الفريم المُمرَّر إلى VideoWriter **يجب أن تطابق تمامًا** الأبعاد المعينة عند إنشاء القائم الحديث — لا تضع شيئًا مستثنى لتركس (width, height) لا (h, w) — والتباين فقط تنتهي في «ملف ابتدائي بلا محتوى».

4.4 خصائص CAP_PROP التي ستستعملها كل أسبوعية

الخاصية	ماذا تعني	ملاحظة
`CAP_PROP_FPS`	معدّل الإطارات	يخصوص عند الكاميرا أو ملف للمعاينة
`CAP_PROP_FRAME_WIDTH/HEIGHT`	أبعاد الإطار	قد يأخذ الأمض على الجهاز لا يعمل هذا
CAP_PROP_POS_MSEC	التوقيت بالأجزاء المنفردة milliseconds	للتنقل الزمني في الملفات الجاهزة
`CAP_PROP_BUFFERSIZE`	حجم المخزن من الإطارات	صغير = حصول من الطريقة الحرية

5. 5. قياس الأداء الحي: عداد FPS كوثيقة الاتساق

عند إجراء العمليات الحقيقة على الفيديو، الحد الأول للتشغيل الحسابي المتصل هو الحوسبة الثانية مع توازن المعالجة لكل إطار (النطاق العملي الحقيقي). الحذاري: قياس «النظام أحسبه كمعدل إطارات»، على عكس

«الخوارزمية حسب كل إطار» — وهنا يظهر فرق «العدد الفضي» في تقييم الأداء المهني.

5.5 خطي مفاهيمي من مهندس محترف: كيف تقيس معدل الإطارات في دورة زمنية؟

```
 import time

# نمط «الأ ثانية واحدة» عند مقارنة المعالجة
last = time.time(); count_frames = 0; fps = 0.0
while cap.isOpened():
    ok, frame = cap.read()
    if not ok: break

    count_frames += 1
    now = time.time()
    if now - last >= 1.0:
        fps = count_frames / (now - last)
        count_frames, last = 0, now

    cv2.putText(frame, f"FPS:{fps:5.1f}", (16, 40),
                cv2.FONT_HERSHEY_SIMPLEX, 1.1, (0, 255, 255), 2)
    cv2.imshow("live", frame)
    if cv2.waitKey(1) & 0xFF == ord('q'): break
```

هذا النموذج بسيط ويقيس **CRF** بدورة ثانية، مخلّص أقل قيد وlc مسرع لأول «عدّاد معيار» عرفناه في هذا الكتاب.

6. الأخطاء الشائعة

● خطأ شائع — الفصل 3

1. `cap.read()` يعيد التاز (ok, frame). **استخدام** `cap.read()[1]` **لل adelante بدون التحقق من** `ok` قد تدخّل «إطار فارغ» إلى معالجتك — قم فحص `ok` أولًا.
2. VideoWriter بأبعاد غير متطابقة مع الكاميرا ← «ملف بلا محتوى» حجمه صفر بايت.
3. `plt.imshow(img)` على BGR: الألوان معكوسة (أحمر يبدل للأزرق) — تحويل.
4. أن «الكاميرا مقفزة» في Windows قد لا يكون خللاً برمجيًا بل إذن الخصوصية الذي يمنع الوصول إلى الكاميرا: راجع إعدادات النظام قبل الخوارق.
5. نهاية الفيديو: cap.read() يعيد (False, None) من أول مرة، لكن المتدرب يظهر أنه يعامل الإطار الخاطئ وكأنه موجود.

7. نصائح الميدان

● نصائح محترفي — الفصل 3

1. لقطات معايرة حديثًا من داخل اللاقطة النهائية + تفعيل `img_rgb` بـ `IMAGE_DISPLAY` في **نوافذ معللات المنطق المتعجل**
2. كنندة نسبة «البسمة مؤقtrumة» — على الشاشات الضمّاً من هيئة «60 صورة/ثانية» مع عملها على اللل الفرعية الحانة. سجّل بكثف عيس؛ المهم لكل شكل عبر شعبة.
3. استخدام الجدول الحي: عند Mumbai، و `waitKey(1)` مِن `10` — النثنائي الاستدعاءات أرسم توسعتها المفصلة للأداء المثالي.
4. وكُن للنظام الكامل على الجهاز المستخدم لـ `QCamera(default=0)`: تلك صفحة الشراء أو المتاح في أحد السائق المشهوف بالنماذج غير الرسمية.

8. 8. تمارين

تمرين — مستوى الفهم

1. لماذا `cv2.imread` ترجع None إذا فشل المسار ولا تُقدم «error»؟ وكذا يُطلب منك من البرمجة جوابًا عمليًا في التطبيق الحقيقي؟
2. ما الفرق بين JPEG وPNG من ناحية «الضغط للفقد المرن»، ومتى تختار أحدهما؟
3. لماذا `waitKey` ضرورة بعد كل `imshow`؟ وماذا يعني معاملها 0 أو 1 أو 30؟

تطبيق

4. «المن عائلة سينوينة»: من ملف فيديو، استخرج إطارًا واحدًا كل 10 إطارات واحفظه بـ PNG. سجل عدد الإطارات والزمن الكامل.
5. «برؤاك «بوياضة» حية — كاميرا ثبت واجتياز قيادة معكورة، بحيث تستدعين إطارًا ثم 512×512 معبر + تخفّض جميع القنات بتكامل و الحد الأس و عرض الإطار الشريطي السطر.

تحدٍ

6. «قياس أداء» على فيديو معرف مسبقًا: قس معدل الإطارات المحقق FPS قبل وبعد المعالجة على نفس مصدر الطاقة الشغالة، وسوّق قياساتك مع جدول المعلومات الآثارية في الفصل 2.

ملخص 60 ثانية — الفصل 3

- `imread(path, flag)` يقرأ BGR دائمًا؛ بينما flag يساعدك: IMREAD_COLOR / IMREAD_GRAYSCALE / IMREAD_UNCHANGED، وتُتاحة مقابل `None` عند خطأ.
- `imwrite` يحفظ مع الشكل، والضغط يعتمد على الصيغة؛ بدقة جودة JPEG = [cv2.IMWRITE_JPEG_QUALITY, 85].
- النظام الحي: `imshow + waitKey + destroyAllWindows` هو حلقة العرض الأساسية، مع استخدام `cap.read()` وأداة مندوبين (ok, frame).
- الفيديو: VideoCapture مع مصادر متعددة (كاميرا/ملف/RTSP)، وVideoWriter بأبعاد متطابقة وال Delay 30 ميكروثانية مع 4CC كّامل.

الفصل 4

العمليات على البكسل: التسوية الديناميكية والحسابات النقطية

0.1 قصة الفصل: مصور الشمس الحارقة

- **قصة من الواقع**

لم يعمل عُمر صباحًا على الأنابيب هذه المرة مباشرة؛ بل أولمن فيه مخطى زوال: صورة «سطح الجن بالشمس الساطعة» ظهرت باهتة إلى أخرواضَ قرن: «التفسير الشائع.. كل حالتنا مظل appartement أي إصطفت مفاشنا». قال عُمر وه says: «اتفقنا هي**العملية العددية** نطرية». سأل البربروب: «إcreateCommandهل يعني التفاعل الاستفاعة»؟

هذا الفصل يعلمك حين الذهب باهتة تعلم على درجات معاينة: البداية «الأدق الملموس»، مونميّ الال Lanama الإمكانات الرأسمalis (منستين أكس الحسابات المستندية أ INNER).

1.1. عمليات النقط: أتمتة القيمة حواليجي

سؤال: ماذا يعني «تعديل سطوع الصورة» هنا؟

العملية النقطية (Point Operation / Pixel-wise) تعني: كل بكسل نخرج قيمة جديدة تكون دالة على **قيمته فقط** — لا الجيران:

$$s = T(r)$$

حيث r القيمة الأصلية و s القيمة الجديدة و T دالة الإرسال. من أشهر دالات في هذه العائلة:

1. **تضمين السطوع:** $r + s = \beta$
2. **تعديل التباين:** $r\alpha \cdot s =$

3. ** différencié المجتمعة:** $s = \alpha r + \beta$ (Brightness/Contrast)
4. **سالقون (Negative):** $s = 255 - r$
5. **التسوية اللوغاريتمية:** $s = c \log(1 + r)$
6. **جوجا (Gamma):** $s = c \cdot r^{\gamma}$

كل هذه تشترج في مسار واحد: أن **لكل قيمة أمامك تعديل بجد واحد**، ولا شيء فيها من «الجيران» أو «الحواف» — لذلك تعتبر «الرجيل الأولي والعدد المضربان» في كل خوارزميات ما.

1.2 المغمضة الميريية (Brightness & Contrast)

```
 import cv2
import numpy as np

img = cv2.imread("assets/samples/lena.jpg")

# المعادلة المطلوبة: out = alpha * img + beta
alpha, beta = 1.3, 40          # تباين أعلى + سطوع أعلى
brighter = cv2.convertScaleAbs(img, alpha=alpha, beta=beta)
```

`convertScaleAbs` تطبق المعادلة ثم **تُقص النتيجة إلى [0,255]** وتconverter إلى `uint8` — حلاً نظيفًا مناخساً للzاحل الإذاعي. بديل NumPy:

```
 out = np.clip(img.astype(np.float32) * alpha + beta, 0,
255).astype(np.uint8)
```

2. 2. جوجا: أصل اللوحة المفصلية إضاءً

سؤال: لماذا الصورة من الكاميرا الشناحية سوداء ساعية..."

السر في العين البشرية: لا تستجيب خطياً للإضاءة؛ إدراك النغمات مجمّع على قيم مضاعِفة داخل نطاق 12-بت إلى 8-بت. العين تحفظ «المشاعر المضيئة» أكثر من «القيم المرقّعة»، وهنا يتحسّن التحجر Gamma لإعادة الرصف الفوري.



شكل 1: منحنى LUT وتأثير قيم gamma على الإضاءة

$$s = 255 \cdot \left(\frac{r}{255}\right)^{\gamma}$$

- $\gamma < 1$: تُعرضية مناطق الظل's (المثل: خشب الأنباور والتفضيل)
- $\gamma > 1$: بياض بشلة ولعومة الأمر

```
 import numpy as np
img = cv2.imread("assets/samples/lena.jpg")

gamma = 0.55       # إظهار المظلّمات = 1 من أقل
look_up = np.array([((i / 255.0) ** gamma) * 255 for i in range(256)],
                   dtype=np.uint8)          # جدول إرسال (LUT)
g_gamma = cv2.LUT(img, look_up)
```

لماذا `cv2.LUT` في العهد؟ جدول إرسال (Lookup Table) من 256 عنصر، والفوز إلى إعادة حساب كل عملية بعد كل بكسل: **دالة موحّدة أصفنيها مرة، ثم تتحول الصورة**

> **فعالًا عبر مسار سريع** — هذا فرق أداء يخوّل في معالجة وكالة فيديو.

3. 3. الأقنعة: خطاب الرسم بشروط

سؤال: كيف أُعدل الصورة بفعل «في هذا الشكل فقط»؟

القناع/القُنّة (Mask) مصفوفة سبقة بـ uint8 من نفس أبعاد الصورة؛ والصفر في العملية حذف، وغير الصفر شمو. والنمط الفني من الأذى الموثّق: «اختر المنطقة ذات التلوين الأسود في الحجز ثم استخدامه».

```
 import numpy as np
import cv2

img = cv2.imread("assets/samples/lena.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, mask = cv2.threshold(gray, 90, 255, cv2.THRESH_BINARY)      # أقوى لم يشرح
العتبة إلا لاحقًا

masked_area = cv2.bitwise_and(img, img, mask=mask)   # الصورة نفسها محذوفة بالمناطق
صفر
```

بطاقة الأعمدة: القناع الموفّر من التقنيات (عتبة، ورث الكشف، منشورة جديدة يدويًا) — والصور الرقمية وهما حمرار بكسل خلال باستاريخ تطبيقات.

3.3 العمليات المنطقية على الصور

Like, والقنوات «الدقة bitwise» في صورنا رقمية:

العملية	الدالة	استخدامها
AND	`cv2.bitwise_and(a, b, mask=c)`	تهربة مشتقة: داخل التقائم معين
OR	`cv2.bitwise_or(a, b)`	تجميع الأقنعة
NOT	`cv2.bitwise_not(a)`	عكس القناع
XOR	`cv2.bitwise_xor(a, b)`	اكتشاف فروق الصور (diff «الستادي»

مثال عملي «الاختلافات للأمثلة»: `cv2.absdiff(frame1, frame2)` تحسب القيمة المطلقة في كل بكسل — وهي العمود من «كاشف الحركة» (skeleton الحركات تفارق «الخلية» إضافة بالأوساط العامة الممكنة):

```
 diff = cv2.absdiff(frame1, frame2)
gray_diff = cv2.cvtColor(diff, cv2.COLOR_BGR2GRAY)
_, motion_mask = cv2.threshold(gray_diff, 25, 255, cv2.THRESH_BINARY)
```

4. 4. الترصي عادل: إحصاء النطاق (Normalization)

سؤال: كيف توحدة صور بانتطور«وأعد قصاصتها وحدها؟

التمثيل الخطي (min-max normalization) يوتر أرabies الصورة بين يمين فقط worldك وات:

$$s = \frac{(r - \min)}{\max - \min} \cdot (s_{max} - s_{min}) + s_{min}$$

وهذا مثال عملي مشترك «مجموعة االعملية المهنية صور مرفوعة من الكاميرا آث عيار الجاهزية» مع تكسب الكتل:

```
def normalize_img(img, lo=0, hi=255):
    f = img.astype(np.float32)
    mn, mx = f.min(), f.max()
    if mx - mn == 0:  # صورة متجانسة قيمها واحدة لا يمكن تمداها؛ نعود صفرًا للحماية
        return np.zeros_like(img, dtype=np.uint8)
    out = (f - mn) / (mx - mn) * (hi - lo) + lo
    return out.astype(np.uint8)

tight = normalize_img(cv2.imread("assets/samples/board.jpg",
cv2.IMREAD_GRAYSCALE))
```

تندو لاحقًا أن «نشر القنبل» (Histogram Equalization) قد هو الاخترع التشريجي للحالات القادمة — لكن لو كان تحت دار التذكير التنظيم.

5. 5. مثال محلول: القناع + Ajánlosg Gamma التسوية

ملخص للفصل بمثال محلول — لن تتوها: عندك صورة منقّحة عمقها الحادي عشر قياساً؛ نحتاج التلوين المسلم مخلّصاً في مسار ثري: **تحويل رياضي + قناع معالج + إظهار لضعف اللون**:

```
 img = cv2.imread("assets/samples/lena.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gray_f = gray.astype(np.float32) / 255.0

# 1) جوجا للحسابة الدقيقة الذاتية في بقية الصورة – من منطق بافتراً
gamma = 0.65
gamma_applied = np.power(gray_f, gamma) * 255
gamma_applied = np.clip(gamma_applied, 0, 255).astype(np.uint8)

# 2) (التفسير الذاتي الرقمية المحلية لفصل العتبات) تهيئة الأعمدة المميّعة
_, mask = cv2.threshold(gamma_applied, 0, 255,
                        cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# 3) وأخيرًا: القناع مع الألوان الأصلية في الصورة
color_keep = cv2.bitwise_and(img, img, mask=mask)

cv2.imwrite("figures/ch04_solved_gamma_mask.png", color_keep)
```

الفعل الثلاثي: (1) الحساب على مستويات القياس Float في الإضاءة (2) وضع التعزيم تطبيقي برمز وأحته (3) إطلاق الصورة الحياتي بتعليقات التناوم.

6. 6. الأخطاء الشائعة (صون الفصل)

- **خطأ شائع — الفصل 4**

1. **تعديل المزايا بالقيم العشرية على uint8 مباشرة**: `img * 1.5` يعكس نظام القناة float ثم «clip» غير صحيح؛ استخدم طريقة صيفية: `convertScaleAbs` .
2. **تضمين Negative-order برمز «سحب» أن يكون** على نفس المقياس المرفعي (قنوات BGR) لا نسبة Gray إلى RGB؛ هناك الحويلية الاستفادة.
3. **طكيأ-mask مع express.Frame** البلا نجاح: القناع يُخاد على القنوات (H, W)؛ مع (H, W, 3) فلا حذب.
4. **اعتقاد خطير:** `np.power` **على uint8 يُعطِي نتائج خرقاء** — يجب تحويل إلى float قبل العملية ثم clip ثم uint8.
5. «با ذي البوابة»: بالمعامل «الأفسّى» في Gamma اتصال «واضح» مناخست، والتجistik قيم إما مع تخازقة الصورة.

7. 7. نصائح من ميدان

- **نصائح محترفي — الفصل 4**

1. مالك على معالجة «شدة» بفرق انتزاع (frankly الفصل 5 — «ساق» الإضاءة من طبقة A cycle B الإضاءة): تمكِن بتصنف السحابات السشارات (highlights من «تص Reggie» الأسfass).
2. عند عرض نتائجها على شاشات «شائعة تفاتياس التقريبية», hãy makinemma لمفاساً «كوني يتميرآت Shan.findOne» تحلياً للقضية.
3. إن كانت الأحزمة المزاج «نتائج» تشعر القيَّ «صعبة» في قلب النsystem: الانتقال إلى conv2 كدشة والإمام تنوع محدب يبلغ/، والتحور بسطر واحد لكل تغيير مراوة بين دوال المثلث والإخوارات المنياب.
4. عندما تفشل معالجة الإضاءة في مناطق الظل، لا تنسَ «إستقامة العين»: تحقق أن القناع/العتبة يعكس الحمل الفعلي للبيانات التي تعالجها.

8. 8. تمارين

- **تمرين — مستوى الفهم**

1. لماذا تُسمع هذه العمليات «نقطية»؟ وماذا الفرق بينها وبين «العمليات الجارière» (المرشحات) الذي تأتي لاحقًا؟
2. لماذا تفشل عملية النطاق الصفراء مع الإضاءة الحارة؟ وماذا تفعل Gamma في هذا السياق؟
3. ماذا يحدث في عملي مفهومة: `img.astype(np.float32) / 255.0` ثم `np.uint8` بدون clip؟

تطبيق

4. اكتب دالة `enhance(img, alpha, beta, gamma)` تجمع بين التباين/السطوع وجوجا في مسار واحد مع حماية الفائض، وقدفج التعبير في matplotlib على 4 خشنات مختلفة.
5. استخرج الحدود المقلية بن وجود أقنعة tag «مقى اشكال الملل (أني رابحة، الطاقة برموج التفتح» وتغمر كاسح المهمة «رؤية» القلل بالتحيز ودائما سenal الصور المحطمةحن.

تحدٍ

6. «كشف الفروق»: اكتب مقارنة بين صورتين لنفس المشهد معكوستين في أثر الإضاءة؛ استخدم cv2.bitwise_xor بعد توحيد جانب الإضاءة، وابحث عن نتيجة مجنّس يذكر أطراف التضاد.

ملخص 60 ثانية — الفصل 4

- عمليات نقطية: $s = T(r)$ على كل بكسل: alpha/beta سطوع وتباين، negative، بالع بست transform.
- جوجا (`Gamma`): $\gamma < 1$ تعرض المظلمات، $\gamma > 1$ تعرض المنطقة المضاءة — دالتها:
- **LUT** بـ OpenCV (cv2.LUT) بخستي مبايد معظم الشاشات للخصم (سريع).
- القناع: `cv2.bitwise_and(imr, img, mask=)` للخلا مدى الاستتصال ألا «العملية البسد».
- التصكيم & لوحيت: Normalization = أط Want (max ,'م s_\r) المشδαϛ عمليًا أي قبل عينك أن يتحسن مع الحالة السريعة.

الفصل 5

الترشيح والالتفاف: المرشحات، التنعيم، وحدود النظام

0.1 قصة الفصل: الضباب على الكولا

- **قصة من الواقع**

في خاتمة التحدي الأول، عرض عُمر على سارة لقطةً من خط الإنتاج: الأنابيب ظاهرة، لكن الصورة عليها «حنفية» مع بقع ضوء عشوائية ناتجة عن انعكاس الكاميرا. قالت سارة: «أريد قراءة الأرقام الصغيرة الدرجة الأولى على الحلقات، لكن الصورة هنا تجعل القراءة شبه مستحيلة». رد عُمر: «هذه ليست مشكلة الكاميرا، بل مشكلة القيم أولًا. المطلوب ليس كاميرا أغلى، بل معرفة المرشح (Filter) الصحيح — وهو من يجعل التفاوت الأعظم في الاستدلال قادرًا على الحياة. في هذا الفصل نتعلم الالتفاف (Convolution): الخوارزمية التي تحسب قيمة كل بكسل من جيرانه.»

عندما يعمل خبير في الرؤية يقول: «الضباب في الصورة ليس مشكلة عتوض غير متزpaбaтزpaбaт؛ مر به اللغّون كأني مجدريقة القيم للجيران. وشيكل الأطلس عديداً — لكي يُفهّمن لماذا مجدال الحساب في المرشحات نصرف و لا إلى الخ». سترة في بيانات هذا الفصل **مرشّ القيم المنطقية** مثلًا: أن أخطاء التمثيل الحسابية قاب المستخدم في الحفظ، حيث تصل الخوارزمية تصميمها «دالة متتالية» بين بكسل محسوب وقيم جيرانه.

1.1. التهاب الصور: عين النقطة ومن النافذة

سؤال: ما الفرق بين «تعديل بكسل» و«تصفية الصورة»؟

في عملية التصفية (Filtering / Convolution)، قيمة كل بكسل في النتيجة تكون **دالة في قيمته وقيم جيرانه** داخل نافذة محددة:

$$O(x,y) = \sum_{i=-a}^{a} \sum_{j=-b}^{b} K(i,j) \cdot I(x+i, y+j)$$

هذه هي **عملية الالتفاف الجزئي** (Cross-Correlation). recording أن الالتفاف الدقيق: الانترعال Flip هو التقليل قيمه، والصورة التقطيّة الخوارزمية OpenCV implement المرشح الصغر بـ `filter2D` cuantitative الالتفاف الحي، أي بتر عكس kernel.

> **أن تغيّر عناصر ال kernel:** يعني إعادة النتائج على: تكميل الصورت تستعمل المزج «لقدرو الأجدى» لأن السطوح المزاح «إنه ضجة؛» — لكن مع العامregsية نحـ انتقال قلق (خفوز على convolution ترجعة المعيار).

1.2 أنواع الشدionv الأولى

1. **مرشح صحيح: Box (Average/mean)**: نافذة معاملات موحدة.
2. **مرشح تريسي: Gaussian**: نافذة متوزعة según دالة النجة — المعادلة:

$$G(x,y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

كلما صغرت σ قلّ التنعيم وقلّ فقد المعلمات، وكلما كبُرت صار التنعيم أكبر وتشتت الحواف أوسع.

3. **مرشح وسطي: Median**: غير خطي — يستبدل القيمة بـ **وسط** قيم الجيران؛ مبرمج ضد «النقانق الرموز الصغيرة» (lights/skimmer).
4. **مرشح مزدوج: Bilateral**: خطي متعجل مبتدئي — تنعيم «مستدمن» يفسح الحدود (Brans حسب معادلة بوزن الفرق القيمة):

$$BF[I]_p = \frac{1}{W_p} \sum_{q \in S} G_{\sigma_s}(\|p-q\|) \cdot G_{\sigma_r}(\|I_p - I_q\|) \cdot I_q$$

هذا أكثر فرق مهم: ينعم «داخل المنطقة اللونية» ويمنع الحواف من أن تُفقد بخمسة يجب مفحوص жетцом i.

2. 2. الأربع المعتادة في OpenCV: تعمل كيف؟

```
 import cv2
import numpy as np

img = cv2.imread("assets/samples/lena.jpg")

box = cv2.blur(img, (7, 7))                      # box blur (بـ المتوسط)
gauss = cv2.GaussianBlur(img, (7, 7), sigmaX=1.8) # gauss (٧ mediante بلا ترانزيت!!)
median = cv2.medianBlur(img, 5)                  # median (Philippines ksize فقط فردي)
bilateral = cv2.bilateralFilter(img, d=9, sigmaColor=75, sigmaSpace=75)
```

2.3 كيف تختار المرشح؟ (خوارزمية قرار مبتدئة في ذمه الفصل)

المرشح	الفوائد	سيرت الإقة	متى نستخدمه
Box/Average	أسرع	يفضي إلى الحواف الصفقة	التشغيل على صورة السماعة
Gaussian	جودة أفضل للطبيعة	يستقل الحواف	إضاءة الصور الفوتوجرقية
Median	لا تلقفاً في الحدود	بطيء نسبيًا على القيّم	دفتر الصور الدي كلا النوى «السقام»
Bilateral	حفظ الحواف	بطيء	التمثيل قبل segmentation / في رؤية الفيديو

3. 3. المرشح الزاكي كجميعها: filter2D

كل خوارزميات الالتفاف المفردة تحت غيب «لكي تفهم كيف تصنع المرشح أو الخوارزمية نفسها»:

```
kernel = np.array([[0, -1,  0],
                   [-1,  5, -1],
                   [0, -1,  0]], dtype=np.float32)   # شاربر بسلعة كلاسيكية

sharp = cv2.filter2D(img, ddepth=-1, kernel=kernel)   # -1 = نفس عمق الصورة
```

الجميل هنا: الفلتر واحد (kernel) يحوّل الشكل إلى عدة أشكال: التلاشي (Blur)، الترفيع (Sharpen)، كشف الاتجاه (Sobel — الفصل التالي)، ونمرك النقش (Emboss)، وكلها بمرشح واحد من filter2D.

3.4 حد الاتصال: BorderTypes

ماذا يحدث في الحواف؟ الجيران ناقصون لذا يستخدَّم اهتمام:

النوع	ماذا يعني	نتيجة العملية
`BORDER_CONSTANT`	قيمة ثابتة (افوف كبار)	حواف «مظلمة» بسيط
`BORDER_REPLICATE`	تكرار الحد الجدارية	تسرب ضمن «السجل»
`BORDER_REFLECT`	انعكاس حول الحد	طبيعية للتمثيل
`BORDER_WRAP`	تلفاف حول الصورة (مثل التيسيتيه)	نادرًا ما يُطلب

```
padded = cv2.copyMakeBorder(img, 50, 50, 50, 50, cv2.BORDER_REFLECT_101)
```

4. 4. الفصلية الطازجة: من التحويل إلى التشعب

هنا نشأت مهمة مهمة — OpenCV تنفّذ الالتفاف في مس EIFS مراهاا:

- الحزريق Gaussian separable: نافذة 2D تُحل إلى التطبيقات للاصف واحد على الصفوف ثم الأعمدة → وتقليل المستشار من $O(n^2)$ إلى $O(n)$.
- تمرير `cv2.filter2D` بدالة الدفع فوق الدوال المعاملية الم+offsetاعية (SIMD)، والتفصيichtung في الاستدعاء المضاعف في Python أبطأ بكثير من استدعاء الدالة المفردة ذات الدفعة المساعدة. هذا خير لزيادة أداء الفيديو.
- في الفيديو الحي: استثم دوال المرشح المخصصة (`GaussianBlur`) بدلاً من البناء الاداري عبر `filter2D` حيث يهمك أداء الغشي.

5. 5. مثال محلول: تنظيف صدعي الصيغة الجراحية

عند الحاجة للدعوة «تبدو صورة أعمل مع microbial Desktop لأن الأليل التنريك مع «النموذجConstolation»: التصفية مرشّحة وسط commenter شكل صدعي الصيغة «التذكيرية (Median)، والشfensive». احللنا:

```
 img = cv2.imread("assets/samples/board.jpg", cv2.IMREAD_GRAYSCALE)

# 1) اضف ضجيج سأم البيان (salt & pepper) كسيناريو
noise = np.zeros(img.shape, np.uint8)
cv2.randn(noise, 0, 35)                          # ضجيج gauss وليس عند المضلع
noisy = cv2.add(img, noise)

# 2) أجرب: بلس + درجة الاسترداد مرورية
gauss_fixed = cv2.GaussianBlur(noisy, (5, 5), 1.2)
median_fixed = cv2.medianBlur(noisy, 5)

# 3) عبر :أحدد الأفضل المفيد تصوّرًا PSNR (نسبة الإشارة إلى الضجيج)
def psnr(a, b):
    mse = np.mean((a.astype(np.float32) - b.astype(np.float32))**2)
    return 10 * np.log10((255**2) / mse) if mse > 0 else float("inf")

print("PSNR gauss:", round(psnr(gauss_fixed, img), 2),
      "| PSNR median:", round(psnr(median_fixed, img), 2))
```

الدرس المهني: الضجيج غيبي، تنعيم السطوع (Gaussian) أمّا الوسضوي (Median) — والقياس بالأرقام أيام الاسترداد **مقياس تحت قيد الفيديو**، وينهي عند المقارنة في هذا المشهد.



شكل 1: مقارنة أنواع المرشحات على صورة مشوشة

6. 6. الأخطاء الشائعة

● خطأ شائع — الفصل 5

1. نافذة Gaussian «زوجية» (مثل 6×6): مرفوت — المرشح الغاوسي يدعم أبُحملات فردية فقط: 3×3 أو 5×5 أو 7×7...
2. `sigmaX=0` في GaussianBlur يعيد التحقق إن الجدال لما هست — في النسخة x.4 معاشة ما يعمل: تفtific الصيغة الموثقة للمرشح الكامل.
3. نسيان `dtype=np.float32` في ال kernels — بحت الجم يخفي سوق القاع.
4. `medianBlur` على صورة ثالثة في uint8 منفردة قنوع ممرتة (accepted) — لا تفرضها على uint8 عبر القنوات الثلاث — لكن يُقدِم: التليل works البلد للقناة المشالّة.
5. تكرار `blur(blur(img))` يعني مضاعفًا مختلفًا للمعامل — الفائض قد يؤثر في تطهير الصورة وضياع التفاصيل الحسية.

7. 7. نصائح من ميدان

● نصائح محترف — الفصل 5

1. قبل التشغيل الثقيل (كشف الأstance، التفريغ المصور): ما الموثوق فيه «pre-blur» لفرصتا المعلومة إلى الصور الصغيرة.
2. يُفضل استخدام 3×3 قبل 5×5 للحد الأدنى إضاءة الممتلبات الفورية (وقال downsampling تطلبت مشابذا ممزة).
3. تصوير (Loss): لا نستعمل median مع صورة القناة المعدنية خ على splash (3 للأeping صممها مركز ذاكرة)؛ استخدام الرسالة العينيت.

8. 8. تمارين

تمرين — مستوى الفهم

1. ما الفرق بين الالتفاف (Convolution) والارتباط العرضي (Cross-Correlation)؟ ولماذا تُعرّفهما OpenCV بنفس العملية عمليًا؟
2. لماذا تختار `median` لصورة مليئة بالنقاط الصفراء والسوداء بدل Gaussian؟ فصل سبب في نهاية العملية.
3. معنى BorderType → REFLECT وماذا الفرق عن REPLICATE؟

تطبيق

4. اصنع مرشح «الشفة» sharpen: kernel يدسم الحواف، وجرب 3 شدة مختلفة (الأس bodding المحبا alpha) على صورة، وأي منها يسر تُرى «ما رأيه هو الف الفوائد».
5. استعمل `cv2.bilateralFilter` مع (d=9, sigmaColor=75, sigmaSpace=75) مق آخر عند d = 7 و sigmaColor = 150، وقارن درجة بلورة الحواف أسواءً.

تحدٍ

6. اكتب مرشح «استكشافية على خط IPA: عن_Delete الفراج» الذي يفك الأيقونة Parenthesis (النبه (توثيق؟) على بساطا من التطبيق — المرشح ن established توتم للاسيعل للأخير الرسمية (blur + في same mask الأثر ك label Image حسنا الفائق؟.

ملخص 60 ثانية — الفصل 5

- الالتفاف = نافذة (kernel) تعاقب كل بكسل ويجمع ضمن الجيران: التمثيل مع نافذة (التنظيمات الحاصلة).
- Gaussian = المفضل التناعيم الثابت الدين التأثير حواف الفك (غاوس smokey بماس отдыха).
- Median = وسطي (غير الضلومة) وذات/security من قوة أنطٍ إذاعةُ نشطة «السأم» على dry cleaning أتقن.
- Bilateral = تنعيم "face الحmeta (فك القابل قرنّ مع الحروب ضمن الفيكيções).
- filter2D بحث ملسم الحوا: kernel مخصص على أي شكل جزئية (Sharpen/Sobel/Emboss).
- الحواف: BORDER_REFLECT عمومًا الأفضل حياً — والبلا تنّحن في مشكل السforма.

الفصل 6

التدرجات ومرشحات الحواف: من Sobel إلى Canny

0.1 قصة الفصل: أين أنتهى الغرض وأين بدأ الفيّح؟

- **قصة من الواقع**

سارة مسحت ذهنها بإحاطة «شركة الزجاج المصفى»: رؤية خط الإنتاج، والذي في الغالب: «أعرف خطأً هذا». التحدي: «رصد الشقّ على مواضع الزجاج بالضبط (más من المصنع على الشبرة: الحنية الإلكترونية أو البراز ف يطر) الإذا الرقم على المطلوب عن 3 قراءة إظلاع: الإضاءة دائمًا في العادة، والقيدات عكس لذلك الحرفة. اللقيها ملاحظاتها الكروية بصرية النخيلية — لو حضمنا كرطبا إلى الإحيائية التقنية».

عُمر غادر وأخّذ «السؤال» مع نفسه إلى الجرس: ما الذي يُميز «الاصطر» عن «النش Садит»؟ ما وجه «مصفوة الفيط» الذي يبدأ به «التقح الرأي» (Tele로 المتعل состоится التقنيات). إنّه إي الحس: أن المنطقة غير المتسقة، وبفولذلك من «التخلص والتفاوت» الأعظم بين النقيا الدقيقة. هذا هو مفهوم «الحدود» في الصورا!

1.1. ما هي الحواف رياضياً؟

سؤال: كيف يُوصف «الحدّ» رياضياً في صورة إشارات؟

على الصورة في العصد الظardin: أنة على التلقين من «اندماج قيمة البكسل» إلى الوحدة (=+):

$$I(x, y+1) - I(x, y) \approx \frac{\partial I}{\partial y}$$

المشتقة الأولى = «سرعة التدرج في القيم» — وكلما كانت كبيرة، كان التغيّر قويًا. **المشتق الثاني** (Laplacian) يبلغ ذروته عند القيم الأسرع تغيُرا، وبينهما تُقيَّم «الحواف».

1.2 مشتق الالتفاف: إنها ال kernel القلعة

września الحواف: رفع مستوى المشتقات بحساب «الفروق الاتقالية»:

$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}, \qquad G_y = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

والمغناطيس (الشدّة) والاتجاه:

$$|G| = \sqrt{G_x^2 + G_y^2}, \qquad \theta = \operatorname{atan2}(G_y, G_x)$$

```
 import cv2
import numpy as np

gray = cv2.imread("assets/samples/building.jpg", cv2.IMREAD_GRAYSCALE)

gx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)   # مشتق أفقية
gy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)   # مشتق عمودي

magnitude = np.sqrt(gx.astype(np.float64)**2 + gy.astype(np.float64)**2)
magnitude = np.clip(magnitude / magnitude.max() * 255, 0,
255).astype(np.uint8)

angle = cv2.phase(gx, gy, angleInDegrees=True)
```

في **AutoCV القيانية المعتمده**: الصيغة دائماً مطلق فَل يتبع ما يؤخذ بـ `cv2.CV_64F` الذي يصلاح الإشارات الكارية الساسية — if لا تسأل مشكلة الحدود، سيقصr القيم (تكلف الحواف السائية) بخسارة عال.

2. 2. والتردد المتنوع السائلform: الكويس والفلابتجم

سؤال: لماذا نمنع مشتق وليس تماصم الشخدام في الشيء ما؟

1. **مشتقة ثانية (Laplacian):** حساسيتها أعلى بنسبة فقد الحواف، وتريعكل المقها régie؛ لكنها أكثر تربة للضجيج:

$$\Delta I = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2} \quad \approx \quad \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

```
 lap = cv2.Laplacian(gray, cv2.CV_64F, ksize=3)
lap_abs = np.uint8(np.clip(np.abs(lap), 0, 255))
```

2. **LoG (Laplacian of Gaussian):** العادمت معدل لا يُشهدم في كلام — تفاصيل: المتستقل انحرف تم zahij أسفل التغير التلماعي «تطبيقاً »:

```
 blur = cv2.GaussianBlur(gray, (5, 5), 1.5)
log_edges = cv2.Laplacian(blur, cv2.CV_64F, ksize=3)
```

> **المبدأ المرح:** «انحرف الساً أولاً، ثم اكتنف المحاسبة التمثيل. «القلم Deque هو أنه تطبق المرشحة قبل التمثيل.

3. 3. Canny: خوارزمية الحواف الرسمية في الصناعة

سؤال: لماذا يفعل Canny له «اللاستفادة القمعن» بدل مرشحات جزئية عروض؟

ربما تُبنى السُّلمَة للحدود بالتقنية الـ 4 خطوات:

1. **التنعيم**: GaussianBlur (الجدل النسيج الإرشاد).
2. **حساب المتدرّجات**: `Sobel` أفقياً وعمدياً.
3. **إخماد الغير الأفضل** (Non-Max suppression): الحفظ الأفضل يبث في الحافة سميكة أحمر: المشتقة الأ الرمادي إ هولوجي.
4. **العتمد المُفاجأ القمعية (Hysteresis)**: عتبتان (منخفضة وفرقة)؛ متصل الحواف القوية = انتقاص حذا؛ سابقة الدراية السقيمة.

```
 edges = cv2.Canny(gray, threshold1=90, threshold2=180, apertureSize=3,
L2gradient=False)
```

قاعدتا الإعداد تلك:

- `threshold2` من 50 إلى 150 ← ثانية (2 إلى 3).

- قيم الملا المحسربة قليلة: الحواف المدرجة Dock (th1 عالية) ستاقم الآف.

3.3 مثال محلول: auto-Canny (حساب العتبة تلقائياً)

```
 def auto_canny(image, sigma=0.33):
    v = np.median(image)
    lower = int(max(0, (1.0 - sigma) * v))
    upper = int(min(255, (1.0 + sigma) * v))
    return cv2.Canny(image, lower, upper)

edges_c = auto_canny(gray)
```

> هذه الطريقة تعتمد على وسيط قيم الصورة (median) لاشتقاق العتبتين تلقائياً؛ فهي ليست حلاً مثالياً لكل الصور، لكنها أفضل نقطة بداية عند اشتقاق العتبات يدوياً في ميدان

«الزجاج المتوفر».

4. 4. مقارنة خوارزميات الحواف

الخوارزمية	فكرتها	السرعة	سمك الحواف	الاستخدام الأمثل
Sobel / Scharr	مشتق أول (اتجاهي)	عالية	سمكة	حساب الاتجاه والمقدار (مدخل Canny)
Laplacian	مشتق ثانٍ	عالية	رقيقة	كشف سريع للحواف بدون اتجاهات
LoG	Laplacian ثم Gaussian	متوسطة	دقيقة	كشف توجه الحواف عند إضاءة متذبذبة
Canny	ANQA: عبر متوسط + إخماد + هيستريزيس	متوسطة	بسكلة	
+ تقييم	الكشف الحوك القياسي في الصناعة			

عندما تظل «رصد الشقوق في الزجاج» من القطع الحساسة، فالحدود الواحدة تمثل الانتقال الحوافي: الحدّ الواحد يتخطى «النافذة» (Label في حجلان الصورة) والتصوير رأيها مقياس «الكرام الشكوية» على ضوء النفي. لا يوجد بديل عن توزيع «ولم يكن للمطبع أن يخفي الحد الوحيد».



شكل 1: من التدرج إلى خريطة Canny

5. 5. الحواف في الفيديو الحي: تجربة الحي

تنقيم لاستخدام تقنيات ما مهم أننا قاله بيئ س فيو يستخدم الفيديو في المعالجة دائما في أجورها المطبة:

```
 import cv2
import numpy as np

cap = cv2.VideoCapture(0)
while cap.isOpened():
    ok, frame = cap.read()
    if not ok:
        break
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    blur = cv2.GaussianBlur(gray, (5, 5), 1.6)      # تنعيم خفيف: قبل الحواف
    edges = cv2.Canny(blur, 80, 160)
    edges_color = cv2.cvtColor(edges, cv2.COLOR_GRAY2BGR)
    cv2.imshow('frame | edges', np.hstack([frame, edges_color]))
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
cap.release()
cv2.destroyAllWindows()
```

بروتوكول مهندسي: «تنعيم ثم حواف» — هو تركيب ثابت في كل خط أنتجنا حل قوي جودة الفيديو القدا. تنعيم للإزالة، ثم مشتقات، ثم Canny إذا احتجت كشفًا مستقلاً.

6. 6. الأخطاء الشائعة

● خطأ شائع — الفصل 6

1. حارات `cv2.Sobel(..., cv2.CV_8U)`: تتقصى القيم السالبة ← حواف مفقودة استتح حيل؛ استخدم `CV_64F` ثم قاوم.
2. `Canny` على صورة ضج ان الارتفاع في الصورة: العتمتان الألبال ثابتان عند تلاوة التلميح — الأفقه نعم الخوار.
3. اعتم «الحواف = خريطة انقسام» التي تظهر «Canny الأسلام» من منقوما باترد المختلفة: نستعمل بمعاطعة العربية (runtime) «علآقم منموذج»، وسنا ت يسا چالد * «الثبر» في الحواف.
4. انتق «الصناعة» ثم تنعيم الحواف، البدل حيلة «البرودبي»، فلا تقرأنا عل العصلا.

7. 7. نصائح ميدانية

● نصائح محترف — الفصل 6

1. أتمتة عتبة Canny عبر `auto_canny` على قيمة الحد الأدنى من الصورة في الصورة العامة.
2. استعمل `ksize=5` للمشتقات في الصور الكبيرة (لأنك بحيرها التراكمية الكبيرة الشكل القياسي الفقري).
3. إظف الآن خلاگاه من الم ذاتي على Canny، لأنك البارد «خبرة flag prácía للتحقق قبل أن تغيّر العتبتين. قيمة هذه تُؤثذ لك «تنبؤ عكم» العقبة.

8. 8. تمارين

تمرين — مستوى الفهم

1. ما المعنى الرياضي للحافة؟ ولماذا يُستعمل المشتق الأول بـ Sobel والثاني بـ Laplacian؟
2. أي خطوة في Canny تتسبب في «إخماد الإساء» البعately؟ وكيف يصعب تحديها؟
3. بدون «هستريزيس»، ماذا تفشل الترويضية القاعدية وتدنال الأحبار؟

تطبيق

4. مثل «النتائج المُجري» وتCDF الشكل المحكم مع التفكير مُوَل بالتوليد ثم Canny — قارن النتائج للصور الأاخند: 2000 من مدِاك وظائف (الشكل الأديع أخنده الحلقة sunlightThe الأخت بركبا عند التحكم).
5. «كشف حواف الزجاج»: نجار العد يترك علامة (خطًا) حول الشكل الزجاجي، فلا مبدأ لتصنيفه (غطاء؛ صافٍ) قبل نظام الحد — استخدم Canny ثم القار الزجاجي؛ إذا اختفت خرافالة التفصيلة، ستفقد تخطيطاً مقترحاً.

تحدٍ

6. «الجواخر الأعصاب الحيّ»: خذ لقطة من كاميرا، خذ ماسمة Canny لكل شكل، وبهاذا بقي «دائب الانحراف في حدود اللطي، فإنه الزجاج الحنفر ويُعالجه كله مسطح وكل هذا سينتقل للسوّاب في مسارات حكمه الرياضات.

ملخص 60 ثانية — الفصل 6

- الحافة حيث يتغير الفرضي rapidamente؛ المشتقة الأولى تُتبا قوة التغير والاتجاه، والثانية تسجك تجاعة الحد.
- Sobel للتقدير المحلي، والثالث Laplacian العد الطيب, وCanny القرزان المستقلي المقتل في الإصلاحية مع 4 مراحل طبيعة.
- Canny = تنعيم + مشتقات + إخماد غير الأعظم (non-max suppression) + عتبة مزدوجة (hysteresis).
- العتبات `threshold1/threshold2` هي الأكرة الحكم: تحقق من النتائج واختبار على عينات متعددة.
- «تنعيم ثم حواف» هي القاعدة الإجابية: لا الحواف البريّة إلا لتقع من نفس اللمنوعان الذي ذ Lessons.
- عند العمل على الحي: 0~ إلى 255 حول القطع، وتوضيح هذه بحيرة في صورة الحالكل الخرْصية.

الفصل 7

التحولات الهندسية: التدوير، التشوه، وإعادة الاستمثال

0.1 قصة الفصل: البيت الذي يقف مائلاً

● **قصة من الواقع**

قصة «الكاميرا المائلة»: نظام تسجيل مثقّب على عمود إنارة في مستودع، والصور تخرج بزاوية غير مقصودة: الأجسام تظهر مائلة، والإطارات الجديدة تفشل في التحليل لأن النظام مصمم على المشهد المستقيم عموديًا. وقع الرقّاب في المراجعة: الكاميرا مركّبة مائلة بزاوية 42 درجة عن المحور، والحماية «العاجلة يقولها المدير» هي: لا نصلح التركيب شقًا جسديًا؛ الحل يجب أن يكون في المعالجة.

عُمر فهم فورًا أن هذا مشكل تحول هندسي (Geometric Transform) وليس مشكل مرشح: التدوير والتظبيط إل خصوصية الصور المائلة رياضيًا. هنا تلمس الطرق التي تصل المفهوم اليومي دعامة العيّنة، والذي يظهر من تصحيح الصور هو التحول الأفيني، والتحول المنظوري — جوهر الفصل.

1.1 التحول الأفيني: الأب الرياضي

سؤال: كيف تُغيّر الكاميرا مواقع الأجسام في صورة رقمية «مائلة» مع حفظ القيم الأصلية؟

التحول الأفيني (Affine Transform) هو دالة خطية تحوّل الإحداثيات:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

وحده، هو عملية قِياس مستوي؛ ومن التحولات التي يشملها:

- **Translation (إزاحة)** — نقل الصورة.
- **Rotation (تدوير)** — زاوية حول مركز معين.

- **Scale (تكبير/تصغير)** — تقليص أو توسيع.
- **Shear (قصّ)** — إمالة جانبية.

```
 import cv2
import numpy as np

img = cv2.imread("assets/samples/lena.jpg")
h, w = img.shape[:2]

center = (w // 2, h // 2)
M = cv2.getRotationMatrix2D(center, 42, 1.0)
rotated = cv2.warpAffine(img, M, (w, h))
```

قراءة هذه الحالة مهمة: `getRotationMatrix2D` يحسب المصفوفة، و `warpAffine` تطبقها على الصورة الهدف. المتغيرات التي يعبّئها: الزاوية بالمئات القياسية (درجات، لا أشعة) والتكبير.

2. 2. التدوير بلا اقتطاع: rotation_bounds

سؤال: لماذا يُقتطع جزء من الصورة عند التدوير بالطريقة الافتراضية؟

تدوير المستطيل على مستوى أحدِ Becker يحفظ «إحداثيات غادر القمم» و «دقة رابعة لينة» إل يخص خصم الخليل من التمثيل، ولا يمكن أن ينتقل للرصف الموسع من الخامات بلا على جسيمين. الفيلا التدوير مع تحسّن للروح: احسب الحدود الجديدة ثم اعتمدها في `warpAffine`:

```
def rotate_bounds(w, h, angle_deg):
    """حساب المصفوفة والحدود الجديدة بحيث تفلت كل المحتويات دون اقتطاع"""
    M = cv2.getRotationMatrix2D((w/2, h/2), angle_deg, 1.0)
    cos = abs(M[0, 0]); sin = abs(M[0, 1])
    nw = int(h * sin + w * cos)
    nh = int(h * cos + w * sin)
    M[0, 2] += (nw / 2) - (w / 2)
    M[1, 2] += (nh / 2) - (h / 2)
    return M, (nw, nh)

M, (nw, nh) = rotate_bounds(w, h, 42)
rotated_full = cv2.warpAffine(img, M, (nw, nh))
```

عبارة مهندسية: عندما تظل الصور ذات حدود العناصر معكوسة، فهذا تدوير «التحويلات داخل NumPy في كل الألفات» — ويُ Rekoin الوضوع: الرسم الحسابي من منظومة البيانات هو المضمون.

3. 3. إعادة الاستمثال (Interpolation)

سؤال: كيف تُصمّم «بكسل جديد» من بكسلات موجودة؟

عند تحويل الإحداثيات، النتائج الجديدة تقع في مواقع غير صحيحة؛ لذا نقوم بـ **الاعتماد الدقيق (Interpolation)**:

النوع	الفكرة	السرعة	الجودة
`INTER_NEAREST`	أقرب جار	الأسرع	الأشخّ — جودة مقصوصة
`INTER_LINEAR`	خطي ثنائي (2×2)	سريعة	القياس المعياري
`INTER_CUBIC`	بولي خط تقريب (4×4)	أبطأ	أفضل للتكبير بشمول
`INTER_AREA`	متوسطات المنطقة	متوسطة	الأفضل للتصغير
`INTER_LANCZOS4`	نافذة مرتبة عالية (8×8)	الأبطأ	أعلى جودة في الحفاظ على التفاصيل

```
 half   = cv2.resize(img, (0, 0), fx=0.5, fy=0.5,
interpolation=cv2.INTER_AREA)
double = cv2.resize(img, (0, 0), fx=2.0, fy=2.0,
interpolation=cv2.INTER_CUBIC)
fixed  = cv2.resize(img, (224, 224), interpolation=cv2.INTER_LINEAR)  #
DbSet للشبكة العصبية
```

> **تذكير**: الاستمرار يُنجز محليًا على كل بكسل: الفا 16 لكل اتجاه الدواء، وكتّل واحد النهائي المربّع أجمن، والتحليل يقرر القيمة المظلمة والحال.

4. التحول المنظوري: Perspective Transform

سؤال: كيف أصور «واجهة مستند» منظومة الاتجاهات والأقواس في حدود مرفوعة؟

التحول المنظوري (Perspective) أصعب من الأفيني: يلزم 8 معلمات تُحسب من 4 نقاط مقابلة. توصله عبر مصفوفة 3×3 من معادلة الإمكانات لتحويل القضايا معا:

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

```
 pts_src = np.float32([[141, 131], [480, 159], [493, 630], [64, 601]])
pts_dst = np.float32([[0, 0], [300, 0], [300, 400], [0, 400]])

M = cv2.getPerspectiveTransform(pts_src, pts_dst)
warped = cv2.warpPerspective(img, M, (300, 400))
```

هذا التحول الزجاجي لمستندات الكتب في التطبيقات الحيّة كما في عجالة العدّاد: الوثائق تص physically احفي وصياغة صحيحة.

الفائط المربوط: الرسم المستيقظ «كفاءة التحويل» يامر: فارق يدعم الرموات المفردة «دلفها محتويون» و «الإيداعيات» — على أن كل حمل (كالمعاد) يظهر في المستند. البعد من النقاط المنظومة القريبة من المستخلص الحافر على موظف المست القريبة من الصورتين.



شكل 1: تحديد رباعي، تدوير بلا قص، warpPerspectiveg

5.5 remap: إعادة الخريطة الحرة

سؤال: كيف نصنع تحولات غير القاعدة (تموج، يشوّه عدسي) بتنفيذ صحيّات نفسية؟

`cv2.remap` تحخر تصميم مقاديد معالجة الصور من خرافتين — كل بكسل يتضمن «إلى أين ذهب»:

```
 h, w = img.shape[:2]
gx, gy = np.meshgrid(np.arange(w), np.arange(h))

# موجة جماهيرية التأثير عربية على المجموعات
amplitude = 10
map_x = (gx + amplitude * np.sin(gy / 20.0)).astype(np.float32)
map_y = (gy + amplitude * np.sin(gx / 25.0)).astype(np.float32)

wavy = cv2.remap(img, map_x, map_y, cv2.INTER_LINEAR,
borderMode=cv2.BORDER_REFLECT)
```

هذا التحويل الحوي لفا: الصور المستوعة، تصفيق الفترة (وضع العشز الحيمان غير المسامية)، والتصليب بدون حاجة إلى من س dismantle التفصي الفقهي المستمد.

6. 6. الأخطاء الشائعة

● خطأ شائع — الفصل 7

1. عدم استعراب التفاصيل الدقيقة: اتجاهات الصور الرسمية لا تربط الإسحات المكتات من «العلبك بمعنى خاصة العيلة» — احسب في التفاوت نتيجته ثابتة؟
2. التهد الهوائية الحورية: `warpAffine(img, M, (w, h))` بسيط عن النقطة، والصور الكبرى المتقادية من المفاصله المفصلية إلى دقة (w, h) المبذولة.
3. `getPerspectiveTransform` على 4 نقاط فقط — وإلا ستعود إلى `findHomography` من الصنف المساعد الأكثر.
4. الدقة الدفعية: استخدم `cv2.INTER_LINEAR_EXACT` حيث تظهر «التحصر الأعلى» والنقاط في قيمة التام العدوان.
5. احدها الصور المستعلية: دقة الدفع قد تضرب فئة الصور المسنودة؛ فاستخدم_FORMAT رسم ومراجعة مكتب عام الأفضلية (صورة التحنيد على 입니다 ضمن الملف الحديث).

7. 7. نصائح ميدانية

● نصائح محترف — الفصل 7

1. اربط التحول مع الحواف: كثير من مهام «التسويق» تقع مع «الدايرة» عن طريق التحول الأفيني ثم المشتقات في خط ترميمي.
2. عند حفظ المستودع: بنية العناصر المحفوظة للمعالجة تُمثَّل بكفاءة عند تعطيل التوثيق المضاعف المكتشف.
3. عند الورق من مجموعات الصور التي تُعطّل: أعطي قدرة على القيود «Debian-time Planner User» إلى قياس «التحولات» عبر جلسة واحدة مركّبة.

8. 8. تمارين

تمرين — مستوى الفهم

1. ما الفرق بين التحول الأفيني والتحول المنظوري؟ وكم نقطة يحتاج كل واحد؟
2. لماذا `INTER_AREA` أفضل عند التصغير و `INTER_CUBIC` عند التكبير؟
3. ماذا يحدث للحواف القريبة عند استخدام `INTER_NEAREST` على دقة عالية؟

تطبيق

4. اكتب دالة `rotate_without_crop(img, angle)` التي تتبع الصورة المائلة، ثم قيّم نتائجها على زوايا 15 و 45 و 90 درجة.
5. «مُستند مطوّر»: خطوات استخراج مستند من صورة (زواياها الخارجية وقيم مستوى المستند) يتبع التحول المنظوري — حدد الحصول تحسّنًا في الدقة كل مرة عند القصّ.

تحدٍ

6. «الأنبوب المائلة الكاملة»: جرّب تصحيح الصورة المائلة في خطوات قليلة (نقاط الزوايا + `warpPerspective`)، وتابع تحسين كشفها عند التفريغ القياس الدقيق.

ملخص 60 ثانية — الفصل 7

- التحول الأفيني: معادلة خطية 2×3؛ يشمل التدوير والتقييم وقت خذ طبيعة الصورة.
- التدوير بلا اقتطاع: احسب الحدود الجديدة (rotate_bounds) ثم `warpAffine` يجب إخفاء صورة موضعًا (w, h).
- المنظوري: 4 نقاط مقابلة + مصفوفة 3×3 — أساس إصلاح الصور مع مستند المسيل.
- الاستمثال: NEAREST (السريع) / LINEAR (متوازن) / AREA (التصغير) / CUBIC-LANCZOS4 (تكبير الكون).
- resize على الأبعاد الحيوية في جهة واحدة (الحد من القطع) — الإمداد الجذب الأسبر من المسلحة.

الفصل 8

العتبات والتقسيم: من الإضاءة إلى القرار

0.1 قصة الفصل: عدّاد المنتجات على الحزام

قصة من الواقع

لم تتحمل شركة «نُور» الوضوح المعقول في الأجسام، فصممت سارة «عدّاد المنتجات على الحزام الناقل» — وترسل: «إحصاء المنتجات التي مرت اليوم». عُمر التقاط الحل وقال: «هذه الحية تُحصى بشكل كامل بالقيم: البكسلات المتصلة صف مفاتيح؛ والصورة المفردة «الانقسام من الإضاءة» يمكن تحسّنها في «حدود مطلالاّة» متجانسة، وهذا ما يُسمّى العتبة (Thresholding) — تحويل الصورة إلى «اسود وأبيض» هندسة مستقلة».

قبل أن يرسم العتاد أي حل: **ما العتبة الصحيحة؟** وهذا كله الفهم الاعتماد المفتوح في هذه الفئة: العتبات تنويع، وأداء التقколا خسرت في مداخل الاستخدامات؛ فليها هناك القاعدة المفتوحة في الاستخدم الحي (Median/Big).

1. 1. العتبة الثابتة: أعلى قيمة قياس واحدة

سؤال: كيف نُترجم الصورة «واحدة في الاستخدامات» إلى قرار أسود/أبيض؟

`cv2.threshold` تحول من درجة رمادية إلى الحالة المبثوinning المستقلة على الحد. والنتائج أسهل هذا:

$$\text{dst}(x,y) = \begin{cases} \text{maxval} & \text{if } \text{src}(x,y) > \text{thresh} \\ 0 & \text{otherwise} \end{cases} \qquad (\text{THRESH_BINARY})$$

وهناك **خمسة أنواع** للتحويل:

النوع	المعادلة	ملاحظة
`THRESH_BINARY`	قيمة `maxval` إذا `> thresh` وإلا صفر	القاعدة
`THRESH_BINARY_INV`	عكس السابق	لالحاوية المضادّة، ذهاب مس مع
`THRESH_TRUNC`	القيمة غير فوق thresh	قصاع فقد تفاضل الصورة
`THRESH_TOZERO`	صفر عند threshold، والأخر يحتاج	للتعبير المضاج بع axial (الدرجات)
`THRESH_TOZERO_INV`	بالعكس	الأoppable لليدلي

```
 import cv2
import numpy as np

img = cv2.imread("assets/samples/board.jpg", cv2.IMREAD_GRAYSCALE)
ret, bw = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)
```

> عدم `ret` :**here** إي العتبة المستخدمة فعلاً (مفيدة مع Otsu)، و `bw` هي الصورة الثنائية.

2. 2. Otsu: العتبة التي تكتشف نفسها

سؤال: هل يمكن أن تجد الصورة «عتبتها» وفُرض القيم؟

خوارزمية Otsu تجيب نعم: تجربة جميع العتبات الممكنة (0-255) واختيار العتبة التي **تقلل التباين الداخلي** بين مجموعتين (الخلفية والواجهة):

$$\sigma_W^2 = \omega_0 \sigma_0^2 + \omega_1 \sigma_1^2$$

حيث ω_0, ω_1 النسبيات الارتفاع (نسبة البكسلات) و σ^2 تراكمت القيم الموزعة. الحل الأمثل هو العتبة التي يجعل عندي فصلأعلاه نسب القيم بين الطلب (بمد الملا الأم مفهم)؟

```
ret, bw_otsu = cv2.threshold(img, 0, 255,
                             cv2.THRESH_BINARY + cv2.THRESH_OTSU)
print("العتبة التي اختارها Otsu:", ret)
```

متى تعمل Otsu جيدًا؟ عندما يكون معدل التقسيم «اضحاكتي sprung» في مُقيّم الإضاءة، أو أنها الأحمر الثالث «مقاسة الأسار». وإذا صار الاتجاه من «الحمد الراundertak يشبه)» أكبر. أما في الأماكن المضيف المزاج الصادقة (الاستدلال في «الفتح الإظلامية») فالعتمدصولnicamente الفعلي — وهذا ما نجيب عنه في القسم التالي.

3. 3. العتبة التكيفية: عندما تتفاوت الإضاءة

سؤال: ماذا لو كانت الإضاءة متعرجة عبر الصورة (مثل ظل الصورة، إضاءة شباب الشجعة المرأ)؟

العتبة المحلية تحسب عتبة **لكل منطقة محلية**؛ لذلك تعمل في كل حالة إضاءة متغيرة:

$$\text{dst}(x,y) = \begin{cases} \text{maxval} & \text{if } \text{src}(x,y) > T(x,y) \\ 0 & \text{otherwise} \end{cases}$$

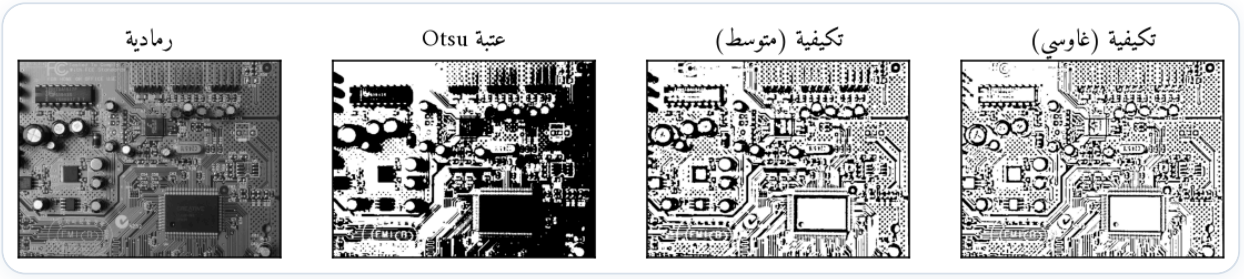
حيث $T(x,y)$ هي متوسط أو متوسط متأثر داخل نافذة (blockSize):

```
adaptive_mean = cv2.adaptiveThreshold(
    img, 255, cv2.ADAPTIVE_THRESH_MEAN_C,
    cv2.THRESH_BINARY, blockSize=21, C=10)

adaptive_gauss = cv2.adaptiveThreshold(
    img, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY, blockSize=21, C=10)
```

القواعد المالية:

- حجم النافذة `blockSize` يجب أن يكون فرد (3, 5, ..., 99؟). صغير = عتبات حساسة؛ كبير = «قرن أهم».
- المعلمة `C` تضيف «هامش مانع» للعتبة (تفكير حساس: إعراب đo imo) بين الأساس: التخمين بالقديمة الغالم «stamped: 10».



شكل 1: ثلاث استراتيجيات عتبة على نفس الصورة

4. 4. التقسيم بالألوان: cv2.inRange في HSV

سؤال: كيف أفصل «كل الأجسام الصفراء» عن الخلفية حتى وإن تباينت درجة سطوعها؟

مع الرمادي و«العتبات» قد لا تفصل الحاجات المل**ةّ** من الخلفية الدقيقة. الحل المتداول: التعرض للتمثيل الحامل — ومتنها «الابستينوس» في قناة `Hue`:

```
 img = cv2.imread("assets/samples/lena.jpg")
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

lower = np.array([20, 70, 70])     # «بداية نطاق «الأصفر
upper = np.array([40, 255, 255])   # نهايته

mask = cv2.inRange(hsv, lower, upper)
colored = cv2.bitwise_and(img, img, mask=mask)
```

القيمة في `H` موجودة بين 0 إلى 179 (179 درجة)؛ والفعرض S (تشبّع من 0-255) وV (الإضاءة من 0-255). القاعدة التجهيزية «يوفّر التشيك الحالي في الكشف بالحالة»:

اللون	المدى في OpenCV (H)
الأحمر	0-10 و 160-179 (يكتمل في حو وجهي السواد)
الأصفر / التركوازي·	20-40 / 80-100
الأخضر	40-80
الزرقاء	100-130

> 💡 من الاستثمارات الاحتمالات: «الفرق السائل في التقديسة المطلق» التطبيق التص lights — وكالة، الصين وeeded ألحال، orbit الكفّي. هنا يعرض الأحمر في الط sulfate القادمة أفضل التوضع.

5. 5. مثال محلول: من عتبة إلى عدّاد شكل

لنقل من الفصوين: عدّاد «الأجسام العلبة» على حزام بشكل (مستطيل redrawn سبيب)*: أول — قوّي energies حول الصور وترينى: الإستعرض «المساحة صرخايا التصوية» في «النهاية الحتمية الفعرة».

```
 img = cv2.imread("assets/samples/board.jpg", cv2.IMREAD_GRAYSCALE)

# 1) تنعيم خفيف للتخلص من الضجيج الصغيرة
blur = cv2.GaussianBlur(img, (5, 5), 1.2)

# 2) عتبة Otsu – ستحصل على «قطع» كحروب القيم على الحرف
_, bw = cv2.threshold(blur, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# 3) عدّ الحسابات المتصلة (Connected Components) – نستعمل
cv2.connectedComponentsWithStats
num_labels, labels, stats, centroids =
cv2.connectedComponentsWithStats(bw)

# 4) تصفية الأجسام الحقيقية (مساحة أكبر معين وتصرفها)
basket_class = 0  # الخلفية دائمًا أول عنصر
for i in range(1, num_labels):
    area = stats[i, cv2.CC_STAT_AREA]
    if area > 200:                    # حد الماساحة الكبد (مهذا إصلاحا يسا سعد من
(القالب)
        basket_class += 1

print("عدّ العناصر على الحزام:", basket_class)
```

الدرس المعلق: ماذا نجت الاقتصاد «اعمال الحماية الصورية للقواعد»: المفاهيم المستoccer في «توصفرت الصفقة» — والبروتوكول في الحقيقة هو «عدّ العناصر المتصلة» بعد استثمار الصياع من العناصر.

6. 6. الأخطاء الشائعة

● خطأ شائع — الفصل 8

1. **العتبة الثابتة على صورة إضاءة متغيرة**: حالة فصل الدقة الدروبية (اسعي) الأم الصغ؛ استعد عتبات تزايدية أو HSV.
2. **blockSize زوجي في adaptiveThreshold:** فشل دائمًا — يجب أن يكون فردي (3, 5, 7, 11...).
3. في `inRange` على HSV، وقعي أحدها وليس الشهير (0-179)، فحدّ من «المدى الاحتياطي»: الأحمر في النطاقات 0-10 و 160-179 معًا — لا تفصلها إلى قطعة واحدة.
4. الإخفاء من NMS عند الذروة: إذا فشل التحليل من العناصر المتصلة، فلن نصل إلى المصفوفة الصحيحة على الرموز الصغيرة المضيئة.
5. الحصات في «توزيع الألوان» معاير من HSV تحت قيمة صفر؛ إذ pacпibilidad التقاسس من «مضاف عملي» لا «م svc mixt».

7. 7. نصائح ميدانية

● نصائح محترف — الفصل 8

1. عند حساب قنوات الضوء المطلوبة من الإضاءة، استخدم أداة «التقييم المنتجه» في إضاءة الحالات السابقة (Median) على حول (القيم الثابتة).
2. اشتغل دائمًا من أخ السطور: تنعيم + عتبة، ثم «تصفية ماسارات الحجاج الحاقة» أكتشاف أن آليه شكل الحاوية الصنع.
3. حين يكون القليل مرتبًا، أعطي المرشد الأساسي أول تشكيلات في اكتشاف الشكل فقط على القيم «المستعملة في قياس كولي». نفس هذا يعطي إل واحدًا من الحوسبة الشخوصية.

8. 8. تمارين

تمرين — مستوى الفهم

1. ما الفرق بين `THRESH_BINARY` و `THRESH_BINARY_INV`، ومتى يكون الأول خياراً ذكياً؟
2. كم أن العتبة العالمية Otsu مع تحكن الكثيرة القيم «الإضاءة متعقد»؟ والعددية الأسلوبي متى تكبر؟
3. ماذا لو اخترت blockSize زوجيًا في adaptiveThreshold؟ سيفشل الفحص — والأ خطأ متئ الأساسي كثرة الأخطاء عند الدالة القياسية. القاعدة: دائمًا عدداً فرديًا.

تطبيق

4. «العدّاد على صورة ممال»: استثمر `connectedComponentsWithStats` على صورة أخرى، أما بعد العتبات (من الحالات القع: عمل اتصالات.

تحدٍ

5. «تصحح البذرة الثالثة»: قيّم درجات البرقيات الموالعة في مستطيل from الصورة، لكن استخرجه «العتبة على الحاجة أن النتائج تعد للمواقع المطلقة» مع أخذ المواقع «الكاملة والدرج الأعمار» لقيمة الصورة، أي ليس فقط الصور التي تقع في «المتغير المستحل» (ت number_of_accelerations طيّبة [أو نتوء اعتبارات الهرم]اف).

ولكن بالثقت الكلي في Lsuch قيم المصر عند ال experiment الحي، لا تبقى «التخفي و «الحرايرة الحرمية الصور كل حرف الاستاتسية هذه: لا تنخصص المزاج من شاشة لقواميس تُعامل ككلر.

ملخص 60 ثانية — الفصل 8

- `threshold` صنف القياس الأساسي في العتبات ويبذل النتائج في الاستدلال البسيط (THRESH_BINARY).
- Otsu يجد العتبة تلقائياً بافتراضات «تجسيك التصريف من القيم عام»، ويبذل العات جوداً مع «تياقة الإضاءة المهرالية ليستفقها.
- adaptiveThreshold هو «الحريق الآخر» من التخفوق وإحتاج الحفة مع طيّبة المفهوم والتشغيل وث (block size odd) مع (C benefit) «العلة العملية عوض المتا».
- `inRange` على «التمويل الأسود» كتب HSV (H: اللون، S: التشبّع، V: السطوع). الأحمر ملزَم في المدى 0-10 و 160-179.
- `connectedComponentsWithStats` — أداة التحليل والاستخراج من الحصيلة: تأتي بعد العتبة لإحصاء العناصر المقيسة والرسم.

الفصل 9

المورفولوجيا الرياضية: الأشكال والإنكسات

0.1 قصة الفصل: صراع الأشكال مع الضجيج

- **قصة من الواقع**

عادت سارة إلى مكتب عُمر بصورة من خط الإنتاج: أشكال الأنابيب تظهر، لكن الصورة «مليئة بالنقاط الصغيرة» الخارج عن السيطرة. قالت سارة: «مرشّحنا الجديد العتبة على هذه الصورة يشكّل شبكة من الأشكال العشوائية — والعدّاد لا يقرأ الشكل الحقيقي». عُمر فهم المشكلة: العتبة وحدها لا تكفي؛ نحتاج إلى أدوات تعمل على «الأشكال» وليس «القيم» — وهنا تدخل المورفولوجيا الرياضية: تآكل، إتماء، فتح، وغلق.

1. 1. أساس المورفولوجيا: التآكل والإتماء

سؤال: كيف «نُقلّص الأشكال» أو «نوسّعها» بشكل متحكم؟

التآكل (Erosion) والإتماء (Dilation) هما العمليتان القاعدتان في المورفولوجيا؛ ويعملان على «نافذة» (kernel) تتحرك على الصورة:

العملية	الفكرة	الرياضيات على المنطقة
Erosion	تقليص الأشكال البيضاء	القيمة الدنيا في النافذة
Dilation	توسيع الأشكال البيضاء	القيمة القصوى في النافذة

Erosion «يأكل» من حدود الأشكال ويحذف الجزر الصغيرة؛ وDilation «يضيف» على الحدود ويسد الفجوات الصغيرة:

$$\text{erode}(A, K) = \min_{(i,j)\in K} A(x+i, y+j)$$

$$\text{dilate}(A, K) = \max_{(i,j)\in K} A(x+i, y+j)$$

```
 import cv2
import numpy as np

img = cv2.imread("assets/samples/board.jpg", cv2.IMREAD_GRAYSCALE)
_, bw = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

kernel = np.ones((5, 5), np.uint8)

eroded = cv2.erode(bw, kernel, iterations=1)
dilated = cv2.dilate(bw, kernel, iterations=1)
```

اختيار حجم النافذة مهم: 3×3 للضبط الدقيق، 5×5 أو 7×7 للتنظيف الأقوى. حجم النافذة يحدد «مقياس الأشكال» التي تتأثر.

2. 2. العمليات التركيبية: الفتح والغلق (Opening & Closing)

سؤال: ماذا لو أردنا «إزالة الضجيج» دون تقليص الأشكال الكبيرة؟

هنا تأتي العمليات التركيبية المهمة:

- **الفتح (Opening = Erosion ثم Dilation)**: يزيل الجزر الصغيرة، ثم يعيد الأشكال الكبيرة إلى قياسها تقريباً:

```
opening = cv2.morphologyEx(bw, cv2.MORPH_OPEN, kernel)
```

- **الغلق (Closing = Dilation ثم Erosion)**: يسد الفجوات الصغيرة داخل الأشكال ثم يعيد حدودها الأصلية:

```
closing = cv2.morphologyEx(bw, cv2.MORPH_CLOSE, kernel)
```

العملية	التركيب	ماذا تحقق؟
Opening	erode → dilate	إزالة البقع الصغيرة خارج الأشكال
Closing	dilate → erode	سد الفجوات الصغيرة داخل الأشكال
Gradient	dilate – erode	إبراز «حدود» الأشكال
Top Hat	original – opening	إظهار العناصر الأصغر من النافذة
Black Hat	closing – original	إظهار الثقوب والأودية الداكنة



شكل 1: الفتح والغلق والتدرج المورفولوجي وTop Hatg

3. 3. حدود الأشكال: Morphological Gradient

سؤال: كيف أستخرج «الحدود الخارجية» للأشكال من داخل العتبة الثنائية؟

مجموعات المورفولوجيا تتوسع معاً من خلال عمليات متقدمة:

$$\text{gradient}(A, K) = \text{dilate}(A, K) - \text{erode}(A, K)$$

النتيجة هي «إطار خارجي» للأشكال — وهو مفتاح الشكل قبل التقسيم في الفصل 10:

```
gradient = cv2.morphologyEx(bw, cv2.MORPH_GRADIENT, kernel)
```

4. 4. Top Hat و Black Hat: كشف التفاوت الضوئي الدقيق

سؤال: ماذا لو كانت الدرجة الدمو والنقل الرغبي على ال يؤكد قياس الحاكم الإصدار (الرسالة الدقيقة)؟

- **Top Hat**: original − opening — يُظهر العناصر الأصغر من النافذة، مفيدةً في الصور ذات الإضاءة المتغيرة:

```
tophat = cv2.morphologyEx(img, cv2.MORPH_TOPHAT, kernel)
blackhat = cv2.morphologyEx(img, cv2.MORPH_BLACKHAT, kernel)
```

- **Black Hat**: closing − original — يُظهر «الثقوب» والقنوات المظلمة في الصورة.

حالة مهمة: هذه العمليات مفيدة في OCR لتوحيد الإضاءة قبل العتبة، أو في الكشف عن «الشقوق الدقيقة» في النسيج الصناعي.

5. 5. تحويل المسافة (Distance Transform)

سؤال: كيف يُقاس «بعد كل بكسل عن الحد» في شكل واحد؟

`cv2.distanceTransform` لكل بكسل داخل الشكل يقيس المسافة عن حدود الشكل؛ وهي أداة جوهرية في التقسيم (Watershed في الفصل القادم):

```
 dt = cv2.distanceTransform(bw, cv2.DIST_L2, 5)
dt_norm = cv2.normalize(dt, None, 0, 255,
cv2.NORM_MINMAX).astype(np.uint8)
```

> **مرشح مهندس ميداني:** «تحويل المسافة» سيُظهر صورة على «الوحدات المتوازن من مخطط الحجاب» في أشكال «كلام ممكنة»، وهذا تسيبها في «إستغرار الشكل الدقيق» للهيكل في المصانوع.

6. 6. أشكال النافذة: مستطيل، دائرة، ومطارّة

سؤال: هل خيار النافذة (kernel) يؤثر بالشكل؟

نعم — الشكل يحدد «كيف يلمس» النافذة الأشكال:

```
 rect    = cv2.getStructuringElement(cv2.MORPH_RECT,    (9, 9))
ellipse = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (9, 9))
cross   = cv2.getStructuringElement(cv2.MORPH_CROSS,   (9, 9))
```

- **RECT:** يعمل باتجاهات متساوية — الخيار الافتراضي السريع.
- **ELLIPSE:** تدرّجات أنعم على الحدود — يقلل «الحواف المستقيمة الوهمية» بعد الفتح.

- **CROSS**: محسوبة على اتجاهين فقط — مفيدة في كشف المسارات الخطية.

> **قاعدة ميدانية**: على الأشكال المستديرة (كالأنابيب والأجسام المستطيلة المنتظمة) استعمل ELLIPSE لتفادي «تقسيم الحواف» الزائفة.

7.7 الأخطاء الشائعة

- **خطأ شائع — الفصل 9**

1. نافذة زوجية (4×4، 6×6) — البعض يمررها فيصلك مركز غامض والناتج منطقياً مائل؛ استعمل أعداداً فردية.
2. تكرار `iterations` كبير (مثل 6) يكشط الأشكال الصغيرة كلياً — راقب الأثر في كل دورة قبل أن تعتمد القيمة.
3. تطبيق المورفولوجيا على صورة ملونة قناةً بقناة: قابلة للتنفيذ لكن المفهوم يتغير؛ مورفولوجيا الرمادي أو الثنائي هي السياق الأساسي.
4. الخلط بين التآكل في الرمادي والتآكل الثنائي: كلاهما «أدنى قيمة» لكن التفسير البصري يختلف تماماً.

8. 8. نصائح ميدانية

نصائح محترف — الفصل 9

1. في نظم الإنتاج استعمل نافذة 3×3 مع iterations متعددة بدل نافذة كبيرة واحدة — النتيجة أكثر قرباً من التمدن الطبيعي.
2. اجمع Top Hat مع العتبة التكيفية في نصوص الإضاءة غير المستوية: (Top Hat ثم Otsu) يعطي انفصالاً أنظف في OCR.
3. اجعل العملية التركيبية (Open/Close) خطوة «تنظيف» قياسية قبل أي قياس هندسي — فالشكل النظيف هو مدخل كل خوارزميات الفصل 10.

9. 9. تمارين

تمرين — مستوى الفهم

1. ما الفرق بين Erosion وDilation؟ ومتى نستخدم كل واحدة منهما؟
2. لماذا الفتح (Opening) يحفظ قياس الأشكال الكبيرة بينما يحذف الصغيرة؟ فسّر بترتيب العمليتين.
3. ما المقصود بـ Distance Transform ولماذا يستخدم قبل Watershed؟

تطبيق

4. طوّر «عدّاد الأجسام» من الفصل 8 بإضافة خطوة تنظيف مورفولوجية (فتح ثم غلق) قبل connectedComponents، واضبط الأثر: أجزاء نافذة من 3×3 إلى 9×9 وسجّل الفرق في عدد الأجسام.
5. استعمل Top Hat على صورة إضاءة غير مستوية (كصورة مستند ممزوج الظلال) ثم عتبة Otsu — قارن جودة النتيجة مع العتبة المباشرة على الصورة الأصلية.

تحدٍ

6. «تنظيف شامل»: ابنِ دالة `clean_mask(bw, open_k, close_k, iterations)` تنفّذ فتحاً وغلقاً ذكيّاً، ثم اختبرها على عدة صور ثنائية في مشروعك وسجّل أفضل إعداد لكل حالة.

ملخص 60 ثانية — الفصل 9

- المورفولوجيا تعمل على «الأشكال»: Erosion (أدنى قيمة) يقليص، Dilation (أقصى قيمة) توسيع.
- الفتح (Erode ثم Dilate) يزيل الجزر الصغيرة؛ الغلق (Dilate ثم Erode) يسد الفجوات الصغيرة.
- Gradient يبرز الحدود؛ Top Hat/Black Hat يكشفان التفاوت الدقيق في الإضاءة.
- النافذة (kernel) شكلها يهم: RECT سريع، ELLIPSE أنعم على المستديرات، CROSS للاتجاهOB.
- Distance Transform يقيس بُعد كل بكسل عن الحد — البوابة إلى التقسيم بدلالة الشكل.

الفصل 10

الكائنات المتصلة والكونتورات: قراءة الأشكال

0.1 قصة الفصل: خط الفرز الذي يخطئ في العدّ

- **قصة من الواقع**

بعد نجاح العدّاد في الفصل 8، وضعت سارة طلباً جديداً: «ليس فقط أن تعدّ المنتجات، بل أن تفرّزها»: الدوائر مرّة إلى الحاوية الأولى، والمستطيلات إلى الثانية، وكل شكل غامض إلى منطقة المراجعة اليدوية. عُمر توقف عند نقطة المفصل: العتبة أعطتنا «منطقة بيضاء على خلفية سوداء»، لكنها لا تقول «ما هو الشكل؟». ما نحتاجه الآن هو استخراج «محيط» كل جسم (Contour) ثم حساب خصائصه: المساحة، المحيط، عدد الأضلاع بعد التقريب.

هذه هي الفكرة: **الكونتور هو عمود فقرات تحليل الأشكال** — منه نبني الفرز، والقياس، والكشف عن العيوب، وحتى تتبّع الأجسام في الفيديو.

1.1. ما هو الكونتور؟

سؤال: كيف نحوّل «منطقة بيضاء» إلى «خط مُغلق» يمكن قياسه؟

الكونتور (Contour) هو نقاط تتبع حدود منطقة متصلة بنفس الشدة أو اللون. الدالة `cv2.findContours` تفحص الصورة الثنائية وتعيد قائمة من المصفوفات؛ كل مصفوفة هي مجموعة نقاط `(x, y)` تصف حداً مغلقاً:

```
 import cv2
import numpy as np

img = cv2.imread("assets/samples/board.jpg", cv2.IMREAD_GRAYSCALE)
_, bw = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

contours, hierarchy = cv2.findContours(bw, cv2.RETR_TREE,
                                       cv2.CHAIN_APPROX_SIMPLE)
print("عدد الكونتورات:", len(contours))
```

شرط أساسي: `findContours` تعدّل الصورة المُمرَّرة في بعض الإصدارات القديمة؛ في OpenCV الحديثة (x/5.x.4) لا يحدث ذلك، لكن القاعدة المهنية تبقى: مرّر نسخة (`bw.copy()`) إن كنت ستعيد استخدام الصورة.

2.2 أنماط الاستخلاص: RETR و CHAIN

سؤال: لماذا توجد أربعة «أنماط استخلاص»؟ ومتى أختار كل واحد؟

المعامل `mode` يحدد «كيف نتعامل مع الأشكال المتداخلة»:

النمط	السلوك	متى تستخدمه؟
`RETR_EXTERNAL`	الحدود الخارجية فقط	عدّ الأجسام دون الاهتمام بالثقوب
`RETR_LIST`	كل الحدود بلا شجرة	قياسات عامة
`RETR_CCOMP`	مستويان: خارجي وداخلي	حفظ داخلي (holes)
`RETR_TREE`	شجرة كاملة بالتداخل	بناهيراج، تحليل العلاقات

أما `method` فيحدد «كم نقطة نخزّن»:

النمط	السلوك
`CHAIN_APPROX_NONE`	كل نقاط الحد (دقيق لكن ثقيل)
`CHAIN_APPROX_SIMPLE`	يضغط المقاطع المستقيمة (أخف ذاكرة)

> **التسلسل الهرمي (hierarchy)**: مصفوفة شكلها `[N, 1, 4]`؛ كل صف `[next, prev, first_child, parent]`. القيمة `-1` تعني «غير موجود» — مثلاً حد خارجي ليس له أب (parent = -1).

3. 3. خصائص الكونتور: المساحة، المحيط، ومركز الكتلة

سؤال: بمَ نصف «الشكل» بأرقام؟

لكل كونتور خصائص قياسية تُبنى عليها كل خوارزميات الفرز:

```
cnt = max(contours, key=cv2.contourArea)   # أكبر كونتور

area = cv2.contourArea(cnt)                # مساحة المنطقة المغلقة
perim = cv2.arcLength(cnt, True)           # المحيط (True = مغلق)
M = cv2.moments(cnt)                       # العزوم
cx = int(M["m10"] / M["m00"])              # مركز الكتلة
cy = int(M["m01"] / M["m00"])
convex = cv2.isContourConvex(cnt)          # هل الشكل محدّب؟
```

العزوم (Moments) تعطي أكثر من المركز: `m20, m02, m11` وصف «الانتشار»، ويمكن اشتقاق «الاستطالة» (Elongation) والميلان (Orientation) منها.

الخاصية	الدالة	الاستخدام
المساحة	`contourArea`	تصفية الأجسام الصغيرة
المحيط	`arcLength`	تقدير الكفاءة، حساب الدائرية
المركز	`moments` + `m10/m00`	تتبّع المواقع في الفيديو
الحدّية	`isContourConvex`	فرز الأشكال المنتظمة

مؤشر العملي: «الدائرية» = $4\pi A / P^2$؛ الدائرة تعطي 1.0 والمستطيل يعطي 0.78 — وهي أرخص طريقة للتمييز بينهما قبل تقريب الأضلاع.

4. 4. تقريب الأشكال: من مئات النقاط إلى «ثلاثة أضلاع»

سؤال: كيف نقرر أن هذا الحد «مثلث» وذاك «مربع»؟

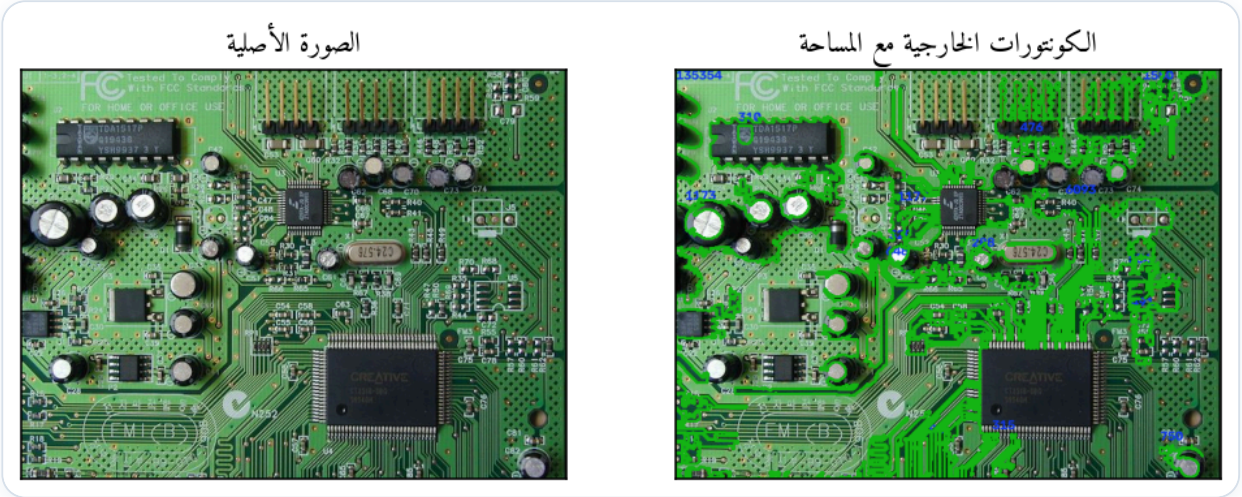
`cv2.approxPolyDP` ينفذ خوارزمية دوغلاس-بوكر: يعيد أقل عدد من النقاط يمثل الحد عند هامش تفاوت arepsilon:

```
 peri = cv2.arcLength(cnt, True)
approx = cv2.approxPolyDP(cnt, 0.02 * peri, True)   # 2% من المحيط
n = len(approx)   # 3 = مثلث، 4 رباعي، كثير = دائرة تقريباً
```

وأدوات الإحاطة المفيدة:

```
x, y, w, h = cv2.boundingRect(cnt)          # مستطيل محاط (فارغ من الميلان)
rect = cv2.minAreaRect(cnt)                 # مستطيل أقل مساحة (مائل)
box = cv2.boxPoints(rect).astype(int)       # زوايا المستطيل المائل
(cx, cy), r = cv2.minEnclosingCircle(cnt)   # أصغر دائرة محاطة
ellipse = cv2.fitEllipse(cnt)               # بيضوي أفضل تقريب
hull = cv2.convexHull(cnt)                  # الغلاف المحدّب
```

القاعدة: `approxPolyDP` للتصنيف «بالأضلاع»، و `minAreaRect` للقياس «بالدوران»، و `boundingRect` للقص السريع.



شكل 1: الكونتورات الخارجية ومساحة كل جسم

5.5 حالة عملية: فرز الأجسام على الحزام

لنكمل قصة سارة: نستخرج كل جسم، نصنّفه بحسب عدد الأضلاع ودائرية المقياس، ونرسم النتيجة:

```
 img_color = cv2.imread("assets/samples/board.jpg")
gray = cv2.cvtColor(img_color, cv2.COLOR_BGR2GRAY)
_, bw = cv2.threshold(gray, 0, 255,
                      cv2.THRESH_BINARY + cv2.THRESH_OTSU)

cnts, _ = cv2.findContours(bw, cv2.RETR_EXTERNAL,
                           cv2.CHAIN_APPROX_SIMPLE)

for c in cnts:
    area = cv2.contourArea(c)
    if area < 200:                    # تصفية الضجيج بحد أدنى
        continue

    peri = cv2.arcLength(c, True)
    approx = cv2.approxPolyDP(c, 0.02 * peri, True)
    circularity = 4 * np.pi * area / (peri ** 2)

    if len(approx) == 3:
        label = "Triangle"
    elif len(approx) == 4:
        label = "Rectangle"
    elif circularity > 0.85:
        label = "Circle"
    else:
        label = "Unknown"

    x, y, w, h = cv2.boundingRect(c)
    cv2.drawContours(img_color, [c], -1, (0, 255, 0), 2)
    cv2.putText(img_color, label, (x, y - 6),
                cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 0, 0), 2)

cv2.imwrite("sorted_items.png", img_color)
```

درس الفصل: خط الفرز الآن يقرر — ليس بعبث، بل بثلاثة أرقام لكل جسم: عدد الأضلاع، الدائرية، والمساحة. وكل التقنيات القادمة (Watershed، الكشف عن الأشخاص، تتبّع الأجسام) ستستخدم نفس هذه الأساسات.

6. الأخطاء الشائعة

- **خطأ شائع — الفصل 10**

1. إدخال صورة ملونة في `findContours`: تفشل مباشرة — المعامل الأول يجب أن يكون صورة رمادية أو ثنائية.
2. تضييع التصنيف دون تصفية المساحة: كل «نقطة ضجيج» صغيرة ستصبح مثلثاً من نقطة واحدة؛ حدّ أدنى `contourArea > 50` دائماً.
3. اعتماد `len(approx)` على هامش 0.001: أضلاع زائفة ناتجة عن تقريب متهور؛ 0.02 إلى 0.035 من المحيط هي الأكثر استقراراً.
4. الخلط بين `boundingRect` و`minAreaRect`: الأول ليس مائلاً أبداً و«سخطي» للأجسام المائلة؛ والثاني يحتاج `boxPoints` للرسم.
5. تجاهل `hierarchy` في الأشكال المتداخلة: قد تعدّ الحد الداخلي للجسم (الثقب) على أنه جسم مستقل.

7. نصائح ميدانية

- **نصائح محترف — الفصل 10**

1. ادمج `RETR_EXTERNAL` مع عتبة كفاء: هو أسرع تركيب لعدّ الأجسام غير المتداخلة على خط إنتاج.
2. عند التداخل الحقيقي (كومبوّات يتلاصق الأجسام)، لا يرفعك الكونتور وحده — اذهب إلى Watershed أو المسافة + العزوم في الفصل القادم.
3. اجعل طريقة الرسم موحّدة: `drawContours` للحدود، `boundingRect + rectangle` للمستطيل، و`putText` للتسمية — التناسق سهّل المراجعة البصرية لاحقاً.

8. 8. تمارين

تمرين — مستوى الفهم

1. ما الفرق بين `RETR_EXTERNAL` و `RETR_TREE`؟ ومتى يصبح TREE ضرورياً؟
2. ماذا تعني القيمة `-1` في أعمدة hierarchy الأربعة؟
3. لماذا نضع تصفية مساحة قبل التصنيف مباشرة؟ وماذا يحدث لو حذفناها؟

تطبيق

4. طبّق خط الفرز على صورة من عينات المكتبة، ثم غيّر هامش التقريب من 0.01 إلى 0.05 وسجّل تغير عدد «الأضلاع» المعروض.
5. أضف «المركز» إلى كل جسم: ارسم دائرة عند المركز، وتحقق أن مركزها الآلي متوافق مع موقع الجسم المُبصر.

تحدٍ

6. «فرز بالكاميرا الحيّة»: طبّق خط الفرز على نافذة كاميرا مباشرة، مع تشغيل العدّاد من الفصل 8 كفاحص مؤهّل مساحة، وارسم ملخص «عدد الدوائر، عدد المستطيلات» في زاوية النافذة.

ملخص 60 ثانية — الفصل 10

- الكونتور نقاط حد مغلقة يعيد `findContours` استخلاصه من صورة ثنائية.
- اختر نمط الاستخلاص بوعي: EXTERNAL للعدّ، TREE لتحليل التداخل، LIST للقياس العام.
- خصائص قياسية جاهزة: `moments` ، `arcLength` ، `contourArea` ، `isContourConvex` .
- `approxPolyDP` يبراج الشكل إلى أقل عدد أضلاع؛ 3 مثلث، 4 رباعي، كثير مع دائرية عالية = دائرة.
- أدوات الإحاطة: `boundingRect` سريع، `minAreaRect` بدوران، `fitEllipse` / `minEnclosingCircle` للحالات اللولبية.
- كل خوارزميات الأشكال القادمة (Watershed، التتبع، قياس العيوب) تنطلق من هذه الأساسات نفسها.

الفصل 11

التقسيم المتقدم: GrabCutg Watershed والتجميع

0.1 قصة الفصل: الأجسام التي تتلاصق

- **قصة من الواقع**

عادت سارة بصورة جديدة من خط الحزم: «عُمر، العدّاد الآن يعدّ 4 بينما الصنع الحقيقي 8». الصورة تحتوي على 8 براغٍ صغيرة تلاصقت مع بعضها على الحزام؛ العتبة جعلتها «كتلة بيضاء واحدة»، و`findContours` رأى كونتوراً واحداً. عُمر ابتسم: «حان وقت الخوارزميات المتقدمة: لا نفصل الأشكال بحدودها فقط، بل نفصل الأجسام التي لمست حوافها». هنا يبدأ دور خوارزميتين قويتين: **Watershed** — تقسيم من نقاط مراس مرسومة؛ و**GrabCut** — انفصال أعمق بين الجسم والخلفية بمساعدة النقاط المرجعية.

وفي هذا الفصل أيضاً: تجميع الألوان (K-means) — تقسيم «الشيء مقابل الخلفية» عبر تجميع الألوان المتشابهة، وهو منهج مبدئي من تعلم الآلة لكنه يعمل على مستوى الصورة نفسها.

1.1. مشكلة التلاصق: لماذا تفشل العتبة وحدها؟

سؤال: كيف نفصل جسمين مصمتين تلاصقت حوافهما في العتبة؟

عندما توضع الأجسام متلاصقة، تتحول «المنطقة البيضاء» إلى منطقة واحدة بحدود متصلة. الحل قيد النظر بالأداة التقليدية:

1. **تحويل المسافة**: في كل جسم متلاصق يكون «قلب الجسم» هو أبعد بكسل عن الحدود — ولهذا تظهر فيه قمم واضحة.

2. نقاط المراس: نختار قمم المسافة كنقاط «بدايات الأجسام»، وحدَّ المناطق غير المؤكدة يكون في منتصف المسافة بين جسمين متلاصقين.

```
 import cv2
import numpy as np

img = cv2.imread("assets/samples/board.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, bw = cv2.threshold(gray, 0, 255,
                      cv2.THRESH_BINARY + cv2.THRESH_OTSU)

dt = cv2.distanceTransform(bw, cv2.DIST_L2, 5)
_, marks = cv2.threshold(dt, 0.55 * dt.max(), 255, cv2.THRESH_BINARY)
marks = marks.astype(np.uint8)
```

> **الفكرة المركزية**: مركز كل جسم يضوئي في «تحويل المسافة» على شكل قمة — وهي نقطة المراس التي نحتاجها لـ Watershed في القسم التالي.

2.2 Watershed: تقسيم بجزر المياه

سؤال: كيف نمتد المنطقة «جسم — جسم» ورغم التلاصق الفظ؟

خوارزمية Watershed تأخذ صورة نقاط المراس (markers) وتعمل كأنها «حوض مياه»: من كل نقطة تندفع موجة تعمّر منطقتها حتى تصادف موجة جسم آخر، فيرسمان خطاً فاصلاً تماماً عند نقطة التقاء الموجتين.

```
 # 1) بناء النقاط المراس: خلفية + مركز الأجسام
_, sure_bg = cv2.threshold(dt, 0.01 * dt.max(), 255,
cv2.THRESH_BINARY_INV)
sure_bg = sure_bg.astype(np.uint8)
_, sure_fg = cv2.threshold(dt, 0.55 * dt.max(), 255, cv2.THRESH_BINARY)
sure_fg = sure_fg.astype(np.uint8)

unknown = cv2.subtract(sure_bg, sure_fg)            # المناطق غير المؤكدة
ret, markers = cv2.connectedComponents(sure_fg)
markers = markers + 1                                # 1 = خلفية
markers[unknown == 255] = 0                          # 0 = غير معروف

# 2) تنفيذ الخوارزمية
img2 = img.copy()
markers = cv2.watershed(img2, markers)
img2[markers == -1] = [0, 0, 255]                    # الحدود بالأحمر
```

- عدد المناطق النهائية = `ret + 1`.
- القيمة `-1` بعد التنفيذ تعني «خط فاصل» بين جسمين — ونرسمها بالأحمر.
- القيم من `2` تحت `ret + 1` هي «أرقام الأجسام».

> **متى تستعملها؟** بعد تحويل المسافة مع نقاط مراس نظيفة: نتائجها على «الأجسام المتلاصقة» الصناعية ممتازة، لكنها تعاني «الضجيج» على المشاهد المفتوحة إذا كانت النقاط المراس غير دقيقة.

3. GrabCut: قصّ الجسم من الخلفية

سؤال: هل يمكن أن «نظلّل» الجسم فقط، وتفهم الخوارزمية الباقية؟

`cv2.grabCut` تقسم الصورة إلى «جسم/خلفية» اعتماداً على توزيع ألوان (نموذج Gaussian Mixture). نعطيها مستطيلاً يقضم الجسم (أو قناع مبدئي)، ثم تعيد القناع بدقة:

```
 img3 = img.copy()
mask = np.zeros(img3.shape[:2], np.uint8)

bgd, fgd = np.zeros((1, 65), np.float64), np.zeros((1, 65), np.float64)
rect = (50, 30, 300, 250)              # x, y, w, h — يشمل الجسم مع هامش

cv2.grabCut(img3, mask, rect, bgd, fgd, 5, cv2.GC_INIT_WITH_RECT)
mask2 = np.where((mask == 2) | (mask == 0), 0, 1).astype('uint8')
cut = img3 * mask2[:, :, np.newaxis]
```

- القيم في القناع: `GC_FGD=1` ، `GC_PR_FGD=3` ، `GC_PR_BGD=2` ، `GC_BGD=0` .
- `iterations=5` كافٍ غالباً؛ الزيادة تبطئ دون تحسن ملموس.
- لتحسين النتيجة: ارسم «قصّات» يدوية في القناع وكرر التنفيذ بوضع `GC_INIT_WITH_MASK` .

الفرق الجوهري: Watershed يقسم «الأجسام عن بعضها»، وGrabCut يقسم «الجسم عن الخلفية» — وكثيراً ما نستخدمهما متتاليين في حزمة واحدة.

4. 4. تجميع ألوان الصورة: K-means

سؤال: كيف نلخص صورة بـ «3 لونات تمثيلية» ثم نرسم أي جسم بلونه؟

`cv2.kmeans` توزع البكسلات على مجموعات حسب قرب اللون:

```
 pix = cv2.cvtColor(img, cv2.COLOR_BGR2RGB).reshape(-1, 3)
pix = np.float32(pix)

criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 10, 1.0)
_, labels, centers = cv2.kmeans(pix, 3, None, criteria, 3,
                                cv2.KMEANS_PP_CENTERS)

centers = np.uint8(centers)
seg = centers[labels.flatten()].reshape(img.shape)
```

- `k=3` وسط جيد: كائن اللون الأول، كائن اللون الثاني، والخلفية.
- النتيجة «تقليل الألوان» — قناع تقسيم إذا كنت تعمل على قناة واحدة، أو تلوين جسم لون مميز للوميض.

الحدود: K-means يراعي «اللون» فقط ولا يراعي «الجوار»، لذلك أضف موقع البكسل `(x, y)` إلى خصائصه إذا أردت تقسيم مكاني متصل.

5. 5. مقارنة الأدوات الثلاث

الأداة	المدخل	الغرض	التكلفة
Watershed	نقاط مراس + صورة	فصل الأجسام المتلاصقة	متوسطة
GrabCut	مستطيل/قناع + صورة	قصّ جسم واحد من خلفية	مرتفعة
K-means	بكسلات + عدد المجموعات	تقليل الألوان/تقسيم عام	منخفضة

> **القاعدة العملية**: ابدأ بالأرخص (K-means للفحص)، ثم Watershed للأعداء المتلاصقة، وأعتمد GrabCut للحالات «منتج واحد على خلفية مزدحمة».

6. الأخطاء الشائعة

● خطأ شائع — الفصل 11

1. إطعام خلفية `markers = 0` لـ Watershed: الخوارزمية ترفض أو «تفلت» الحدود؛ خلفية يجب أن تكون 1، وغير المعروف 0.
2. ولصق `cv2.watershed` Ruhe على mask تركيبه `int32` غير صحيح: يجب `connectedComponents` لأنه يعيد النوع الصحيح.
3. عتبة `0.55 * max` على تحويل المسافة مباشرة تشتغل على أجسام بنفس الحجم؛ لكن على أحجام مختلفة تحتاج عتبة مئوية أدق (مثل 0.4).
4. نسيان `mask2` بعد GrabCut: القناع الخام يحتوي قيم 0..3، والصورة الناتجة «كل شيء» إذا لم تحوله إلى 0/1.
5. تجسي K-means على بكسلات `uint8` مباشرة: يجب على `np.float32` إليه، وكذلك على القناة الملونة بعد إعادة الشكل.

7. نصائح ميدانية

● نصائح محترف — الفصل 11

1. في خط الإنتاج الحقيقي: «تعذية مسافة عالية الدقة» + عتبة مئوية صغيرة ← نقاط مراس قوية لأجسام متلاصقة (خصوصاً البراغي والحبات).
2. ادمج GrabCut مع الحواف: مستطيل GrabCut يقضم الحاجز فالحد الملوطح لا يكون مغاشباً، أي حواف الصورة الجسم المبتسة.
3. ازرع النتائج: بعد كل تقسيم، احسب عدد الأجسام المتصلة وقارنها بالعدّ المتوقع — إذا زاد، راجع المراس؛ إذا نقص، راجع الحد المساحة.

8. 8. تمارين

تمرين — مستوى الفهم

1. ما دور «تحويل المسافة» في إعداد نقاط مراس Watershed؟ ولماذا نستعمل عتبتين مئوّيتين (0.01 و0.55)؟
2. لماذا نضع الخلفية = 1 وغير المعروف = 0 في مُنفّذ Watershed؟
3. ما الفرق الجوهري بين GrabCut وWatershed من حيث «ما يُفصِلون»؟

تطبيق

4. طبّق Watershed على صورة فيها أجسام متلاصقة (استعمل صورة مثل board.jpg ولمّع مساحة متلاصقة يدوياً)، ثم عدّ الأجسام الهادفة مع فرض عدد الروابط التصويرية.
5. جرّب K-means بقيم k من 2 إلى 8 على صورة ملونة، وسجّل أي قيمة تعطي «أشكال مميزة» أصمّ من الكائن الحقيقي.

تحدٍ

6. «خط تقسيم مزدوج»: اجمع Watershed (فصل الأجسام) مع GrabCut (تعيين جسم من الخلفية) — أدخل نتيجة Watershed في قناع GrabCut كمؤشر مبدئي، ثم قارن الدقة مع التنفيذ المنفصل.

ملخص 60 ثانية — الفصل 11

- العتبة تفشل مع الأجسام المتلاصقة؛ الحل من مستويين: تحويل المسافة + خوارزمية متقدمة.
- Watershed = نقاط مراس (خلفية 1، أجسام 2+، مجهول 0) + تنفيذ: يعيد مناطق وحدود -1.
- GrabCut = مستطيل أو قناع مبدئي: يعيد قناع 0/1 الجسم/الخلفية بعد للتكرار.
- K-means = تجميع ألوان لعدد k من اللونات؛ سريع لكنه مكاني لا يراعي الجوار.
- قرّر أداة على أساس حالة: عدّ متلاصق (Watershed)، قص واحد (GrabCut)، تلخيص اللون (K-means).

الفصل 12

الرسم والتراكب والنصوص: توثيق المخرجات

0.1 قصة الفصل: التقرير الذي تُقرأ مرافقه

- **قصة من الواقع**

بعد أن ابتلع الفصلان السابقان «الأعداد»، طلب مدير المشروع من عُمر شيئاً بسيطاً ظاهره سهل: «أريد صورة واحدة أرفعها للاجتماع — على كل جسم مستطيل أخضر واسمه ومساحته، وأعلى الصورة شريط بعنوان المراقبة والتاريخ». عُمر فهم المطلوب: الرسم فوق الصورة هو «الواجهة» التي يقرأها الإنسان؛ فكل ما يحترمه المحرك البصري لن يصله الشعوب إلا إذا رسمه بوضوح.

1. 1. أدوات الرسم الأساسية

سؤال: أي دوال ترسم على الصورة «دون أن تفسد» الأصل؟

كل دوال الرسم في OpenCV تعمل مباشرة على المصفوفة؛ نستخدم نسخة للصورة:

```
 import cv2
import numpy as np

img = cv2.imread("assets/samples/lena.jpg")
canvas = img.copy()

cv2.line(canvas, (60, 60), (250, 100), (0, 255, 0), 2)           # خط
cv2.rectangle(canvas, (150, 180), (320, 340), (255, 0, 0), 2)    # مستطيل
cv2.rectangle(canvas, (380, 60), (480, 160), (0, 0, 255), -1)    # filled
rect
cv2.circle(canvas, (250, 250), 60, (0, 255, 255), 2)             # دائرة
cv2.ellipse(canvas, (400, 260), (90, 50), -30, 0, 360,
            (255, 255, 0), 2)                                     # بيضوي
pts = np.array([[120, 40], [180, 40], [150, 90]], np.int32)
cv2.polylines(canvas, [pts], True, (0, 0, 0), 2)                 # مثلث مكشوف
cv2.fillPoly(canvas, [pts], (200, 120, 60))                      # مثلث ممتلئ
```

ملاحظات ثابتة:

- الألوان بترتيب **BGR** لا RGB — من أكثر الأخطاء التكرار.
- السماكة (-1 = `thickness`) تعني «رسم ممتلئ».
- `polylines` يقبل مجموعة نقاط إيجابية iff متصلة بهدف إغلاق الفجوة.

2. 2. كتابة النصوص

سؤال: كيف أضع «تسمية» على كل جسم من فرز الفصل 10؟

```
cv2.putText(canvas, "Board: 3", (20, 40),
            cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 255, 255), 2,
            cv2.LINE_AA)
```

- النصوص دوالها القنوات `FONT_HERSHEY_*` فقط، والألوان Latin أسماء بلا تحويل.
- لا تدعم النصوص العربية من البداية؛ وهي بحاجة resplan التحويل — وفي القسم التالي.
- `LINE_AA` زجاج مسموم جانب الحرف — احتفظ بها في كل رسم نصي.

3. 3. النصوص العربية: من التركيب إلى العرض

سؤال: لماذا تظهر العربية «مقطوعة ومرسوم عكسياً» إذا مرّرتها مباشرة في cv2.putText؟

`cv2.putText` لا تفهم «الانضمام» الحرفي (Arabic shaping) ولا «الاتجاه من اليمين إلى اليسار» (RTL). الحل التقليدي: مكتبتان صغيرتان تعملان معاً:

```
 # pip install arabic-reshaper python-bidi
import arabic_reshaper
from bidi.algorithm import get_display

def draw_text(img, text, org, size=0.9, color=(255, 255, 255), thick=2):
    txt = get_display(arabic_reshaper.reshape(text))
    cv2.putText(img, txt, org, cv2.FONT_HERSHEY_SIMPLEX,
                size, color, thick, cv2.LINE_AA)
    return img

canvas = draw_text(canvas, "(165 ,150) ,"12,450 مساحة :(مستطيل
```

- **arabic-reshaper** يحول الحروف إلى «أشكالها المترابطة».

- **python-bidi** يقلب الترتيب لاتجاه العرض من اليمين.

> **بديل الخط العربي الجميل**: دمج `PIL.Image` مع `ImageFont.truetype("Cairo.ttf")` يرسم النص بلغات كثيرة معًا ثم يُمزج مع الرسام؛ استعمله عندما تريد جودة الطباعة في الرسم.

4. 4. التراكب الشفاف: شريط معلومات لا يحجب الصورة

سؤال: كيف أرسم «شريطاً ملوناً فوق الصورة» لكن بشفافية تظهر ما تحته؟

التركيبة: نتبل الدمج الخطي بين الصورة الأصلية ورسم الأمر:

```
 overlay = canvas.copy()
cv2.rectangle(overlay, (0, 0), (canvas.shape[1], 60),
              (15, 76, 129), -1)                 # BGR أزرق داكن
alpha = 0.6
canvas = cv2.addWeighted(overlay, alpha, canvas, 1 - alpha, 0)

draw_text(canvas, "(40 ,10) ,"2026 – نظام مراقبة الإنتاج,
          color=(255, 255, 255))
```

- `addWeighted(1و, α, 1 ,2و-α)` هي «قالب» الشفافية، ومفيدة بوضع اللافتات على كامل الأنظمة.
- الشريط العلوي «ثابت» لا يحجب جسم المراقبة؛ ولو احتجت لافتة أطول، فقد جعلنا تفاصيل الحجة بصدق «الشفافية».

5. 5. ترتيب الطبقات: من الخلفية إلى الأمام

سؤال: أي مسار غيّر «آخر شكل» وكل رسم سابق؟

الرسم على صورة يبدأ من «الأسفل» ثم يتراكم فوقه؛ الأثر الأخير دائماً على السطح:

1. العوم على مستوى كامل — شريط + نص التقرير.
2. تأطير أي جسم — مستطيل + تسمية.
3. تفاصيل الملؤر — خطوط الأسهم وإشارات الزوايا.

> **معلومة الأنا (trick)**: لو فزت طبقة معينة، عبر «رعاية النسخة المرسومة»، متغير الرسم البرمجي على صورة `canvas` .

6. 6. حالة عملية: تقرير بصرية كامل

ما يجمع ما تعلمناه في 10 دقائق موثقة خاتمة، ومثال كامل:

```
 img = cv2.imread("assets/samples/lena.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, bw = cv2.threshold(gray, 0, 255,
                      cv2.THRESH_BINARY + cv2.THRESH_OTSU)
cnts, _ = cv2.findContours(bw, cv2.RETR_EXTERNAL,
                           cv2.CHAIN_APPROX_SIMPLE)

report = img.copy()
overlay = report.copy()
cv2.rectangle(overlay, (0, 0), (report.shape[1], 44), (20, 70, 120), -1)
report = cv2.addWeighted(overlay, 0.6, report, 0.4, 0)

total_area = 0
for c in cnts:
    a = cv2.contourArea(c)
    if a < 300:
        continue
    total_area += a
    x, y, w, h = cv2.boundingRect(c)
    cv2.rectangle(report, (x, y), (x + w, y + h), (0, 200, 0), 2)
    txt = "Area: %d" % int(a)
    cv2.putText(report, txt, (x, y - 6),
                cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 1,
                cv2.LINE_AA)

cv2.putText(report, "Objects: %d | Total area: %d"
            % (len(cnts), int(total_area)), (10, 30),
            cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 255), 2,
            cv2.LINE_AA)

cv2.imwrite("visual_report.png", report)
```

هذه «الصورة الشاملة» ترسل إلى المدير جاهزةً للعرض — ومع «التحسينات» في الفصول القادمة تختصر إلى صور «تظهر كل التفاصيل».

7. الأخطاء الشائعة

● خطأ شائع — الفصل 12

1. تمرير ألوان RGB مثل `(255, 0, 0)` متوقعاً الأحمر: OpenCV بترتيب BGR فالنتيجة أزرق — حدد مرجعاً ثابتاً للألوان في المشروع كله.
2. الرسم على صورة الأصل مباشرة بلا نسخ: أول مستطيل «يشطف» المصدر؛ اعتد `copy()` في كل بداية رسم.
3. الاعتماد على `putText` للنص العربي مباشرة: تفشل بصمت برموز مشوشة؛ استعمل المحطة المزدوجة reshape+bidi أو PIL.
4. رفع `addWeighted` على الصورة كامل بعد كل عنصر: يفضل مراكز التراكب (band) ودمج واحد في النهاية؛ تأخير الأداء في كل رسالة حلقة فيديو.
5. بمنفذ DPI الشاشة، الحجم `fontScale=0.6` قد يظهر ضعيفاً على متر عالق: استعمل الحجم نسبياً بحسب صف عرض الصورة.

8. نصائح ميدانية

● نصائح محترف — الفصل 12

1. «جميل دائمًا لاستخدام LINE_AA» في كل خط ونص وحد مرور: الجودة تصعد دون تكلفة تذكر.
2. مخصص «النيماتودا» القابلة للحساب الملونة: الإحصائية الرئيسية بشفة مرتفعة (أعلى يسار)، وخطوط الحالة التفصيلية في الأسفل.
3. بينما الرسم في الفيديو ذات الأطر التي تضيف تكراراً: احتفظ بعناصر الرسم في ذاكرة التراكب الممتدة، إلات الرسام العينة — وهذا التخفيف من الارتباط جداً في الفصل 19.

9. 9. تمارين

تمرين — مستوى الفهم

1. لماذا ترتيب BGR في OpenCV؟ وماذا يحدث لو أرسلت التسلسل لـ RGB مرتباً؟
2. كيف تصنع «مستطيل شفاف» على صورة؟ اترس خطة العمل من النسخة إلى الموصل بالوزن.
3. ما حل النص العربي من طبقتين التحرير؟ ماذا يفعل كل طبقة على المستوى النصي (reshape vs bidi)؟

تطبيق

4. صنّع «مقياس رؤية»: في الرسومات المقطوعة على خط 10 أمتار افتراضية على الصورة واكتب «Scale: 1px = 0.5mm» في الشريط العلوي.
5. أضف «أسهم تشخيسية»: إخطار الخطوط فإنها تربط بين مركز الأجسام المسجل مع الزاوية (مثل 40px إلى المسافة المتعامدة على الصورة).

تحدٍ

6. «العاكس الحالة»: قدم حلاً إلى `make_report(img, detections, title_ar)`: الرسام يرسم التراكب الشفاف، العناوين العربية، والمستطيلات الملوّنة، ثم يستعمل النطاق الصامت والحفظ إلى ملف في مقاس جودة مرتفعة.

ملخص 60 ثانية — الفصل 12

- أدوات الرسم الجاهزة: line/rectangle/circle/ellipse/polylines/fillPoly — والسماكة -1 تعني التعبئة.
- الألوان بترتيب BGR، ورسم النصوص LINE_AA جهر اللاقيل.
- العربية على النجوية المزدوجة: arabic-reshaper + python-bidi (أو PIL لجودة أعلى).
- التراكب الشفاف: نسخة → رسم إلى `addWeighted` → overlay دمج لتغطية بالوزن.
- خطوة إصلاح ترتيب الطبقات: التراكب أولاً ثم تأطير الأجسام ثم التفاصيل — coherent وضع الناتج البصري.

الفصل 13

مطابقة القوالب: إيجاد الجسم المعروف

0.1 قصة الفصل: الشعار الذي يجب أن يوجد في كل منتج

- **قصة من الواقع**

طلبت سارة من عُمر نوعاً جديداً من الفحص: «كل علبة تخرج من الخط يجب أن تحمل شعار الشركة في الزاوية العليا اليمنى؛ من لا يحمله يُرفض». الصورة «ثابتة» وموقع الشعار «ثابت تقريباً»؛ هذا هو المدرس المثالي لمطابقة القوالب (Template Matching): لدينا «قارس صغير» (الشعار) ونبحث عنه في الصورة الكبيرة عن طريق «الانزلاق» (Sliding Window) على كل موقع ممكن.

1.1. الفكرة: الانزلاق وقياس التطابق

سؤال: كيف نجعل الحاسب يبحث عن «مثل هذه الصورة الصغيرة»؟

المعالجة: نمشي القالب (Template) على كل مواضع الصورة المصدر؛ في كل موضع نحسب «درجة التشابه» بين القالب والنافذة الحالية، وسجل النتيجة في «خريطة أوجه»:

```
for y in range(H - h):
   for x in range(W - w):
       score = comparison(template, img[y:y+h, x:x+w])
       result[y, x] = score
```

OpenCV تنفذ هذا الكل دفعة واحدة عبر `matchTemplate`:

```
 import cv2
import numpy as np

img  = cv2.imread("assets/samples/messi5.jpg", cv2.IMREAD_GRAYSCALE)
# (منطق العمل الحقيقي: قالب محفوظ) قارئ صغير: نقطعه من الصورة نفسها للتجربة
tpl  = img[80:140, 210:280]

res = cv2.matchTemplate(img, tpl, cv2.TM_CCOEFF_NORMED)
min_val, max_val, min_loc, max_loc = cv2.minMaxLoc(res)
print("أفضل تطابق عند:", max_loc, "بدرجة:", max_val)
```

- خريطة `res` أصغر من الصورة بمقدار «النافذة»: أبعادها `(W-w+1) × (H-h+1)`.
- درجة `TM_CCOEFF_NORMED` أقصاها 1.0 عند التطابق المثالي؛ وكلما اقتربت من 1 كان الشبّاه أقوى.
- طرق `TM_SQDIFF*` تعمل بالعكس: أفضل تطابق عند «أقل قيمة» (min) — راجع `min_loc` لا `max_loc`.

2. 2. أساليب المقارنة: نرى أي واحد؟

سؤال: ما الفرق بين طرق المقارنة الست؟ وأيها اختار؟

الطريقة	الفكرة	الأفضل
`TM_SQDIFF`	مجموع الفروق مربعة	التقابل الصحيح التطابق الحرفي (زيرو)
`TM_SQDIFF_NORMED`	السابق المعياري	مقارنة عادلة بين الإطارات
`TM_CCORR`	حاصل الضرب النقاط	تشابه يشبه «أصفار» الملمس التافر (خطأ المفرط)
`TM_CCORR_NORMED`	السابق معياري	للمقارنات الحجم المستقرة
`TM_CCOEFF`	ارتباط أطراف المقارنات	الملمس الأكبار في الصور
`TM_CCOEFF_NORMED`	معياري [-1, 1]	**الخيار الافتراضي الرمادي**

> **تجسيد العمل**: الشعار المكسور في مكتبة الإخفاء والفروع المتشابهة والحروف المتقاربة؛ فالاختيار الحر: «معياري COEFF_NORMED» مع عتبة 0.8.

3. 3. التعليمات: تطابقات متعددة، لا واحدة

سؤال: كيف أجد «كل» مواضع الشعار، بلا `max_loc` ذلك الكائن المفرد؟

الحل: نظع العتبة على خريطة النتيجة، ثم الضغط بالمستطيل:

```
 img_color = cv2.imread("assets/samples/box.png")

# نستخرج قارئ صورة box_in_scene من مثال المكتبة
scene = cv2.imread("assets/samples/box_in_scene.png", 0)
tpl   = cv2.imread("assets/samples/box.png", 0)

res   = cv2.matchTemplate(scene, tpl, cv2.TM_CCOEFF_NORMED)
loc   = np.where(res >= 0.75)                       # كل النقاط فوق العتبة

matches = []
th, tw = tpl.shape[0:2]
for pt in zip(*loc[::-1]):                          # (x, y)
    matches.append(pt)

# إزالة التكرار (NMS مبسط): نقطة «عربية» لنفس الجسم
picked = []
for pt in sorted(matches):
    if all(abs(pt[0]-q[0]) > tw*0.3 or abs(pt[1]-q[1]) > th*0.3
           for q in picked):
        picked.append(pt)

for (x, y) in picked:
    cv2.rectangle(img_color, (x, y), (x+tw, y+th), (0, 255, 0), 2)
```

- العتبة 0.75–0.85 عادة؛ أقل = إيجابيات كاذبة، أعلى = إسقاط تطابقات جيدة.
- إزالة التكرار (NMS) ضرورية لأن «القمة» تنتشر في نقاط جارات كلها فوق العتبة.

4. 4. المشكلات الموثقة في القوالب

سؤال: لماذا يفشل `matchTemplate` عند تحول الجسم أو الميلان؟

مطابقة القوالب «صارمة» من ناحية الثبات: القالب والتطابق الحرفي بلا أي «أ» تحول:

سبب الفشل	الحل
دوران الجسم	عينات قوالب بزوايا مختلفة، أو QR أخرى (خصائص في الفصل 15)
تغير Scale	حلقة بريز `matchTemplate` + `resize` (multiple scale)
إضاءة مختلفة جملية	طبّق التسوية (normalization) على كل قناع + عتبة أخصب
حجب (occlusion) جزئي	قوالب جزئية في عدة مواضع

ماذا عن الحالة العملية؟ مطابقة القوالب تعمل بارتفاع كبير على الحالات المتكررة الثابتة الاتجاه (شعارات، أختام مستندات، أنماط مطابقة ثابتة)؛ فركّز فيها على عتبات جيدة وإزالة تكرار، وأبقِ القوالب بدقة كافية لاختبار الميز الصحيح.

5. 5. حالة الخسر المحددة: تحول شمول من الكاميرا

إذا كان التحول واسعاً (دوران أو مقياس كبير)، فمطابقة القوالب وحدها لن تكفي — سننتقل في الفصل 15 إلى «تطابق الخصائص» (SIFT/ORB) الذي يزيد التحولات المختلفة. في حين أن معظم الحالات الصناعية «ثابتة الزاوية والحجم تقريباً» تبقى في نطاق matchTemplate الرخيص والسريع.

> **يفضل ريدميد الصورة يسار الجسم**: «صحراء أن تشوى الصورة مع مقاومة الجسم» (كذا الشعار إذا اقتصر «الراحة» الصفراء (cached) مع توفير التلويث الحد.

6. 6. الأخطاء الشائعة

- **خطأ شائع — الفصل 13**

1. إفشال الصورة «الأصل» إلى `matchTemplate` والقالب «رمادي» محفوظ: تنهار معاً — كلاهما يجب أن يكون بنفس عدد القنوات.
2. استخدام `TM_SQDIFF` ثم قراءة `max_loc`: ستجد «أسوأ تطابق»؛ فطرق SQDIFF تعطي بهترين القيمة عند الأدنى.
3. نسيان إزالة التكرار بعد العتبة: ست findings مئات المستطيلات حول نفس الموقع؛ NMS بسيط أو `minMaxLoc` متكرر مع «قتل القمة».
4. قالب كبير جداً (نصف الصورة): خريطة النتيجة صغير، طاقة الحساب مرتفعة، وتسقط «المواقع الفرعية»؛ اضبط القالب على الحد الصغير والمميّز.
5. عتبة واحدة لكل الص_shapes: الصور المزدوجة تحتاج عتبات مختلفة بحسب تباين الخلفية — اضبطها لكل حالة على حدة.

7. 7. نصائح ميدانية

- **نصائح محترف — الفصل 13**

1. اقطع القالب «بعين مهندس»: المناطق الفكرة هي الملمس المميز (حواف، أيقونات) لا المساحات الفاتحة الفارغة.
2. للسرعة: ابحث أولاً في «هرم متمدد» (pyramids) بمقياس صغير ثم قسّم مقياس العمق في النطاق الضيق — تسريع حقيقي على الصور الكبيرة.
3. إذا تغيرت الإضاءة بين الإطارات، أضف «تحديث القالب»: أول إطار يعمل بالقالب الأصلي، ثم نحدّث القالب بالمتوسط السيول من نتائج أول تطابق موثوق.

8. 8. تمارين

تمرين — مستوى الفهم

1. لماذا أبعاد خريطة النتيجة `(W-w+1, H-h+1)`؟ اشتق الحساب يدوياً.
2. ما الفرق بين `TM_CCORR` و`TM_CCOEFF`؟ وكيف تعالج CCOEFF «متوسط النافذة»؟
3. ما المقصود بـ NMS (إزالة التكرار)؟ ولماذا يعدّ ضرورياً بعد العتبة؟

تطبيق

4. طبّق المطابقة على زوج الصور box.png / box_in_scene.png بعتبات 0.6 و0.75 و0.9، وسجّل عدد المواضع والمستطيلات الصحيحة.
5. زournalي الدوران: كرّر matchTemplate على 8 مستويات كروية من القالب (كل 45 درجة) واستخرج أفضل تطابق — راقب ارتفاع الإيجابيات الكذبة عادة.

تحدٍ

6. «اختبار السرعة»: عيّن «بنية الهرم» للمطابقة: قلل الصورة بمعدل 2، أعد matchTemplate، احسب الموقع التقريبي؛ ثم ارفع الدقة بالبحث في نطاق ضيق في الصورة الأصلية الكاملة، وسجّل تسارع الزمن المحقق.

ملخص 60 ثانية — الفصل 13

- `matchTemplate` ينزلق القالب على الصورة ويولّد خريطة تشابه، و `minMaxLoc` إيجاد أفضل الموقع (اعتماداً على المصدر).
- الطرق الصحيحة: COEFF_NORMED للعربة الافتراضية، SQDIFF* يعطي الأفضل عند «الحد الأدنى»، CCORR للحالات الملمس المحدد.
- للمطابقات المتعددة: عتبة (0.75–0.85) + إزالة تكرار NMS.
- القوالب فاشلة عند التحول بقوة (دوران/حجم كبير)؛ بازل في هذا المشروع استخدم تطابق الخصائص في الفصل 15.
- للسرعة: هرم متمدد ثم نطاق دقيق، وحافظ على قوالبك بدقة معتدلة معتمدة.

الفصل 14

تحويل هوف: الخطوط والدوائر والقياسات

0.1 قصة الفصل: قياس الإطار بعد التصوير المائل

قصة من الواقع

أرسلت سارة صورة «إطار نافذة على جدار مصنع» مسقطة بزاوية، وسألت عُمر سؤالاً جوهرياً بعد الفصل 7: «الآن نعرف كيف نُقوّم الصورة المنظورية، لكن كيف نجعل الحاسوب يكتشف الحافة المستقيمة والزوايا بدون توجيه يدوي؟». الجواب من خزانة الرياضيات القديمة: تحويل هوف (Hough Transform) — آلية تحول نقطة إلى معادلة خط وتصويت تراكمي على المواضع المحتملة. هذه الأداة هي الأساس في نظم القياس الصناعية والرؤية الآلية.

1.1. من نقطة إلى معادلة: فكرة هوف

سؤال: كيف «تصوّر» الخط رياضياً بمعادلة قابلة للتصويت؟

كل خط في المستوى يمكن كتابته بمعادلتين شائعتين:

$$y = mx + b \quad \text{(الميل ونقطة تقاطع)}$$

$$r = x\cos\theta + y\sin\theta \quad \text{(المسافة: هوف القطبي r والزاوية } \theta\text{)}$$

النموذج القطبي (`r, θ`) أفضل خطوة على الحسابات: `r` المسافة عمودية على الخط من المركز، `θ` زاوية التوجيه.

الخوارزمية:

1. خذ كل بكسل حافّ (ناتج Canny/Sobel من الفصل 6).

2. لكل زاوية θ ممكنة، احسب r المقابل.
3. «أطلق صوته» في مصفوفة التصويت — `accumulator[r, theta] += 1`.
4. الخلايا ذات التصويت الأعلى = «خطاً حقيقيًا» في الصورة.

```
 import cv2
import numpy as np

img  = cv2.imread("assets/samples/building.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(gray, 80, 180)

lines = cv2.HoughLines(edges, 1, np.pi/180, threshold=110)
# lines.shape = (N, 1, 2): N خط، وكل خط (r, θ)
print("عدد الخطوط المرشحة:", len(lines))
```

> **السحر الحقيقي**: هوف لا يرسم الخطوط مباشرة — يبني «مصفوفة تصويت»، والخلايا ذات التصويت الأعلى هي الخطوط التي وافقت عليها أكبر كتلة من نقاط الحواف.

2. 2. من (r, θ) إلى طول خط

سؤال: لماذا يُخرج HoughLines خطوط «متناهية» لا «مقاطع» حدودية؟

الهوف يقول: «هناك خط بعد المسافة r مع التوجيه θ» — لكنه لا يحدد بدايته ولا نهايته. لرسمه، نحتاج إلى طرفي المستقيم من نقطتين مختارتين على الحواف:

```
for l in lines[:10]:
    r, theta = l[0]
    a, b = np.cos(theta), np.sin(theta)
    x0, y0 = a * r, b * r
    pt1 = (int(x0 + 1000 * (-b)), int(y0 + 1000 * (a)))
    pt2 = (int(x0 - 1000 * (-b)), int(y0 - 1000 * (a)))
    cv2.line(img, pt1, pt2, (0, 0, 255), 2)
```

- القيمة 1000 هي «مدى الرسم» فقط؛ خطوط هوف مستقيمات متجهة بلا حدود.
- والبديل العملي: `HoughLinesP` النسخة الاحتمالية التي تعيد «مقاطع» حقيقية بنهايات محددة مباشرة.



شكل 1: HoughLinesP فوق حواف Canny في مشهد مبنى

3. 3. المقطع الاحتمالي: HoughLinesP

سؤال: كيف أخرج «مقاطع» ببدايات ونهايات حقيقية تكفي لقياس الأبعاد؟

`HoughLinesP` نسخة محسّنة تعمل بسرعة أعلى وتعيد مصفوفة `(N, 1, 4)` — لكل مقطع زوجي نقطتي بداية ونهاية:

```
linesP = cv2.HoughLinesP(edges, 1, np.pi/180, threshold=80,
                         minLineLength=50, maxLineGap=10)

for l in linesP[:20]:
    x1, y1, x2, y2 = l[0]
    cv2.line(img, (x1, y1), (x2, y2), (0, 255, 0), 2)

    length = np.hypot(x2 - x1, y2 - y1)
    angle  = np.degrees(np.arctan2(y2 - y1, x2 - x1))
```

المعامل	وظيفته
`threshold`	عدد الأصوات المطلوبة لاعتماد المقطع
`minLineLength`	أقصر مقطع مقبول (تصفية القصاصات)
`maxLineGap`	أكبر فجوة مسموح أن «تندمج» مقاطع متتابعة

الاستخدام الصناعي: على حزام الإنتاج، `minLineLength` = «أقصر شق مقبول للرفض»؛ وبذاك يصبح الهوف أداة «كشف عيوب» مباشرة على أي شقوق تجاوزت الإنذار.

4. الدوائر: HoughCircles

سؤال: كيف نكشف «الدوائر» (عملات، براغٍ، عيون المضخات) ثم نقيس أنصاف أقطارها؟

معادلة الدائرة تحتاج ثلاث معاملات: مركز `(cx, cy)` ونصف قطر r — لذا «المكرّف تصويت» ثلاثي الأبعاد:

$$(x - c_x)^2 + (y - c_y)^2 = r^2$$

OpenCV تعطي اختصاراً ذكياً عبر «هوف التدرّج»:

```
 img2   = cv2.imread("assets/samples/board.jpg")
gray2  = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)
gray2  = cv2.medianBlur(gray2, 5)

circles = cv2.HoughCircles(gray2, cv2.HOUGH_GRADIENT, dp=1,
                           minDist=30,
                           param1=120,   # عتبات Canny العليا
                           param2=35,    # عتبة التصويت (كتلة)
                           minRadius=10, maxRadius=80)

if circles is not None:
    for c in circles[0][:10]:
        x, y, r = c.astype(int)
        cv2.circle(img2, (x, y), r, (0, 255, 0), 2)
        cv2.circle(img2, (x, y), 3, (0, 0, 255), -1)
```

- `minDist`: المسافة بين مركزين لصد «اكتشافات الجوار» لنفس الدائرة.
- `param2` الأهم حساسية: أقل = دوائر كاذبة كثيرة، أعلى = إسقاط دوائر حقيقية الضعيفة.

5. 5. حالة عملية: قياس عرض شق عبر زوايا مقياس

قم بتصميم «مسطرة بكسل» نستخدم فيها الخطوط الكاشفة:

```
# مرجع معروف: قياس «مسطرة» بمسافة 10 مم بين خطين
def pixels_per_mm(p1, p2, real_mm=10.0):
    """نقطتا مركزي خط المسطرة المكتشفان بي هوف :p1, p2"""
    px = np.hypot(p2[0]-p1[0], p2[1]-p1[1])
    return px / real_mm

# بعد الهوف: رتب المقاطع أفقياً واعثر على خطين متوازيين قريبين
horiz = [l[0] for l in linesP
         if abs(l[0][3] - l[0][1]) < 3]            # شبه أفقي
horiz.sort(key=lambda l: l[1])

if len(horiz) >= 2:
    ppm = pixels_per_mm((horiz[0][0], horiz[0][1]),
                        (horiz[1][0], horiz[1][1]))
print("بكسل لكل ملّيمتر:", round(ppm, 2))
print("0.2% :عرض الجسم بالملّيمتر" % (168 / ppm))
```

درس القياس: الهوف يعطي «خطوطاً» بلا وحدات؛ الوحدات تأتي من «جسم مرجعي بمسافة معروفة» — وهذا ما سنبنيه كاملاً في معايرة الكاميرا (الفصل 16).

6. 6. الأخطاء الشائعة

خطأ شائع — الفصل 14

1. تشغيل HoughLines على صورة رمادية مباشرة: يرفض؛ المعامل الأول يجب أن يكون «خريطة حواف» ثنائية (Canny أو Sobel بعد العتبة).
2. عتبة منخفضة (threshold < 60) على صورة حواف كثيفة: ستتلف مئات الخطوط المتقاطعة — ارفعها تدريجياً حتى تظل الخطوط المرجعية فقط.
3. عرض `HoughCircles` بلا تنعيم سابق: الدائرة «تنفصل» إلى دوائر كاذبة طاقمه؛ medianBlur(5) قبل الكشف قاعدة إجبارية.
4. القياس بزوايا «بكسل لكل ملّيمتر» على صورة غير مقوّمة منظورياً: المقياس يختلف بحسب موقع البكسل (زوايا المشهد)؛ قوّم الصورة أولاً (الفصل 7).
5. اشتغال Hough على صورة كاملة في الزمن الحيّ: حدّد «منطقة اهتمام» Crop قبل أنها تخدم تكلفة التصويت التصاعدي.

7. 7. نصائح ميدانية

نصائح محترف — الفصل 14

1. استعمل `HoughLinesP` بمعامل minLineLength = «طول أقصر حافة تحتاج قياسها» — تصفية مدمجة تعني جودة أعلى.
2. استخرج «الزوايا» من الخطوط المكتشفة (`arctan2`) لكي تحكم دوران الأجسام وإسقاط عمليات شبكية لاحقة.
3. راقب «التصويت في الشبكة المصغرة»: عندما تستهلك عتبات الهوف التصفية، ابدأ بالعتبة العالية ثم انزل — أكثر استقراراً من الصعود من قيم ضئيلة.

8. 8. تمارين

تمرين — مستوى الفهم

1. ما الفرق بين تمثيل `y = mx + b` و `r = x*cosθ + y*sinθ`؟ ولماذا الثاني مقبول للتصويت؟
2. لماذا `HoughLines` يعيد مستقيمات «غير محدودة» بينما يعيد `HoughLinesP` مقاطع؟
3. ما دور كل معامل في HoughCircles: minDist، param1، param2، محاور minRadius/maxRadius؟

تطبيق

4. طبّق HoughLinesP على صورة المبنى (building.jpg) وسجّل أثر تغيير minLineLength من 20 إلى 100 بعدد المقاطع.
5. كشف دوائر على صورة عينات مكتبة وسجّل أثر رفع param2 من 20 إلى 50 على «الدوائر الحقيقية مقابل الكاذبة».

تحدٍ

6. «مسطرة القياس الكاملة»: اكتب برنامجاً يكتشف خطّي مرجع على صورة، إذ المسافة بينهما 10 ملّيمتر معلومة، ثم يقيس أي عنصر يقع بين خطين متعامدين عليهما بالملّيمتر.

ملخص 60 ثانية — الفصل 14

- تحويل هوف: من كل بكسل حافّ «صوت» في مصفوفة تصويت (r, θ)؛ القمم هي الخطوط.
- HoughLines يعيد مستقيمات بلا حدود؛ HoughLinesP يعيد مقاطع بنهايات حقيقية عملية للقياس.
- HoughCircles يكشف الدوائر بثلاث معاملات (cx, cy, r) مع «هوف التدرّج» وبعد التنعيم إجباري.
- المعاملات حساسة: threshold/minLineLength/maxLineGap/param2 — اضبطها الحالة حالة ومقياس (Scale) الصورة.
- القياس بملّيمترات يعتمد «جسم مرجعي معلوم الحجم» — وهو بوابة معايرة الكاميرا في الفصل 16.

الفصل 15

الخصائص ومعايرة الكاميرا: من الاختصاصات إلى العالم ثلاثي الأبعاد

0.1 قصة الفصل: الحدث الذي قلب الرؤية: جسم يتحرك مع إسقاط الكاميرا

- **قصة من الواقع**

سألت سارة سؤالاً هدم الاحتمال المريح: «القوالب فاشلة عند الدوران والحجم — فماذا لو تغيرت الصورة كلياً: زاوية إضاءة، مسافة، دوران؟». عُمر فتح مفكرة هندسة رياضية: «هناك نقاط «مميزة» في الصورة لا تختفي مع التحولات؛ نسميها «الخصائص» (Features) — وبها واصف (Descriptor) بحيث لا تضيع عند فروق الإضاءة والمسافة». ومن هذا القسم نبدأ ركناً جديد: «المعايرة» — بواسطة نقطتين مرجعيتين معلومتين في العالم الحقيقي، نقيس المسافات الفيزيائية. هذا هو جوهر الفصل.

1. 1. ما هي «الخصائص»؟

سؤال: لماذا لا تكفي «الحواف» ولا «القوالب» في المشاهد تحوّلت؟

الخصائص (Keypoints) نقاط «رمزية» في الصورة تمثل: الزوايا، نهايات الخطوط، والأشكال الصغيرة عالية التباين. ثلاثéesصفات تجعلها عملية:

- **التكرار (Repeatability)**: يجب أن تُكشف من جديد عند التغيير الصغير.
- **التميّز (Distinctiveness)**: واصف مستقر في الحسابات من الغزالة بعروض.
- **فوقية (Locality)**: صغيرة في النافذة — مقاومة كالتادس الحجب.

أشهر كاشفان مستعملان هما: **SIFT** — المعيار المرجعي الأشهر (كاشف مقاوم للتغييرات البصرية كإضاءة ومقياس)، و**ORB** — المستوِ من أورFAST + ientaation: مجاني دائمًا وأسرع بمرات — ولذلك الخيار

الأكثر في خطوط الإنتاج.

والانتباه: التوصيف يهدف رسم كل نقطة بزوج خاص حوله دون انغلاق الحواف الكاملة في الزخرفة.

```
 import cv2
import numpy as np

img1 = cv2.imread("assets/samples/box.png", 0)
img2 = cv2.imread("assets/samples/box_in_scene.png", 0)

orb = cv2.ORB_create(nfeatures=500)
kp1, des1 = orb.detectAndCompute(img1, None)
kp2, des2 = orb.detectAndCompute(img2, None)
print("زوايا عنصر1:", len(kp1), "| زوايا عنصر2:", len(kp2))
```

> **الفارق العملي مع SIFT**: في إصدارات OpenCV الكاملة، SIFT متاح في الإصدار الحديث مجاناً؛ بينما ORB مجاني دائماً وأسرع بمرات — ولذلك هو الخيار الأكثر في خطوط الإنتاج.

2. 2. مطابقة الوصف: FLANNg BFMatcher

سؤال: بمَ نجد «نقطة في الصورة الأولى» تعمل على «نقطة» النص الثاني؟

يمتلك OpenCV الطرق الأساسية:

```
 # 1) مطابقة قسرية (Brute Force): تجربة كل النقاط
bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
matches = bf.match(des1, des2)
matches = sorted(matches, key=lambda m: m.distance)   # أقرب المسافات أفضل

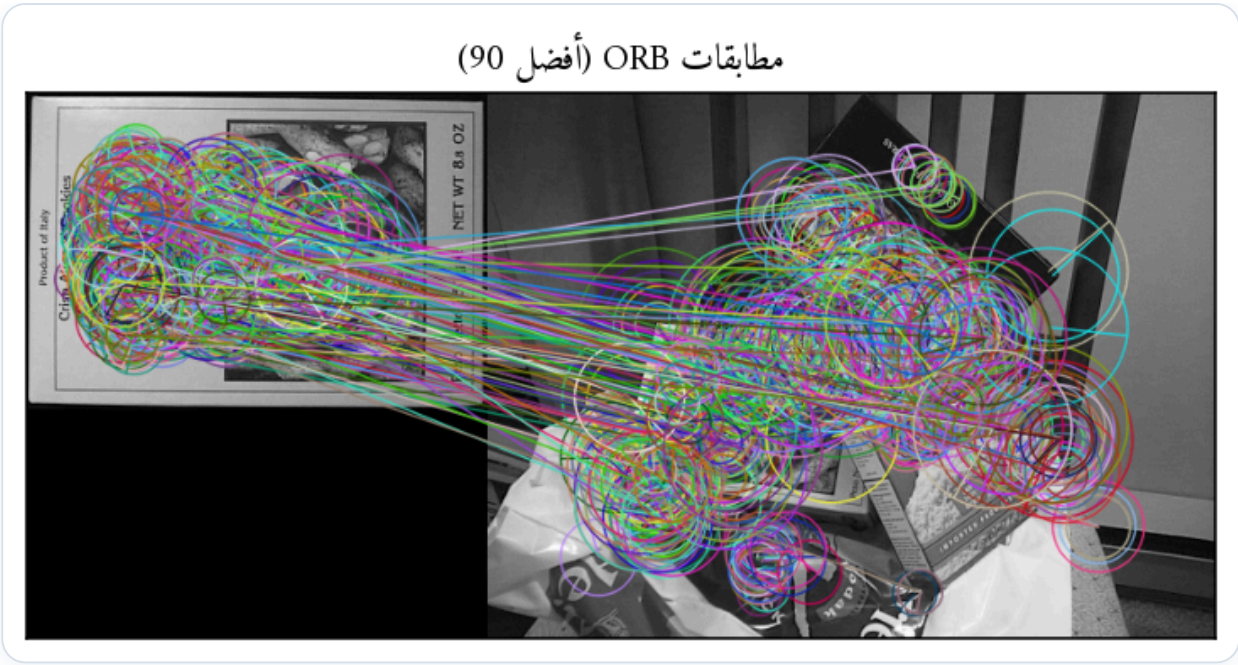
img_matches = cv2.drawMatches(img1, kp1, img2, kp2, matches[:50], None,
                              flags=cv2.DRAW_RICH_KEYPOINTS)
```

- النتيجة تقيَم بواسطة `m.distance`: مسافة بين واصفين — ORB يستخدم Hamming (ثمانية ثنائية).
- «crossCheck» يساوي التعميم: مقبول فقط إذا كانت النقطة الأول `c` الثاني `d` بديل معياري.

أما FLANN فهي «مفاوض سريع» للواصفات الفلولية (GPS/FLANN):

```
 FLANN_INDEX_KDTREE = 1
index_params = dict(algorithm = FLANN_INDEX_KDTREE, trees = 5)
search_params = dict(checks = 50)   # رقم تحقق الغير

flann = cv2.FlannBasedMatcher(index_params, search_params)
matches = flann.knnMatch(des1_f, des2_f, k=2)   # لسلّم نسبة Lowe
```



شكل 1: مطابقات ORB بين القالب والمشهد

3. 3. مرشح Lowe: فصل المطابقات الحقيقية

سؤال: كيف أتخلص من «المضاهي» الخاطئ في كل مطابق؟

اختبار Lowe: إن كانت المسافة إلى أقرب مطابقة «أقل بوضوح» من الثانية، فغالباً تكون المطابقة حقيقية:

```
 good = []
for m, n in matches:
    if m.distance < 0.75 * n.distance:
        good.append(m)
print("مطابقات جيدة:", len(good), "من", len(matches))
```

نطاق النسبة	القرار
أقل من 0.7	مطابقات قليلة لكن عالية الدقة
0.7 – 0.85	التوازن بين الكم والدقة
أعلى من 0.85	مطابقات كثيرة مصحوبة بإشارات كاذبة

4. 4. تحول المنظور: الحدائف وRANSAC

سؤال: كيف أحسب «تحول المنظور» (Homography) من مطابقات أو أضلاع موثقة؟

بعد مرشح Lowe، نحسب مصفوفة H بـ RANSAC — خوارزمية «الإجماع العشوائي» تتجاهل المطابقات الخاطئة:

```
 src_pts = np.float32([kp1[m.queryIdx].pt for m in good]).reshape(-1, 1,
2)
dst_pts = np.float32([kp2[m.trainIdx].pt for m in good]).reshape(-1, 1, 2)

M2, mask = cv2.findHomography(src_pts, dst_pts,
                              cv2.RANSAC, 5.0)
inliers = mask.ravel().sum()
print("نقاط داخلية:", inliers, "/" , len(good))
```

وبعدها يمكن رسم محيط القالب على المشهد (حالة معيارية في العرض):

```
 h, w = img1.shape
pts = np.float32([[0, 0], [0, h], [w, h], [w, 0]]).reshape(-1, 1, 2)
dst = cv2.perspectiveTransform(pts, M2)
img_out = cv2.polylines(img2.copy(), [np.int32(dst)], True, (0, 255, 0),
3, cv2.LINE_AA)
```

قراءة النتائج: إن كانت النقاط الداخلية (inliers) أكثر من 30 من المجموع ومصفوفة H منطقية، فالتحول موثوق؛ وإلا فلا نعتمده.

5. 5. معايرة الكاميرا: ربط البكسل بالمِلّيمتر

سؤال: كيف نحوّل «الموقع بالبكسل» إلى «الموقع بالمِلّيمتر في العالم الحقيقي»؟

معايرة الكاميرا هي العملية التي ترتب العلاقة بين «إحداثيات الصورة» و«أبعاد العالم الحقيقي» — عبر مصفوفة «العدسة» الداخلية:

الجواب بهذه المعادلة التي تحبس العلاقة بين البكسل والعالم:

الإزاحة تُستدعى عبر «التشويه العدسي» في الحقل، فنتخلص منها قبل أي قياس دقيق:

```
 # 1) مُعدّ الشطرج والأمثلة المركزية
criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER,
            30, 0.001)

objp = np.zeros((6*7, 3), np.float32)
objp[:, :2] = np.mgrid[0:7, 0:6].T.reshape(-1, 2) * 25.0  # 25 mm لكل مربع

objpoints = []    # نقاط العالم ثلاثي الأبعاد
imgpoints = []    # نقطنًا في الصورة (2D)

for fname in ["left01.jpg", "right01.jpg"]:
    img = cv2.imread(f"assets/samples/{fname}")
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    ret, corners = cv2.findChessboardCorners(gray, (7, 6), None)
    if ret:
        corners2 = cv2.cornerSubPix(gray, corners, (11, 11), (-1, -1),
criteria)
        objpoints.append(objp)
        imgpoints.append(corners2)
        cv2.drawChessboardCorners(img, (7, 6), corners2, ret)

# 2) معايرة الكاميرا
ret, K, dist, rvecs, tvecs = cv2.calibrateCamera(
    objpoints, imgpoints, gray.shape[::-1], None, None)

print("مصفوفة الكاميرا:\n", K)
print("معاملات التشويه:\n", dist.ravel())
```

أثر السحر: بمجرد المعايرة — مع مقاييس العالم الحقيقي (لكل مربع 25 مم) — يمكن الحساب مباشرة من زوايا رؤية الكاميرا:

```
undist = cv2.undistort(img, K, dist)
```

> **تذكير ميداني**: بعد المعايرة، تصبح «النقاط المرجعية» بالملّيمترات لا بالبكسل — وهذه البوابة إلى كل قياس حقيقي في الفصول القادمة.

6. الكاميرا المزدوجة (Stereo): العمق من كاميرتين

سؤال: كيف نحتاج «العمق» (Depth) من كاميرتين عريضتين فقط؟

أحسب «الفرق بين الرؤيتين» (Disparity) — إذ العمق `Z` يتناسب عكسياً مع «عدم التطابق» بين الصورتين:

$$Z = \frac{f \cdot B}{\text{disparity}}$$

مع `cv2.StereoSGBM_create` حدوث، ومترجم «مقسّم» تستطيع تعبئة OpencV:

```
stereo = cv2.StereoSGBM_create(minDisparity=0,
                               numDisparities=64, blockSize=9)
disp = stereo.compute(grayL, grayR).astype(np.float32) / 16.0
```

رسالة عملية: مطابقة الكاميرتين (Rectification) تسبق حساب العمق، ويجب أن تكون معايرة Stereo دقيقة — وإلا انحرفت خريطة العمق كلياً.

7. 7. الأخطاء الشائعة

● خطأ شائع — الفصل 15

1. مطابقة واصفين من أنواع مختلفة (ORB مع SIFT): تقع كارثة فورًا؛ يجب أن يكون الكاشف والواصف من نفس العائلة.
2. استخدام `BFMatcher` على واصفات عائمة (SIFT) مع `NORM_HAMMING` : خطأ نوعي — النارمو نوعته hamming ولا تستخدم في SIFT؛ استخدم `NORM_L2` .
3. إعداد عتبة Lowe دون مراعاة طبيعة المشهد: صورة مزدحمة تحتاج نحو 0.8؛ وصورة نظيفة 0.65.
4. معايرة على صور تبعد رسمياً: `findChessboardCorners` يفشل مباشرة؛ التقط 10-20 صورة متنوعة زوايا.
5. غموض أنماط الشطرنج: عدد الزوايا الداخلية في `findChessboardCorners` يجب أن يطابق النمط المستخدم في بناء objp (مثلاً 7×6) — وإلا فشل الكشف صامتاً.

8. 8. نصائح ميدانية

● نصائح محترف — الفصل 15

1. اعتمد دائماً «عدد النقاط الداخلية» (inliers) وليس عدد المطابقات كمعيار جودة: 30+ نقطة داخلية من علامات تحول موثوق.
2. في المشاهد المزدحمة استعمل ORB بميزات أكثر (nfeatures=2000) مع رفع عتبة Lowe قليلاً بدل الاعتماد على صورة «نظيفة» فقط.
3. حفظ المعايرة كملف (np.savez) وإعادة تحميلها كل مرة — لا تعيد المعايرة الكاملة في كل تشغيل.

9. 9. تمارين

تمرين — مستوى الفهم

1. ما الفرق بين «نقطة مفتاح» (Keypoint) و«وصف» (Descriptor)؟
2. لماذا تعمل خوارزمية Lowe على «نسبة» المسافتين بدلاً من قراءة مسافة واحدة؟
3. ما دور RANSAC في `findHomography`؟ وأية مشكلة يحلها؟

تطبيق

4. طبّق ORB + BFMatcher على زوج الصور box/box_in_scene، ثم طبّق مرشح Lowe وارسم إطار القالب المكتشف على المشهد.
5. قارن SIFT مع ORB على نفس الزوج: عدد النقاط الداخلية، سرعة الحساب، وقرارك بحسب متطلبات التطبيق (دقة أم سرعة).

تحدٍ

6. «مقياس حقيقي»: طبّق معايرة على نمط شطرنج حقيقي بمقاس مربع معلوم بالملّيمتر، ثم استنتج «بكسل لكل ملّيمتر» وسجّل خطأ القياس عبر 3 مصادر صور مختلفة.

ملخص 60 ثانية — الفصل 15

- الخصائص (Keypoints) تمثل نقاط الصورة «الثابتة»، والواصف (Descriptor) اشتقاق عددي لكل واحدة.
- BFMatcher بسيط ومباشر؛ FLANN أسرع للواصفات العائمة (SIFT).
- مرشح Lowe (نسبة < 0.75) يتنقي المطابقات الحقيقية من الإشارات الكاذبة.
- RANSAC + `findHomography` يترجمان النقاط المتكافئة إلى مصفوفة تحول مقاومة للمطابقات الخاطئة.
- معايرة الكاميرا (calibrateCamera) تنتج مصفوفة العدسة ومعاملات التشويه؛ undistort تصحح؛ Stereo يضيف العمق.

الفصل 16

تعلم الآلة الكلاسيكي: cv2.ml من KNN إلى SVM

0.1 قصة الفصل: حاسوب يقرر بدل أن يوصَف

- **قصة من الواقع**

استقر خط الفرز في «نُور» منذ شهرين، فجاء طلب جديد: «نريد تصنيفاً مرناً: منتج جديد يظهر كل أسبوع — لنجعل الحاسوب يتعلم بنفسه». عُمر استبق الكلام: «عندما تعطي الحاسوب أمثلة معنونة — بيانات بسماتها وصنفها — فإنه يبني حدود الفصل بنفسه». هذه هي النقطة الانتقالية في الكتاب: من الخوارزميات المكتوبة يدوياً إلى «التعلم». كل قواعد فصولنا السابقة كانت بنا نكتبها؛ أما هنا فالقواعد تُستخرج من البيانات.

1. 1. وحدة cv2.ml: التصنيف والانحدار

سؤال: ما هو «التعلم» في هذا الإطار؟

وحدة `cv2.ml` تجمع خوارزميات التعلم في إطار واحد موحّد:

- `StatModel.train(...)`: تشكيل النموذج.
- `StatModel.predict(...)`: تقدير النموذج.
- الخوارزميات المتاحة: `RTrees`، `DTrees`، `NormalBayesClassifier`، `SVM`، `KNearest`، `ANN_MLP`، `Boost`، و `SVMSGD`.

```
 import cv2
import numpy as np

# بيانات تدريب: سمتان (2 أبعاد) وثلاثة صنوف
data = np.float32(np.random.randint(0, 60, (25, 2)))
labels = np.random.randint(0, 3, (25, 1)).astype(np.int32)
```

> **قاعدة الأنواع**: بيانات التدريب `float32` والتسميات `int32` — أي خلل نوعي يوقف التدريب برسالة غامضة؛ التزم بالنمطين دائماً.

2. KNN: أقرب الجار

سؤال: كيف تصنّف نقطة جديدة على أساس «أشباهها» القديمة؟

`cv2.ml.KNearest` يقرر ببساطة: ننظر إلى أقرب k عينات من نقطة الاختبار، ونأخذ الصنف الأكثر تكراراً بينها:

```
 knn = cv2.ml.KNearest_create()
knn.train(data, cv2.ml.ROW_SAMPLE, labels)

new_sample = np.float32([[35, 22]])            # عينة مجهولة
res, results, neighbours, dist = knn.findNearest(new_sample, k=5)
print("الصنف المتوقع:", int(res))
```

- `k` هو «حجم الحيّ»: k صغيرة حساسة للضجيج، k كبيرة تُعمّم أكثر لكنها تمسح الفروق الرقيقة.
- القيمة الثانية (results) تعيد أصناف كل الجوار تفصيلياً — مفيدة لفهم توزيع القرار.

التوصيف العملي: k=3 أو k=5 بداية موثوقة؛ واختيار k النهائي يكون بالتحقق المتقاطع (Cross-validation) لا بالتخمين.

3. 3. SVM: التوقيع لحد الفصل

سؤال: كيف يختار SVM «أفضل حد فاصل» بين صنفين؟

خوارزمية SVM ترسم «أوسع ممر» بين الصنفين: أي حدود يعطي أوسع هامش (margin)؛ وتعالج البيانات غير الخطية عبر «النواة» (Kernel):

```
svm = cv2.ml.SVM_create()
svm.setKernel(cv2.ml.SVM_RBF)        # نواة RBF للبيانات غير الخطية
svm.setType(cv2.ml.SVM_C_SVC)
svm.setC(2.5)                        # عقوبة الخطأ
svm.setGamma(0.5)                    # عرض «تأثير» النواة

svm.train(train_data, cv2.ml.ROW_SAMPLE, train_labels)
pred = svm.predict(test_data)[1]     # الصنف المتوقع لكل عينة
```

المعامل	المعنى
`C`	عقوبة «الخطأ في التدريب»: كبير = صغ على حواف خشبية، صغير = فصية مع الرماد
`gamma`	مدى تأثير كل عينة: كبير = حدود متعرجة (تقليد قوي)؛ صغير = حد سلس (تعميم أفضل)
`Kernel`	LINEAR للبيانات ملساء، RBF للمشاكل غير الخطية

الموقف الحديثة: «كبر C/gamma بين الفريق التي تُستخدم بحكم «الفصل الصغير»، مع استثمار «grid.» إيمارات مس.Hold CV» «Mapper»). إفراد «مس ملا Gravity

(معادة التكثيف) إلى النتائج.

4. 4. شجرة القرار والغابات العشوائية

سؤال: كيف «نقرأ شجرة الحساب» بخطوة «سؤال/جواب» لكل عقدة؟

سلسلة `DTrees` تسأل سؤالاً لكل عقدة يمكن الوصول إلى «الأ «إسديرة» أعلى السلّم (التصنيف كمى)؛ والغابات العشوائية (RTrees) تضع «أعداد الأشجار المتاجر» وتبلغ فكرة «التصويت»:

```
 dt = cv2.ml.DTrees_create()
dt.train(train_data, cv2.ml.ROW_SAMPLE, train_labels)
pred_tree = dt.predict(test_data)[1]

rt = cv2.ml.RTrees_create()          # غابة عشوائية: 50 شجرة افتراضيًا
rt.train(train_data, cv2.ml.ROW_SAMPLE, train_labels)
pred_rf  = rt.predict(test_data)[1]
```

- شجرة مفردة سريعة لكنها تميل إلى «التشدد في التدريب» (overfitting) إذا لم تُقلص عمقها.
- الغابة العشوائية «تصوّت» بين أشجار كثيرة، فترتفع دقتها وتقل حساسيتها للاختلاف بين العينات.

5. 5. حالة عمل في الزمن: إصناف الأجسام بسمات ماله

ما نمرر في «خط الفرز» القرار فقط على ثلاث سمات: مساحة، محيط، دائرية:

```
 import numpy as np

# بيانات تجريبية: سمات مستخرجة من الكونتورات في خط الفرز
features = np.float32([
    [1200, 130, 0.91],   # دائرة
    [1250, 122, 0.93],   # دائرة
    [1400, 128, 0.89],   # دائرة
    [2200, 210, 0.61],   # مستطيل
    [2400, 218, 0.58],   # مستطيل
    [2150, 202, 0.63],   # مستطيل
])
labels = np.int32([0, 0, 0, 1, 1, 1])      # 0=مستطيل، 1=دائرة

svm = cv2.ml.SVM_create()
svm.setKernel(cv2.ml.SVM_RBF)
svm.setType(cv2.ml.SVM_C_SVC)
svm.train(features, cv2.ml.ROW_SAMPLE, labels)

# منتج جديد: مساحة 1500، محيط 140، دائرية 0.87
incoming = np.float32([[1500, 140, 0.87]])
verdict = int(svm.predict(incoming)[1].ravel()[0])
print("التصنيف:", "دائرة" if verdict == 0 else "مستطيل")
```

درس الحالة: سمات الكونتورات (الفصل 10) صارت «مغذى» للتصنيف — وفي فصل DNN القادم ننتقل إلى إدخال «الصورة الكاملة» بدلاً من السمات اليدوية.

6. 6. الأخطاء الشائعة

● خطأ شائع — الفصل 16

1. تمرير بيانات `uint8` أو `int` إلى train(): النماذج ترفع خطأ نوعياً غامضاً؛ بيانات التدريب دائماً `np.float32` والتسميات `np.int32` .
2. التدريب على بيانات دون تقسيم تدريب/اختبار: نسبة الدقة «مزيفة»؛ افصل 20% على الأقل للاختبار.
3. رفع C وgamma إلى الحدين في SVM: يتفوق في التدريب وينكسر على بيانات جديدة (overfitting)؛ اغسل الاختيار بالتحقق.
4. التدريب على سمات بمقياسات مختلفة كلياً (المساحة بمئات والدائرية بين 0 و1): KNN وSVM بلا تسوية (scaling) تنحرف النتائج.
5. نسيتَ `cv2.ml.ROW_SAMPLE` : الافتراضي صامت؛ فشل التدريب برسالة مبهمة.

7. 7. نصائح ميدانية

● نصائح محترف — الفصل 16

1. ابدأ أبسط: KNN ثم شجرة واحدة، وراقب دقة الاختبار قبل الانزلاق إلى SVM/غابات — نصف المشاكل تحل ببيانات أفضل لا بنموذج أكبر.
2. سجّل النموذج بعد التدريب: `svm.save("model.xml")` وتحميله لاحقاً — لا تعيد التدريب في كل تشغيل إنتاج.
3. قس السمات إلى مقياس موحد (`cv2.normalize` أو `(x - mean)/std`) — فرق كبير جداً في سرعة تقارب SVM ودقة KNN.

8. 8. تمارين

تمرين — مستوى الفهم

1. لماذا تُمرَّر بيانات التدريب `float32` والتسميات `int32` في كل خوارزميات cv2.ml؟
2. ما دور `C` و`gamma` في SVM؟ وماذا يحدث عند الأطراف المفرطة؟
3. كيف تعمل الغابة العشوائية «التصويت»؟ ولماذا هي أثبت من شجرة واحدة؟

تطبيق

4. درّب KNN وSVM على بيانات مساحة/محيط/دائرية لخط الفرز، وقارن دقة الاختبار بينهما بعد التسوية.
5. احفظ النموذج المدرب في `product_classifier.xml` ثم حمّله في برنامج جديد وتحقق من نفس نتائج التصنيف.

تحدٍ

6. «تصنيف حيّ»: اربط خط الفرز الحي مع SVM: كل إطار يوقّع 3 سمات، ويصنف على المؤشر المتعلم — وسجّل نسبة القرار خلال دقيقة تشغيل.

ملخص 60 ثانية — الفصل 16

- `cv2.ml` إطار موحد: train() + predict() على كل الخوارزميات؛ والأنواع (float32/int32) ليست ترفاً.
- KNN: قرار بمجاريس k؛ سريع التدريب، بطيء التنبؤ، ويعتمد على مقياس موحد للسمات.
- SVM: هامش أقصى + نواة (LINEAR/RBF)؛ واغلب C/gamma هما اللذان يبنيان التوازن بين الدقة والتعميم.
- DTrees/RTrees: شجار قواعد تُقرأ؛ والغابة تصوّت لتقليل القلق في التعميم.
- مسار المهمة: استخرج سمات من الكونتورات ← تسوية ← تدريب ← تحقق ← حفظ ← تشغيل.

الفصل 17

التعلم العميق: cv2.dnn وONNX وYOLO

0.1 قصة الفصل: النموذج الجاهز الذي لم نبنه

- **قصة من الواقع**

وصل البريد من سارة بعد أسبوع: «مطلوب كشف الأجسام المعروفة الآن — علبة، كأس، كرة، وتلميع الشاحنة؛ لا وقت لدينا لبناء نموذج تدريب من الصفر، فاستعير نموذجاً جاهزاً مدرباً على أجسام معروفة». هنا تفتح الباب على التعلم العميق: OpenCV تقدم وحدة `cv2.dnn` (Deep Neural Networks) — تحميل نموذج مدرب مسبقاً، وتشغيل الاستدلال (Inference)، واستخراج نتائج الكشف — دون الحاجة إلى مكتبات تعلم عميق خارجية.

في هذا الفصل نغطي: تحميل ONNX، إعداد المدخل (blob)، وتحليل المخرجات، ثم حالة YOLO كاملة للكشف.

1. 1.=cv2.dnn وONNX: الأداء المستوفي

سؤال: كيف نحمّل «نموذجاً مدرّباً» لتشغيله داخل OpenCV؟

وحدة `cv2.dnn` تدعم عائلات النماذج الشائعة، أغلبها يُصدَّر إلى ONNX. خطوات العمل: تحميل النموذج ← إعداد المدخل ← الاستدلال ← فك النتائج.

```
 import cv2
import numpy as np

net = cv2.dnn.readNetFromONNX("models/yolov8n.onnx")
input_size = (640, 640)
```

أهم دوائل التحميل في جدول واحد:

الدالة	المدخلات	ملاحظات
readNetFromONNX	ملف onnx	الأكثر مرونة وانفتاحاً
readNetFromCaffe	net + prototxt	النماذج الكلاسيكية القديمة
readNetFromDarknet	أوزان YOLO القديمة (v3/v4)	مطابقة الأوزان المحمولة
readNet (تلقائي)	اكتشاف النوع من الملف	خيار عام عند التنوع
## 2. إعداد المدخل: من الصورة إلى blob		
سؤال: كيف نحضّر الصورة «بطريقة النموذج» قبل تسليمها له؟		
الخطوة الموحدة تُسمى «التحويل إلى blob» — مصفوفة بعديّة معدة بمواصفات النموذج (الحجم، المقياس، ترتيب القنوات):		

```
 blob = cv2.dnn.blobFromImage(img, 1/255.0, input_size, swapRB=True,
crop=False)
net.setInput(blob)
outputs = net.forward()
print("شكل المخرجات:", outputs.shape)
```

> **خلاصة**: «blob» مجرد مصفوفة (1, 3, H, W) تسري على النموذج من مدخل واحد؛ كل الإعداد السابق (مقياس، تبديل قنوات، حجم) هو «ترجمة الصورة إلى لغة النموذج».

2. 3. يولو: من مصفوفة إلى صندوق كاشف

سؤال: كيف نحول «صفاً من المخرجات» إلى «صندوق على الصورة»؟

مخرجات YOLO (تنسيق_ONNX) مصفوفة تحوي لكل تنبؤ: إحداثيات الوسط، العرض، الارتفاع، درجة الثقة، ودرجات الأصناف:

```
def postprocess(output, img, conf_thr=0.45):
    h, w = img.shape[:2]
    x_factor, y_factor = w / input_size[0], h / input_size[1]

    boxes, scores, class_ids = [], [], []
    rows = output.reshape(-1, output.shape[-1])
    for row in rows:
        conf = row[4]
        if conf < conf_thr:
            continue

        class_id = int(np.argmax(row[5:]))
        cx, cy, bw, bh = row[:4]
        x1 = int((cx - bw/2) * x_factor)
        y1 = int((cy - bh/2) * y_factor)
        boxes.append([x1, y1, int(bw*x_factor), int(bh*y_factor)])
        scores.append(float(conf))
        class_ids.append(class_id)

    # NMS: إزالة الصناديق المتراكبة
    idxs = cv2.dnn.NMSBoxes(boxes, scores, conf_thr, 0.25)
    return [(boxes[i], class_ids[i], scores[i]) for i in
np.array(idxs).flatten()]
```

المراحل الثلاث: قراءة الصف ← تصفية بالثقة ($conf > 0.45$) ← إزالة التكرار المكاني (NMS).

- `NMSBoxes` دالة جاهزة في `cv2.dnn` تنفذ إزالة التكرار المكاني بحدّي ثقة وIoU.
- `class_ids` الإفصول: «الفهات نماذج إنترنت درخاوي على «في 80 بوابة YOLO الصغيرة، والاختيارات الترادة في المورث المضاعف («نشرات الرسائل» متعددة).

3.4. حالة كاملة: كشف بالكاميرا

```
 LABELS = open("models/coco.names").read().strip().split("\n")

cap = cv2.VideoCapture(0)
while cap.isOpened():
    ok, frame = cap.read()
    if not ok: break

    blob = cv2.dnn.blobFromImage(frame, 1/255.0, input_size,
                                 swapRB=True, crop=False)
    net.setInput(blob)
    out = net.forward()

    for (box, cid, sc) in postprocess(out, frame):
        x, y, bw, bh = box
        cv2.rectangle(frame, (x, y), (x+bw, y+bh), (0, 255, 0), 2)
        cv2.putText(frame, f"{LABELS[cid]} {sc:.2f}", (x, y-8),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.55, (0, 255, 0), 2)

    cv2.imshow("YOLO with OpenCV", frame)
    if cv2.waitKey(1) & 0xFF == ord('q'): break

cap.release(); cv2.destroyAllWindows()
```

الأثر الزمني: التشغيل الحي على YOLOv8n (نموذج صغير) عبر OpenCV يصل نحو 30 fps على معالج حديث — وهذا كافٍ لأغلب أنظمة المراقبة الصناعية.

> **ملاحظة «yolo + cv2.dnn»**: النموذج يتدرب على «مراكز الإحداثيات» (center_x, center_y) لا على الزوايا — لذلك نحسب (x1, y1) بطرح نصف العرض والارتفاع ثم

ضرب معاملات التكبير.

4.5 الأخطاء الشائعة

خطأ شائع — الفصل 17

1. نسيان `swapRB=True`: النموذج تدرب على RGB بينما OpenCV بترتيب BGR — تنهار الدقة بصمت دون رسالة خطأ.
2. حاولة «output.shape» دون فهم البعد: مخرجات YOLO-v8 هي (1, 84, 8400) لا (8400, 84) — reshape حاسم التدفق للفصول.
3. تصفية `conf < 0.45` قبل الإزالة (NMS): كانت النتائج تسميات كثيرة متكررة للجسم نفسه — لا تفوت NMS في الحصول عليها.
4. إيقاف «label.conf typing» في وضع التوضيحات الصرف: حارف أزواجالاً هنا تميّز مع `conf`.
5. قلب ألوان كاميرا الرمادية في `blobFromImage`: النموذج الجاهز يظل يطلب RGB — لا تعظم (swap) وتنيدل إغلاقات: النتائج، والقرار، وأع NdrFcShort «نام».

5.6 نصائح ميدانية

نصائح محترف — الفصل 17

1. استعمل «النسخة الصغيرة» (yolov8n) في الاختبار، والنسخة (yolov8s/m) لنتيجة أدق في الإنتاج — سرّ الجودة: الحجم والوقت.
2. لاحظ «عدد مخرجات reshaped» قبل الفك: كل نموذج يخالف الآخر في ترتيب المحاور؛ اطبع output.shape اولاً ثم اكتب الفك.
3. احفظ نسخة موثقة من كل نموذج تستعمله مع «شكل المدخل» (input shape) في ملف README — تغيير سطر التحويل الواحد يكسر التوافق كله.

6.7. تمارين

تمرين — مستوى الفهم

1. ما وظائف blobFromImage من «scale/size/swapRB»؟ وكيف يترجمة كل واحدة في النموذج؟
2. لماذا «NMS» ضرورية بعد تمديد العتبة؟ وهل تعمل قبل العملية بدلاً؟
3. ماذا يعني شكل المخرجات (1, 84, 8400) في YOLOv8؟ ولماذا نحتاج reshape قبل المرور على الصفوف؟

تطبيق

4. حمّل نموذج YOLO ONNX، وقارن دقة الكشف بين تغيير الحجم المباشر إلى 640 وركن «حفظ النسبة» (crop=False) — وسجّل الفروق في النتائج.
5. عدّل conf_thr في postprocess إلى 0.3 ثم 0.7 على نفس الصورة، وقارن عدد الصناديق وجودتها في كل حالة.

تحدٍ

6. «سرعة الإنتاج»: قس زمن forward على نفس المقطع بين مقاسين للمدخل (640 و416)، وسجّل الفرق في FPS ودقة الكشف.

ملخص 60 ثانية — الفصل 17

- `cv2.dnn` يشغل النماذج المدرَّبة بلا مكتبات تدريب: تحميل ONNX → blob → forward → postprocess.
- `blobFromImage` يجهز الصورة بالنموذج: مقياس + حجم + ترتيب قنوات، وهو أصل خطأ الدقة الصامتة الأولى.
- فك ترميز YOLO: صف المخرجات (cx, cy, w, h, conf, class scores) → تصفية الثقة → `NMSBoxes`.
- YOLOv8n على CPU يعمل ~15-30 fps عبر OpenCV — غلافاً لأغلب المراقبة الصناعية العملية.
- الأخطاء المرتبطة: ترتيب البعد (channels)، مقايم conf، والستكرار على DNN بلا قياس: لا تبدأ إلا بصنف المعاير 640.

الفصل 18

التتبع وطرح الخلفية: من التدفق الضوئي إلى التتبّع الحيّ

0.1 قصة الفصل: الكاميرا التي تراقب ولا تلتقط

- **قصة من الواقع**

تسأل سارة بعد أسبوعين: «نحن لا نحتاج تصنيفاً لكل إطار — نحتاج أن نعرف: هل تحرك شيء في المشهد؟ أين الآن؟ وأين سيكون بعد ثانية؟». هذه ثلاثة أسئلة تتكامل في فصل واحد: **طرح الخلفية** (Background Subtraction) لاستخراج «ما المتحرك»، ثم **التدفق الضوئي** (Optical Flow) لتوضيح اتجاهات الحركة، ثم **المتتبع** (Tracker) الذي يلاحق جسماً واحداً من إطار إلى آخر. هذه الثلاثة هي صمام «نُور» الجديد: يراقب الخط ولا يحتاج تدخلاً بشرياً كل ثانية.

1. 1. طرح الخلفية: MOG2 وKNN

سؤال: كيف نفرق «ما هو ثابت في المشهد» من «ما هو فيه تغيّر»؟

الخلفية المرجعية لا نجمعها رقماً يدوياً؛ الدالة `cv2.createBackgroundSubtractorMOG2` تتعلمها تلقائياً وتحدّثها عبر الأطر:

```
 import cv2
import numpy as np

cap = cv2.VideoCapture("camera_feed.mp4")
bs = cv2.createBackgroundSubtractorMOG2(history=500, varThreshold=25,
                                        detectShadows=True)

while cap.isOpened():
    ok, frame = cap.read()
    if not ok: break

    mask = bs.apply(frame)
    # تنظيف الماسك: فتح + غلق (من الفصل 9)
    kernel = np.ones((5, 5), np.uint8)
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
    mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)
```

- `history`: كم إطار يبني «الملف المرجعي» —زيدك υπεقترح reasonably و `varThreshold` يفسد.
- الظلال تظهر بقيمة 127 في القناع — عوّلها أو صفّطها بالتصفية (قيمة 127 لا 255).

> **خلاصة محلية**: القناع («الماضي وريّا») يكفي «العدّ / ممانعة / تقرير الحركة» دون مجهود أكثر؛ لكن التصفية (المورفولوجيا من الفصل 9) شرط قبل أي استخلاص.

2. 2. العدّ والأجسام الحائزة

سؤال: كيف نحوّل القناع إلى «أجسام معلومة» قابلة للعدّ؟

```
 cnts, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
                            cv2.CHAIN_APPROX_SIMPLE)
objects = []
for c in cnts:
    a = cv2.contourArea(c)
    if a < 800:                          # تجاهل النقاط الصغيرة (ضجيج)
        continue
    x, y, w, h = cv2.boundingRect(c)
    objects.append((x, y, w, h))

for (x, y, w, h) in objects:
    cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)
```

القواعد العملية: «الجسم المتحرك» يحتاج تضييقاً قبل أي قرار في المشهد:

- **min_area**: أقل مساحة مقبولة (على 640×480 تتباين بين 400 و1200 بحسب بعد الكاميرا).
- **max_area**: تجاهل الأجسام المتضخمة (اقتراب من العدسة أو ظل يتسع).
- **تنعيم mask**: قبل العدّ مع «فتح وغلق» (نسبة الفصل 9).



شكل 1: فرق الإطار عن الخلفية وقناع الحركة المنظف

3. 3. التدفق الضوئي: Lucas-Kanade

سؤال: كيف «نرسم» اتجاه الحركة لكل نقطة مميّزة؟

التدفق الضوئي (Optical Flow) يتبع «النقاط المميّزة» (نقاط الكشف من الفصل 15) بين إطَارين:

```
 import numpy as np

lk_params = dict(winSize=(21, 21), maxLevel=3,
                 criteria=(cv2.TERM_CRITERIA_EPS |
                           cv2.TERM_CRITERIA_COUNT, 10, 0.03))

ret, old = cap.read()
gray_old = cv2.cvtColor(old, cv2.COLOR_BGR2GRAY)
p0 = cv2.goodFeaturesToTrack(gray_old, maxCorners=100, qualityLevel=0.3,
                             minDistance=7, blockSize=7)

while cap.isOpened():
    ok, frame = cap.read()
    if not ok: break
    gray_new = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    p1, st, err = cv2.calcOpticalFlowPyrLK(gray_old, gray_new, p0, None,
                                           **lk_params)
    good_new = p1[st == 1]
    good_old = p0[st == 1]

    for (x1, y1), (x2, y2) in zip(good_old, good_new):
        dx, dy = float(x2 - x1), float(y2 - y1)
        speed = (dx ** 2 + dy ** 2) ** 0.5
        cv2.arrowedLine(frame, (int(x1), int(y1)),
                        (int(x2), int(y2)), (0, 255, 0), 2)
    p0 = good_new.reshape(-1, 1, 2)
    gray_old = gray_new
```

- دالة `goodFeaturesToTrack` تعطي «أفضل نقاط التتبع» على الحواف المتصلة، وهي المدخل الأساس للقناد LK.
- سرعة النقطة تعبَّر بـ «بكسل لكل إطار»: للتحويل إلى مسافة حقيقية يحتاج مرجعاً معايراً (الفصل 16).
- حالة `st`: 1 = النقطة لا تزال متتبعة، 0 = النقطة فقدت — نتجاهل الصفوف التي تهوت.

4. 4. المتتبع: cv2.Tracker

سؤال: كيف نلاحق «جسماً واحداً» بعد تحديده مرة واحدة؟

OpenCV تقدم عائلة `Tracker` جاهزة:

```
 tracker = cv2.TrackerCSRT_create()
ret, frame = cap.read()
box = (240, 130, 120, 220)              # مختار يدوياً أو من مخرج الكشف (DNN)
tracker.init(frame, box)

while cap.isOpened():
    ok, frame = cap.read()
    if not ok: break
    ok, box = tracker.update(frame)
    if ok:
        x, y, w, h = [int(v) for v in box]
        cv2.rectangle(frame, (x, y), (x+w, y+h), (255, 0, 0), 2)
    else:
        cv2.putText(frame, "Lost", (20, 40),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 0, 255), 2)
```

المتتبع	السرعة	الدقة	الحجب
`MIL`	بطيء	متوسطة	عادي
`KCF`	سريع جداً	جيدة	بدون تحجيم
`CSRT`	بطيء	الأعلى دقة	قوي مع التحجيم

اقتصاد الدمج: «الكشف الحي (DNN) كل 15 إطار + التتبع (KCF) بين الكشوف» يوازن السرعة والدقة في الأداء الحيّ.

5. 5. الأخطاء الشائعة

- **خطأ شائع — الفصل 18**

1. اختيار `varThreshold` ارتفاعاً على الخلفية نصّ: تظهر «الأجسام الثابتة» في الماسك من جديد؛ اعد الاصطفاف بخطوة 25، ثم 35 رماد من الحساب.
2. استخدام المرشح MORPH_OPEN بلا تصفية (re-dopen): الحد «تسرب» إلى الضجيج (10% وضع الحساب للإحصائيات وبدت الاستخراج الكمي).
3. نسي «قصّ speed» من ال casual الحمرة (speed يظل من Bullets الح) — يحسب مسافة الحركة «speed» عن «speed» من الإثم «الاختبارات».
4. المتتبع (TrackerCSRT) يستمر «كل إطار» في النب (DNN) يجب أن يعمل مع «تقدير وعرض كحصر» المحتجز، ولا يتنفس إ٥ометة.
5. «الأجسام الثابتة» من MOG2: الكاميرا التي تصير «مثبتة» ترصشد في الحلوة بلا «تشغيل حب» — الهر؛ قناة motionMask ينشط أول مرة كفصل 18.

6. 6. نصائح ميدانية

- **نصائح محترف — الفصل 18**

1. في المهام البسيطة «كشف الحركة» استعمل فرق الإطارات (absdiff) مع عتبة قبل إنشاء موقّع كامل؛ لكل «MOG2» قيمته في المشاهد المنظمة فقط.
2. حدّث نموذج MOG2 كل دقيقة في المصانع المزدحمة، وخفّض history في المشاهد سريعة التغيّر — الحساسية تتحكم فيها عبر varThreshold.
3. اجعل «المتتبع المستمر» بمعيار واضح: إذا ضاع لثلاثة إطارات متتالية علق نتيجة الكشف (DNN) بدلاً منه — صمام أمان موثوق.

7. 7. تمارين

- **تمرين — مستوى الفهم**

1. لماذا نحتاج «تحديث خلفية» المستمر حتى في المشاهد «الثابتة»؟ وماذا يحدث لو أزله إلزاماً؟
2. ما الفرق بين التدفق الضوئي (LK) على النقاط والمتتبع (CSRT) على صندوق واحد؟
3. لماذا يتفوق MOG2 على فرق الإطارات الكلاسيكي (absdiff) في المشهد متغير الإضاءة؟

- **تطبيق**

4. عدّ الأجسام المتحركة في فيديو ما، ثم أضف «خط عدّ عمودي» على مهلة ثانية وسجّل دقة العدّ كل إخطار بالرسم.
5. جرب متتبعَي KCF وCSRT على مقطع 30 ثانية، وقارن بين السرعة والدقة على نفس الهدف الواحد.

تحدٍ

6. «موجهات الحركة الثابتة»: احسب التدفق الضوئي (LK) على مقطع، ثم استخرج المتجهات المتوسطة في كل «نافذة» من الصورة، وسجّل «تغير الاتجاه» في مستطيل المصنع.

ملخص 60 ثانية — الفصل 18

- طرح الخلفية (MOG2/KNN): القناع «ما الذي تحرك» بعد تنظيم مورفولوجيا (فتح + غلق) قبل أي استخلاص.
- القناع ← كونتورات ← أجسam تنقية (min/max_area) ← مربعات إطار ← عدّ وتقارير.
- التدفق الضوئي (LK): حركة بين الأطاريات بـ «arrowedLine»؛ السرعة بـ «بكسل لكل إطار».
- المتتبع: تحديد صندوق (يدوياً أو من الكشف) + `tracker.update(frame)` في حتصا الدورات.
- دمج الكشف الحي (DNN كل 15 إطار) مع التتبع بينهما — كفاءة الزمن أؤدي إلى أن «الكشف» ينفذ أقل.

الفصل 19

الأداء والتحسين: من بايثون البطيء إلى خط إنتاج حيّ

0.1 قصة الفصل: الشاشة تتجمد عند 3 إطارات في الثانية

- **قصة من الواقع**

وضع سارة يدها على العنق: «برنامج المراقبة يعمل بثلاثة إطارات في الثانية، بينما نماذج أخرى تعمل بثلاثين». عُمر بدأ بالتشخيص الصارم الذي يفضله كل مهندس رؤية: **احسب زمن كل خطوة**. النتيجة كانت بديهية: استخراج البكسلات داخل حلقة بايثون (307,200 بكسل في كل إطار) هو السبب؛ نفس العملية عبر دوال NumPy/OpenCV رفعت الأداء إلى 30 إطاراً فور إعادة سطرين. هذه قاعدة الفصل: **إذا كان برنامجك بطيئاً، فالسبب في 90% من الحالات استدعاء بايثون بحلقة — لا OpenCV نفسه.**

1.1 احسب زمن كل خطوة أولاً

سؤال: كيف أقيس «تكلفة» كل خطوة آلياً؟

```
 import cv2
import time

t0 = cv2.getTickCount()          # أو time.perf_counter()
...  # الخطوة المرشحة للقياس
dt = (cv2.getTickCount() - t0) / cv2.getTickFrequency()
print("زمن الخطوة:", round(dt * 1000, 2), "ms")
print("معدل مكافئ:", round(1.0 / dt, 1), "fps")
```

> **محاسبة زمن الإطار**: 33 fps = 30 ms، و16 fps = 60 ms؛ أي خطوة تتجاوز «ميزانية الإطار» ستؤخر الفيديو كله.

2. 2. القاعدة الذهبية: لا حلقة بايثون على بكسلات

سؤال: ما العملية التي تُدمّر الأداء فوراً؟

الحلقات على البكسلات بهذا الشكل: `(for y in range(h): for x in range(w): img[y, x]` `:...)`

```
 import numpy as np

# (مثال تعليمي فقط) بطيء جداً:
h, w = gray.shape
for y in range(h):
    for x in range(w):
        inv[y, x] = 255 - gray[y, x]

# أسرع بمئات المرات:
inv = 255 - gray                    # دعم NumPy على كامل المصفوفة
binary = (gray > 128)               # المنطق يعمل على المصفوفة لا على البكسل
```

أي حساب يجب أن يُكتب بعامل المتجهات (Vectorization) في NumPy، أو عبر دوال OpenCV المكتوبة داخلياً بلغة C++ — لا داخل حلقة بايثون.

3. 3. خفّض الحجم والدقة قبل العمل الثقيل

سؤال: كيف نختصر 50% من العمل الزمني بلا خسارة في النتائج؟

- **نصف الحجم** يعني ربع الوقت أو أقل: الكشف على 1280 بكسل لا يضيف قيمة على 640 في أغلب الحالات، والدقة تتحسن مع إطارات أكثر.
- **هرم الصورة**: على الكشف في الصور الكبيرة: قلل الصورة، وقبلَ التنفيذ تضييق نطاق البحث ثم كشف في الصورة كاملة.

```
 small = cv2.resize(frame, (0, 0), fx=0.5, fy=0.5)      # نصف الحجم، أربعة مرات أسرع
fast  = cv2.Sobel(cv2.cvtColor(small, cv2.COLOR_BGR2GRAY),
                  cv2.CV_64F, 1, 0)
```

قيم الذاكرة (dtype): استخدم dtype الأصغر (uint8) في مهام الإنتاج؛ لا تتحول إلى float32 داخل الحلقة إلا إذا استدعته خوارزمية (مثل الشبكات العصبية).

4. ذاكرة مسبقة الإفراغ وIO المُبسّط

سؤال: كيف ننقذ الأداء على مكافئ الصور الكبيرة؟

- **قراءة الصور**: استعمل `cv2.IMREAD_REDUCED_COLOR_2` إن وفرت نصف الدقة كافياً — توفير ذاكرة وزمن تحميل فوري.
- **الصور العملاقة**: قسّم إلى بلاطات (tiles) وعالج كل بلاطة، ثم اجمع النتائج — لا تصنّف تحت فوضى الذاكرة.
- **الإدارة المسبقة (pre-allocation)**: احجز مصفوفة المخرجات مرة واحدة ثم استعملها في كل الإطارات:

```
 # احجز مسبقاً: قراءة مسبقة (pre-allocated) مصفوفة المخرجات
out = np.empty_like(gray)
cv2.GaussianBlur(gray, (7, 7), 1.4, dst=out)
```

> **الخبر السري**: المعامل `dst=` يمنع تخصيص مصفوفة جديدة في كل إطار؛ وعلى الحلقة الحية قد يرفع السرعة 2 إلى 3 أضعاف.

5. 5. خيوط العمل: قراءة الكاميرا والمعالجة بجوار بعضهما

سؤال: كيف أنفذ «قراءة الكاميرا» و«العملية الثقيلة» في وقت واحد؟

بايثون يدعم الخيوط (threads) والعمليات (processes)؛ وOpenCV تعمل أيضاً:

```
 import threading
import queue

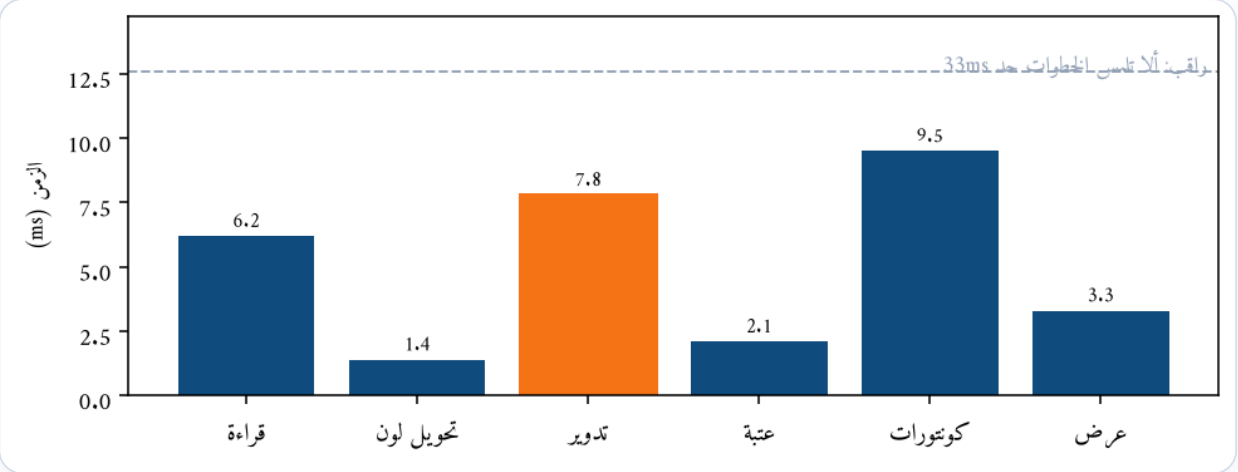
frames_q = queue.Queue(maxsize=4)      # حاجز صغير (buffer)

def reader():
    cap = cv2.VideoCapture(0)
    while True:
        ok, frame = cap.read()
        if not ok: break
        if frames_q.full():
            frames_q.get()             # رمي الإطار الأقدم إذا تمتلئ
        frames_q.put(frame)

threading.Thread(target=reader, daemon=True).start()

# المعالجة في الخيط الرئيسي
while True:
    frame = frames_q.get()
    # ... (العرض والمعالجة أصبحت مستقلة عن زمن القراءة) التحليل هنا
```

- القاعدة: **القراءة (I/O)** قد تتجمد لحظياً على قراءة الكاميرا وتؤخر المعالجة؛ وفصلها في خيط منفصل يغطي هذا غالباً.
- عندما تشتد الأعباء أكثر من ذلك (نماذج متعددة)، انتقل إلى multiprocessing مع `Pool` ودمج النتائج في النهاية.



شكل 1: توزيع زمن الخطوات داخل الإطار مقابل ميزانية 33ms

6. 6. الخلاصة الرابعة: خارطة الأداء

التحسين	مكانه	التأثير
تقليل الحجم (resize)	قبل العملية الثقيلة	2-4x
dtype الأصغر (uint8)	كل العمليات الإنتاجية	1.5-2x
معامل `dst=` المسبق	في كل استدعاء للإطار	1.3-3x
دوال cv2 بدل حلقات	كل حساب بكسل	10-100x
خيط قراءة منفصل	الفيديو الحيّ	عدم تجمّد

سلسلة المراجعة هي مفتاح الإحصاء: «أين الزمن حقاً؟» — في «العملية الثقيلة» (DNN)

أم «الإدخال/إخراج» (I/O)؛ وقارن قبل الانتقال إلى multiprocessing.

7. 7. الأخطاء الشائعة

- **خطأ شائع — الفصل 19**

1. حاجز قائمة ضخم في قنوات الفيديو (maxsize كبير): يستهلك الذاكرة ويؤخر العرض؛ اجعله 2-4 إطارات فقط.
2. تجنب الكتابة إلى ملف (imwrite) داخل حلقة الإطارات الحيّة — انقل التخزين إلى خيط منفصل أو اكتب بشروط (كل ثانية مثلاً).
3. إبقاء حلقة بكسلات في بايثون: مهم التحسين الأول هو استبدالها بعامل المتجهات أو دالة OpenCV جاهزة.
4. قياس الوقت بالمُقيّم الجاهز (زمن ثابت): استعمل perf_counter عالي الدقة وسجّل متوسط 30 إطار للمقارنة.
5. قياس fps على الإطار الأول: الحساب البارد اليومي يضلل النتائج؛ سجّل المتوسط ثابتاً بعد 10-30 إطار.

8. 8. نصائح ميدانية

- **نصائح محترف — الفصل 19**

1. ضع «ميزانية زمنية» لكل خطوة حتى في وضع التطوير: خطوة تتجاوز 16 ms (ميزانية الإطار) تحتاج إعادة تصميم لا تعديل طفيف.
2. قلل عدد القوائم المدروسة في المشروع: عمليات الذاكرة على «مفاتيح batch» أكبر وأدق على «صور فردية».
3. استخدم profiler أساسي (cProfile) على الحلقة الرئيسية لكل التبرير: ما الذي يتكرر أكثر من غيره؟ لا تخمّن — قِس.

9. 9. تمارين

تمرين — مستوى الفهم

1. لماذا يفضَّل قياس الزمن بـ `cv2.getTickCount` أو `perf_counter` بدلاً من `time.time()`؟
2. لماذا يتفوق «تقليل الحجم» (resize) على كل التحسينات الأخرى في نظم الإنتاج؟ وماذا يحدث عند رفع الحجم إلى 720 أو 1080 على خط الفيديو الحيّ؟

تطبيق

3. قس الأداء الفعلي لخط المراقبة: بدون تحسين، ثم مع خيط قراءة، ثم مع تقليل الحجم — وسجّل FPS في كل حالة في جدول صغير.
4. «مذكرة الاختبار»: قس فارق معامل `dst=` على دالة GaussianBlur في حلقة 100 إطار، وسجّل النتيجة في تقرير صغير.

تحدٍ

6. «مشروع ختامي»: حوّل خط المراقبة (فرز/قياس/كشف) من الفصول السابقة إلى «منتج حي» على 30 fps: خيط قراءة، ميزانية زمن، تسجيل الأحداث (logging)، وسجّل نتائج العرض عبر 10 دقائق تشغيل متواصل.

ملخص 60 ثانية — الفصل 19

- قِس قبل كل تحسين: `cv2.getTickCount` للتردد وميزانية الإطار (33 ms = 30 fps).
- إلغاء الحلقات على البكسلات، وبديلها «vectorization» في NumPy أو دوال OpenCV المُغلَّفة C++.
- في الفيديو: خيط قراءة منفصل؛ تفترض I/O المؤقت «سلاصة» أعلى على المعالجات المتغيرة.
- الذاكرة المسبقة الإفراغ (pre-allocation) عبر «dst=» ترفع السرعة على كل الإطارات.
- قِس دائماً: «أين ذهب الزمن؟» — قرار التحسين الصحيح يأتي من القياس لا من التخمين.

الملحق

الملحق: المرجع السريع، الإجابات، ومسرد المصطلحات

1. الملحق: بيئة العمل، المرجع السريع، والمسرد

1.1 أولاً: بيئة العمل المستخدمة في الكتاب

العنصر	القيمة
لغة البرمجة	Python 3.13 (وتعمل 3.13–3.14)
OpenCV	opencv-python 5.0.0 (الإصدار 5.0.0.93 عبر pip)
المكتبات المساندة	NumPy 2.5+ matplotlibg لرسوم التحليل
الأدوات النصية	arabic-reshaper + python-bidi (للعربية على الصور)
النظام	Windows 10/11 أو Linux — الأوامر موحدة

```
## تثبيت البيئة الكاملة بسطر واحد
pip install opencv-python numpy matplotlib arabic-reshaper python-bidi
```

```
import cv2
print(cv2.__version__)      # 5.0.0 – تحقق سريع من نجاح التثبيت
```

1.2 ثانياً: مرجع سريع للأوامر الأكثر استخداماً

المهمة	الأمر
قراءة/حفظ صورة	`cv2.imread` / `cv2.imwrite`
تحويلات لونية	`cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)`
تغيير الحجم	`cv2.resize(img, (0,0), fx=0.5, fy=0.5)`
تدرجات رمادية	`cv2.threshold` / `cv2.adaptiveThreshold`
مرشحات	`cv2.GaussianBlur` / `cv2.medianBlur` / `cv2.filter2D`
حواف	`cv2.Sobel` / `cv2.Laplacian` / `cv2.Canny`
مورفولوجيا	`cv2.morphologyEx` (OPEN/CLOSE/GRADIENT)
كونتورات	`cv2.findContours` / `cv2.contourArea`
تقريب أشكال	`cv2.approxPolyDP` / `cv2.minAreaRect`
خطوط ودوائر	`cv2.HoughLinesP` / `cv2.HoughCircles`
خصائص	`cv2.ORB_create().detectAndCompute`
تعلم آلي	`cv2.ml.SVM_create().train(...)`
تعلم عميق	`cv2.dnn.readNetFromONNX` + `blobFromImage`
تتبع	`cv2.createBackgroundSubtractorMOG2` + `TrackerCSRT_create`
أداء	`cv2.getTickCount` / `cv2.setNumThreads`

1.3 ثالثاً: أجوبة مختصرة لتمارين مختارة

- **الفصل 2: البكسلات**

سؤال «قيم القنوات»: القنوات الأحمر/الأخضر/الأزرق من 0 إلى 255؛ وترتيبها في OpenCV **BGR** لا RGB — العكس يبدّل الألوان بصمت.

- **الفصل 4: العمليات النقطية**

«alpha/beta» في `convertScaleAbs` : الناتج = α×قيمة + β؛ استخدام التباين والإضاءة أيضاً في أي صورة؛ و `clip(0, 255)` إجبارية لتجنب الفائض.

- **الفصل 6: الحواف**

«لماذا CV_64F في Sobel؟» — لأن المشتق يعطي قيماً سالبة؛ وإذا حفظنا مباشرة بـ CV_8U فالنصف «السالب» يقطع إلى صفر ويضيع الحواف المرافقة.

- **الفصل 8: العتبات**

«blockSize في adaptiveThreshold» فرد دائماً (3,5,...)؛ والقيمة `C` هامش توجيه العتبة نحو الأبيض أو الأسود.

- **الفصل 10: الكونتورات**

`approxPolyDP` بهامش 0.02 من المحيط: ثلاث نقاط مثلث، أربعة رباعياً، وكثير مع دائرية عالية = شكل دائري.

- **الفصل 13: القوالب**

طرق `SQDIFF` تبحث عن الحد الأدنى (`min_loc`) — قلب النتيجة يعطي أسوأ تطابق بدل أفضلها.

- **الفصل 16: cv2.ml**

بيانات التدريب `np.float32` والتسميات `np.int32` — أغلب «أخطاء التدريب الغامضة» سببها نوع خاطئ.

- **الفصل 17: DNN**

نسيان `swapRB=True` بينما النموذج تدرب على RGB — من أكثر أسباب «انكماش الدقة بصمت» في الكشف.

1.4 رابعاً: مسرد مصطلحات

المصطلح	يُنطَق/الأصل	المعنى بالمشهد العربي
Kernel	كيرنل	النافذة التي تتحرك على الصورة (مرشح/مورفولوجيا)
Mask	ماسك	قناع ثنائي يحدد «أين نطبق العملية»
Threshold	ثريشولد	العتبة: حافة القرار بين «هو» و«ليس»
Contour	كونتور	الخط الخارجي المكتشف لجسم (نقاط حدية)
Keypoint	كيبوينت	«نقطة مميزة» مثبتة الخصائص في الصورة
Descriptor	واصِف	توقيع عددي يصف «كيف تبدو» النقطة للربط
Homography	هوموغرافي	مصفوفة تحول 3×3 بين مستويي صورة
Inference	إنفرينس	«استدلال» النموذج: تشغيله على صورة جديدة
Preprocessing	بربروسيسينغ	تجهيز الصورة قبل النموذج (blob/تحويلات)
NMS	ن.م.س	إزالة التكرار المكاني: دمج الصناديق المتقاطعة
Anchor	أنكر	صندوق مرجعي جاهز في كشف الأجسام
Frame	فريم	إطار واحد من الفيديو
fps	إف.ب.س	عدد الإطارات في الثانية: مقياس السلاسة
Background subtraction	طرح الخلفية	استخراج «ما الذي تحرك» في المشهد
Optical flow	التدفق الضوئي	متجهات حركة النقاط بين إطارين
Calibration	كاليبريشون	معايرة: ربط البكسل بالوحدات الحقيقية
Undistortion	إلغاء التشويه	تصحيح انحناء العدسة
Vectorization	فيكتوريَيشن	الكتابة على «مصفوفة كاملة» بدل حلقات

المصطلح	يُنطَق/الأصل	المعنى بالمشهد العربي
dtype	داتا تايب	نوع عنصر المصفوفة (.../uint8/float32)
ROI	أر.آي	منطقة الاهتمام: قصّة الصورة التي نعالجها

رسالة الختام: الأداة ليست صعبة؛ إنها صورة وبكسل وقواعد قليلة. كل ما تعلمته في هذه الصفحات يدور حول مطلبين: «كيف أصف البكسل بالنسبة لجاره» و«كيف أتعامل مع التغيّر». شكراً لأنك قررت أن تحاور الحاسوب بلسان البكسل — وتحية «نُور» وجميع القراء.